

UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE CIENCIA
DEPARTAMENTO DE MATEMÁTICA
Y CIENCIA DE LA COMPUTACIÓN



Diseño e implementación de plataforma de ciencia de datos para investigación colaborativa

Cristobal Omar Gallardo Cubillos

Vicente Tomás Santos Varas

Profesor guía:

Oscar Francisco Rojas Díaz

Tesis para optar al título de
Analista en Computación Científica

Santiago - Chile

2025

Resumen

El volumen y complejidad de la investigación científica actual exigen infraestructuras para el procesamiento eficiente y colaborativo de datos. En Chile, la carencia de plataformas estandarizadas genera duplicidad de esfuerzos y limita la replicabilidad de resultados. Este trabajo propone el diseño e implementación de una plataforma experimental de ciencia de datos con un enfoque modular y adaptable a diversas necesidades de investigación.

La arquitectura basada en microservicios de la solución emplea componentes independientes organizados por niveles para separar la interfaz, la gestión y el análisis. Este diseño facilita la integración de funciones, simplifica el mantenimiento y permite el despliegue en diversos entornos, incorporando mecanismos para la gestión óptima de datos heterogéneos.

La validación técnica, mediante pruebas de funcionamiento, desempeño y orquestación de microservicios, evidencia la portabilidad y capacidad de escalamiento funcional de la plataforma, lo cual es útil para el desarrollo de investigación colaborativa e interdisciplinar donde se aplique ciencia de datos.

Palabras clave: Microservicios; Interoperabilidad; Ciencia de Datos; Modular; Escalabilidad.

Abstract

The volume and complexity of current scientific research demand infrastructures for efficient and collaborative data processing. In Chile, the lack of standardized platforms leads to a duplication of efforts and limits the replicability of results. This work proposes the design and implementation of an experimental data science platform with a modular approach, adaptable to diverse research needs.

The solution's microservices-based architecture employs independent components organized into layers to decouple the interface, management, and analysis. This design facilitates feature integration, simplifies maintenance, and enables deployment across diverse environments, incorporating mechanisms for the optimal management of heterogeneous data.

Technical validation, conducted through functional, performance, and microservices orchestration testing, demonstrates the platform's portability and functional scalability. These results highlight its utility for developing collaborative and interdisciplinary research where data science is applied.

Keywords: Microservices; Interoperability; Data Science; Modular; Scalability.

Agradecimientos

Agradecemos a nuestro profesor guía, por su tiempo, exigencia y guía técnica, que ayudaron a dar dirección y solidez al trabajo. También agradecemos a quienes participaron probando funcionalidades, entregando retroalimentación y apoyando en decisiones de implementación. Este trabajo es el resultado de muchas conversaciones, iteraciones y aprendizaje acumulado.

Tabla de contenido

1	Introducción	1
1.1	Definición del problema	1
1.2	Solución propuesta	2
1.3	Objetivo general	3
1.3.1	Objetivos específicos	3
1.4	Alcances y limitaciones	3
1.4.1	Alcances	4
1.4.2	Limitaciones	4
1.5	Metodología	4
1.6	Hardware y software	5
1.6.1	Hardware	5
1.6.2	Software	6
1.7	Organización del documento	7
2	Marco Teórico	8
2.1	Ciencia de Datos	8
2.1.1	Definición	8
2.1.2	Big Data	9
2.1.3	Principios FAIR y reproducibilidad	10
2.1.4	Gobernanza: Identidad y Linaje de Datos	11
2.1.5	Diseño de plataformas de ciencia de datos	12
2.2	Arquitectura de plataformas de ciencia de datos	13
2.2.1	Comunicación síncrona	14

2.2.2	Comunicación asincrónica	14
2.2.3	Patrón API Gateway	15
2.2.4	Persistencia Políglota	17
2.2.5	Contenedorización y empaquetamiento de software	17
2.2.6	Arquitectura Cliente-Servidor	18
2.2.7	Componentes técnicos y ciclo de vida	19
2.2.7.1	Ingesta, procesamiento distribuido y orquestación	19
2.2.7.2	Gestión de Modelos: Reproducibilidad Mínima Viable	20
2.2.7.3	Despliegue y Motores de Inferencia	21
2.3	Inteligencia Artificial y Ciencia de Datos	23
2.3.1	Machine Learning	24
2.3.2	Métricas de Evaluación	27
2.3.2.1	Evaluación de Predicción (Regresión)	27
2.3.2.2	Evaluación de Clasificación	28
2.3.2.3	Evaluación de Agrupamiento (Clustering)	29
2.3.3	Consideraciones éticas e interpretabilidad	30
2.4	Ingeniería de software	31
2.4.1	Metodologías ágiles: Scrum y UML	31
2.4.2	Patrones de arquitectura: MVC y EDA	33
2.4.3	MLOps y prácticas de despliegue continuo	35
2.5	Calidad de software y evaluación de rendimiento	36
2.5.1	Estrategias de QA	36
2.5.2	Métricas de rendimiento	37
2.5.3	Herramientas de evaluación y observabilidad	39
2.6	Conclusiones del capítulo	40
3	Diseño de propuesta de software	42
3.1	Análisis de requerimientos	42
3.2	Diseño de software	44
3.2.1	Casos de uso	44

3.2.2	Arquitectura de Plataforma	46
3.2.3	Arquitectura de microservicios	49
3.2.3.1	Diseño de Frontend	50
3.2.3.2	Diseño de Backend	55
3.2.3.3	Diseño de servicios de ML	56
3.2.4	Arquitectura de Despliegue	58
3.3	Plan de Pruebas y QA	60
3.3.1	Pruebas de eficacia	60
3.3.2	Pruebas de eficiencia	61
3.3.3	Pruebas de trazabilidad	61
3.4	Modelo de ingeniería de software	62
3.5	Conclusiones del capítulo	63
4	Desarrollo, Implementación y Despliegue	64
4.1	Herramientas de desarrollo	64
4.2	Desarrollo e implementación	65
4.2.1	Desarrollo de Frontend	65
4.2.1.1	Panel de control de acceso (RBAC)	65
4.2.1.2	Panel de gestión de experimentos	65
4.2.1.3	Interfaz de entrenamiento de ML	67
4.2.1.4	Interfaz de inferencia	70
4.2.1.5	Interfaz de seguimiento	72
4.2.2	Desarrollo de Backend	73
4.2.2.1	Desarrollo del Orquestador (API Gateway)	73
4.2.2.2	Desarrollo de Bases de Datos	74
4.2.3	Desarrollo de servicios de ML	78
4.2.3.1	Microservicio de Ingesta	78
4.2.3.2	Microservicio de Preprocesamiento	79
4.2.3.3	Microservicio de Clasificación	79
4.2.3.4	Microservicio de Predicción	79

4.2.3.5	Microservicio de Clustering	80
4.2.3.6	Microservicio de Inferencia	80
4.2.4	Implementación de infraestructura	80
4.3	Empaquetamiento y despliegue	83
4.3.1	Empaquetamiento	83
4.3.2	Despliegue	83
4.4	Conclusiones del capítulo	84
5	Resultados	85
5.1	Resultados de pruebas de eficacia	85
5.1.1	Eficacia de los servicios de Machine Learning	85
5.1.2	Análisis de correspondencia e integridad del dato	86
5.2	Resultados de pruebas de eficiencia	87
5.2.1	Entorno experimental y escenarios evaluados	87
5.2.2	Resultados de rendimiento y latencia	88
5.2.3	Resistencia a la carga concurrente	88
5.2.3.1	Evaluación de carga en VPS de producción.	89
5.3	Resultados de pruebas de usabilidad y validación funcional	90
5.3.1	Pruebas de componentes lógicos (Unitarias e Integración)	91
5.3.2	Pruebas End-to-End (E2E) y experiencia de usuario	92
5.3.3	Evaluación de QA y análisis estático	92
5.3.3.1	Análisis estático de código (ESLint)	92
5.3.3.2	Auditoría de dependencias (npm audit)	93
5.4	Resultados de pruebas de trazabilidad y gobernanza	94
5.4.1	Validación de persistencia políglota y linaje de datos	94
5.5	Conclusiones del capítulo	94
	Conclusiones	96
	Trabajos Futuros	97

Referencias	99
Anexo A. Fundamentos Matemáticos de Algoritmos de Aprendizaje Automático	112
Anexo B. Especificación Casos de Uso	115
Anexo C. Infraestructura y Orquestación de Contenedores	119
C.1 Empaquetamiento de Imágenes (Dockerfiles)	119
C.1.1 Microservicios de Procesamiento Analítico	119
C.1.2 Backend Gateway	120
C.1.3 Frontend y Estrategia de Construcción	120
C.2 Orquestación y Levantamiento con Docker Compose	120
C.3 Guía de Despliegue de la Plataforma	126
C.3.1 Requisitos Previos	127
C.3.2 Paso 1: Obtención del Código Fuente	127
C.3.3 Paso 2: Configuración del Entorno (.env)	127
C.3.4 Paso 3: Ejecución de la Plataforma	128
C.3.5 Paso 4: Verificación del Despliegue	128
Anexo D. Configuración y Seguridad del Sistema	129
D.1 Parámetros del Entorno	129
Anexo E. Especificaciones Técnicas de los Microservicios Analíticos	131
E.1 Protocolo de Comunicación en Tiempo Real (Server-Sent Events) . . .	131
E.1.1 Estructura de Payloads (Mensajes JSON)	131
E.2 Especificación de Métricas Analíticas	132
E.3 Persistencia de Resultados de Inferencia (Auditoría)	132
E.3.1 Estructura del Documento de Inferencia	133
E.4 Estructura del Artefacto de Persistencia (.joblib)	133
E.4.1 Componentes del Artefacto	133
E.5 Organización del Almacenamiento y Volúmenes	134

E.5.1	Convención de Nomenclatura y Trazabilidad	134
Anexo F. Contratos de la API REST (API Gateway)		135
F.1	Convenciones Generales	135
F.2	Autenticación y Gestión de Usuarios	136
F.3	Ingesta y Gestión de Datos	136
F.4	Preprocesamiento de Datos	136
F.5	Entrenamiento e Inferencia de Modelos	137
F.6	Monitorización de Infraestructura	138
F.7	Historial y Trazabilidad de Experimentos	138
F.8	Ejemplos de Respuestas JSON (Contratos de Salida)	139
F.8.1	Autenticación y Sesión	139
F.8.2	Gestión de Ingesta y Estadísticas	140
F.8.3	Monitorización de Microservicios	140
F.8.4	Historial de Entrenamientos y Trazabilidad	141
F.8.5	Canal de Streaming (Server-Sent Events)	142
Anexo G. Esquema Relacional y Estructura de Persistencia		143
G.1	Script de Creación de Objetos (DDL)	143
G.2	Estructura del Volumen de Modelos (models_storage)	147
Anexo H. Manual de Usuario		148
H.1	Introducción	148
H.2	Acceso al sistema	148
H.3	Flujo de Operación	149
H.3.1	Ingesta de Datos	149
H.3.2	Preprocesamiento y Selección de Variables	149
H.3.3	Entrenamiento de Modelos	150
H.3.4	Registro y Serialización del Modelo	151
H.4	Gestión de Activos y Auditoría	152
H.4.1	Data Lake Manager (Gestión de Archivos)	152

H.4.2	Historial de Entrenamientos	152
H.4.3	Historial de Inferencias	153
H.5	Ejecución de Inferencia	154
Anexo I. Despliegue en Entorno de Producción (VPS)		156
I.1	Configuración del Proxy Inverso (Nginx)	156
I.2	Monitorización de Infraestructura (Docker Socket)	157
I.3	Comandos de Operación y Mantenimiento	158
I.3.1	Construcción y Despliegue Inicial	159
I.3.2	Actualización en Caliente de Microservicios	159
I.3.3	Reinicio Forzado y Depuración	159

Índice de tablas

Tabla 1	Comparativa de modelos de estrategias de aseguramiento de calidad.	37
Tabla 2	Casos de uso.	46
Tabla 3	Contratos de interfaz RESTful del sistema.	73
Tabla 4	Diccionario de entidades.	77
Tabla 5	Diccionario de colecciones.	78
Tabla 6	Comparativa de resultados de eficacia para Clasificación.	86
Tabla 7	Comparativa de resultados de eficacia para Predicción.	86
Tabla 8	Rendimiento consolidado con 10 usuarios concurrentes.	89
Tabla 9	Rendimiento consolidado con 25 usuarios concurrentes.	89
Tabla 10	Rendimiento consolidado con 50 usuarios concurrentes.	90
Tabla 11	Rendimiento consolidado con 100 usuarios concurrentes.	90
Tabla 12	Rendimiento consolidado en VPS de producción para E3.	91
Tabla 13	Rendimiento consolidado en VPS de producción para E4.	91
Tabla 14	Resultados de pruebas de experiencia de usuario (End-to-End).	92
Tabla 15	Especificación: CU-01 Gestión de Usuarios.	115
Tabla 16	Especificación: CU-02 Ingesta de Datos.	115
Tabla 17	Especificación: CU-03 Preprocesamiento de Datos.	116
Tabla 18	Especificación: CU-04 Experimentación Visual.	116
Tabla 19	Especificación: CU-05 MLOps y Repertorio de Modelos.	116
Tabla 20	Especificación: CU-06 Ejecución de Inferencia.	117

Tabla 21	Especificación: CU-07 Exportación de Reportes.	117
Tabla 22	Especificación: CU-08 Historial de Inferencias.	117
Tabla 23	Especificación: CU-09 Historial de Entrenamientos.	118
Tabla 24	Resumen de variables de entorno configuradas.	129
Tabla 25	Métricas por tipo de tarea analítica e implementación.	132
Tabla 26	Endpoints de autenticación y administración de usuarios.	136
Tabla 27	Endpoints del módulo de ingesta de datos.	137
Tabla 28	Endpoints del módulo de preprocesamiento.	137
Tabla 29	Endpoints asociados al ciclo de vida de modelos.	138
Tabla 30	Endpoints de monitorización de infraestructura.	138
Tabla 31	Endpoints de trazabilidad y recuperación de resultados.	139

Índice de figuras

Figura 1	Patrón de arquitectura de microservicios (Richards, 2015).	13
Figura 2	Comparativa de patrones de acceso a microservicios. Modificada de (Richardson, 2018).	16
Figura 3	Arquitectura de tres niveles (S. Kumar, 2019).	18
Figura 4	Flujo de trabajo Scrum. (Elaboración propia)	32
Figura 5	Diagramas UML 2.0. Modificado de Sommerville (2011)	32
Figura 6	Arquitectura Modelo-Vista-Controlador (MVC) (M. Kumar & Choppella, 2024).	34
Figura 7	Arquitectura dirigida por eventos (Kreps et al., 2011).	34
Figura 8	Diagrama de casos de uso del sistema.	45
Figura 9	Diagrama general de Arquitectura de plataforma.	47
Figura 10	Arquitectura de componentes de servidor.	48
Figura 11	Arquitectura de distribución de microservicios.	49
Figura 12	Arquitectura funcional de microservicios.	50
Figura 13	Diseño de Layout.	51
Figura 14	Diseño del panel principal del investigador.	51
Figura 15	Diseño del módulo de ingesta de datos.	52
Figura 16	Diseño del módulo de Entrenamiento.	52
Figura 17	Diseño de ejecución de inferencias.	53
Figura 18	Diseño del historial de entrenamientos y búsqueda.	54
Figura 19	Diseño del historial de inferencias.	54
Figura 20	Pipeline de servicios de procesamiento, entrenamiento e inferencia.	58

Figura 21	Arquitectura de despliegue de la plataforma.	59
Figura 22	Carta Gantt.	63
Figura 23	Interfaz de autenticación y control de acceso (RBAC).	66
Figura 24	Panel de control principal del investigador.	67
Figura 25	Interfaz funcional de ingesta de datos.	67
Figura 26	Interfaz funcional de preprocesamiento de datos.	68
Figura 27	Componentes de experimentación y entrenamiento.	69
Figura 28	Interfaces de ejecución de inferencias.	71
Figura 29	Interfaz del historial de entrenamientos.	72
Figura 30	Interfaz de auditoría de inferencias.	72
Figura 31	Esquema físico de la base de datos relacional.	75
Figura 32	Estructura documental de colecciones en MongoDB.	76
Figura 33	Diagrama de secuencia UML del pipeline transaccional.	82
Figura 34	Pantalla de inicio de sesión de D-Science.	148
Figura 35	Confirmación de carga exitosa y resumen del dataset.	149
Figura 36	Selección de variables y objetivo (Target).	150
Figura 37	Preprocesamiento exitoso.	150
Figura 38	Métricas de desempeño post-entrenamiento.	151
Figura 39	Modal de registro de modelo analítico.	151
Figura 40	Confirmación de versionado de modelo.	151
Figura 41	Data Lake Manager para la gestión centralizada de archivos. . .	152
Figura 42	Historial detallado de modelos entrenados y auditoría de parámetros.	153
Figura 43	Historial y descarga de inferencias.	153
Figura 44	Panel principal de inferencia.	154
Figura 45	Vista de predicción inferencia.	154
Figura 46	Resultados finales de la inferencia con opción de exportación. .	155

Capítulo 1. Introducción

En el último tiempo, la ciencia de datos ha tomado un gran rol en distintas áreas de la ciencia e investigación, generando altas expectativas de cómo esta impactaría en la investigación científica y tecnológica. A nivel internacional ya existen iniciativas, tales como la *Precision Medicine Initiative*¹ o el proyecto *The Cancer Genome Atlas*² (TCGA), las cuales han demostrado el potencial de analizar grandes volúmenes de datos en contextos complejos, así impulsando infraestructuras de *Big Data*³ para la investigación.

Sin embargo, en Chile el aprovechamiento de estos avances continúa siendo limitado. Si bien existen iniciativas como *Dash-AI*⁴, estas se centran primordialmente en la etapa de modelado, dejando de lado aspectos críticos de la ciencia de datos como la gobernanza y la gestión integral de los datos. Esta brecha entre el ámbito académico y la aplicación práctica genera la oportunidad de diseñar una plataforma tecnológica que facilite la integración, gestión y análisis de la información, impulsando así la investigación interdisciplinar y el uso efectivo de la ciencia de datos.

1.1. Definición del problema

Actualmente se evidencia la falta de una plataforma tecnológica integral, modular y escalable que permita gestionar grandes volúmenes de datos en proyectos científicos de manera eficiente y colaborativa (National Institutes of Health, 2025). Esto conlleva diversas problemáticas específicas:

¹Iniciativa de medicina de precisión lanzada en EE. UU. (Collins & Varmus, 2015)

²Proyecto de caracterización genómica (The Cancer Genome Atlas Research Network, 2013).

³Sistemas para la gestión y análisis distribuido de grandes conjuntos de *datos heterogéneos*

⁴Plataforma para el entrenamiento automatizado de modelos de IA (Abarca et al., 2026).

- **Esfuerzos:** Cada equipo de investigación debe crear sus propias plataformas desde cero, invirtiendo tiempo y recursos en tareas básicas de configuración e integración tecnológica, lo que genera duplicación de esfuerzos y reduce la eficiencia investigativa (Martínez et al., 2021).
- **Modularidad:** Las plataformas disponibles no cuentan con un diseño modular efectivo, haciendo compleja la incorporación flexible y ágil de nuevas tecnologías como inteligencia artificial, procesamiento avanzado o herramientas de análisis de datos (Fritzsche et al., 2019).
- **Despliegue:** La falta de un protocolo claro y estandarizado para desplegar los componentes analíticos confiables en diferentes contextos limita el impacto potencial de la investigación, pues reduce la reproducibilidad y portabilidad de las soluciones (Boettiger, 2015).
- **Experiencia de usuario:** Las plataformas existentes generalmente no priorizan una interfaz intuitiva ni amigable para los investigadores y usuarios finales, lo cual afecta negativamente el uso y la productividad (McPadden et al., 2019).

Estos problemas mencionados dificultan no solo la generación de nuevos conocimientos basados en los datos, sino también la difusión, replicabilidad y aplicación efectiva de la experimentación. Frente a esta realidad, aparece la necesidad de realizar el desarrollo de una solución tecnológica integral que permita superar estos problemas, potenciando así la capacidad investigativa del país mediante el aprovechamiento eficiente y colaborativo de grandes volúmenes de datos.

1.2. Solución propuesta

Se propone el desarrollo de una plataforma modular de ciencia de datos basada en *microservicios*⁵ y contenedores *Docker*⁶. El sistema integrará una interfaz intuitiva

⁵Arquitectura de servicios autónomos y desacoplados que interactúan mediante protocolos de red ligeros.

⁶Software de computación para el despliegue de aplicaciones en contenedores (Merkel, 2014).

para abstraer la complejidad técnica a los investigadores, siguiendo metodologías de ingeniería de software (diagramas *UML*⁷ y pruebas de escalabilidad) que garantizarán su mantenibilidad. La contenedorización permitirá un despliegue ágil tanto en entornos locales como en la nube.

1.3. Objetivo general

Diseñar una plataforma experimental basada en ciencia de datos para la gestión oportuna y pertinente de la investigación colaborativa e interdisciplinar, con enfoque en un diseño arquitectónico modular y escalable.

1.3.1. Objetivos específicos

- Analizar el estado del arte y definir lineamientos de ingeniería de software para el diseño de una plataforma modular con microservicios.
- Diseñar e implementar una arquitectura modular para la plataforma mediante la configuración de contenedores Docker para cada módulo.
- Integrar los módulos de la plataforma, utilizando interfaces basadas en microservicios, protocolos de comunicación estándar y herramientas de gestión de datos.
- Realizar pruebas para evaluar el rendimiento y escalabilidad funcional de la implementación, mediante pruebas unitarias y pruebas de integración.

1.4. Alcances y limitaciones

Para el desarrollo del proyecto se consideran los siguientes alcances y limitaciones.

⁷Estándar visual para la modelación, especificación y documentación de arquitecturas de software.

1.4.1. Alcances

- El software desarrollado quedará disponible como una herramienta de libre acceso bajo la *licencia MIT*⁸, permitiendo que cualquier entidad interesada pueda utilizarlo, adaptarlo o distribuirlo según sus propios fines.
- El prototipo de plataforma se integrará al “Ecosistema de Inteligencia Artificial Sostenible y Escalable” del proyecto de investigación de frontera FIU137139-USACH, el cual tiene como eje estratégico el desarrollo de herramientas tecnológicas que fomenten la investigación colaborativa e interdisciplinaria mediante la aplicación de ciencia de datos e inteligencia artificial.

1.4.2. Limitaciones

- El procesamiento está acotado a la memoria RAM y capacidad de un servidor único, no se implementan clústeres de computación distribuida (*Big Data*)
- El sistema no es una herramienta de *AutoML*⁹. El ajuste de parámetros y la selección de algoritmos son responsabilidad manual del usuario.

1.5. Metodología

La estrategia metodológica de este trabajo integra pilares de investigación aplicada, ingeniería de software y experimentación técnica. Este enfoque multidisciplinario permite abordar de forma sistemática los desafíos técnicos y funcionales detallados en los objetivos específicos de la Sección 1.3.1.

Para dar cumplimiento al primer objetivo, se emplea una metodología de carácter exploratorio y documental. Esta fase se centra en la revisión sistemática de literatura científica y estándares sobre orquestación de contenedores y arquitecturas de microservicios. Mediante el análisis comparativo de las mejores prácticas actuales, se

⁸Licencia permisiva que autoriza el uso, modificación y distribución gratuita del código fuente.

⁹Sistemas de optimización y selección autónoma de modelos e hiperparámetros.

busca establecer un marco normativo y técnico que guíe el diseño de infraestructuras optimizadas para entornos de investigación.

El segundo y tercer objetivo se abordan bajo una metodología de modelado y diseño de sistemas. Se utilizarán herramientas de ingeniería de software, como el lenguaje de modelado unificado (UML), para estructurar la lógica de los contenedores y definir los protocolos de comunicación.

Finalmente, el cuarto objetivo se rige por una metodología de desarrollo basado en prototipos y experimentación controlada. Se implementará un prototipo funcional de la plataforma que será sometido a ciclos iterativos de pruebas unitarias y de integración. Este proceso de validación técnica permitirá medir empíricamente la escalabilidad funcional y el rendimiento del sistema frente a cargas de datos masivos.

1.6. Hardware y software

1.6.1. Hardware

Para cubrir las distintas fases del ciclo de vida del proyecto, se utilizaron tres configuraciones de hardware:

- **Hardware de experimentación y administración:** Equipo equipado con un procesador Intel Core i7, 32 GB de memoria RAM y una unidad de procesamiento gráfico (GPU) NVIDIA RTX 4060 Ti. Este entorno se destinó a la experimentación base de *aprendizaje automático*, la práctica de implementación de microservicios y las tareas de administración de servidores.
- **Hardware de desarrollo de software:** Equipo de escritorio con un procesador AMD Ryzen 5 5600X, 16 GB de memoria RAM DDR4 y una GPU NVIDIA GeForce GTX 1060 con 6 GB de memoria dedicada. Se utilizó para el desarrollo del código fuente, el análisis exploratorio de datos y la implementación de microservicios de baja y media demanda.

- **Hardware de producción y despliegue:** Servidor Privado Virtual (VPS) con procesador Intel Core i9 de 8 núcleos, 64 GB de memoria RAM DDR5 y 200 GB de almacenamiento. Este entorno hospeda la plataforma en el subdominio <https://dscience.cedetec.cl/>, perteneciente al Centro de Desarrollo, Experimentación y Transferencia de Tecnología Educativa (CEDETEC)¹⁰.

1.6.2. Software

El ecosistema de software se seleccionó para asegurar el aislamiento de procesos y la mantenibilidad del sistema modular:

- **Herramientas de desarrollo:** Para la escritura y control del código se utilizó *Visual Studio Code*¹¹ y *Git*¹², gestionando la colaboración mediante el servicio de *GitHub*¹³. Para el empaquetamiento y orquestación de la arquitectura se empleó *Docker*, permitiendo que cada microservicio opere de forma aislada y reproducible.
- **Lenguajes de programación:** La interfaz se construyó con *JavaScript*¹⁴ utilizando la librería *React*¹⁵ bajo el entorno *Node.js*¹⁶. La lógica de procesamiento y gestión de datos se desarrolló en *Python*¹⁷, seleccionado por su alta eficiencia en tareas de computación científica.
- **Análisis de calidad de software:** Se utilizó *Jest*¹⁸ para pruebas unitarias y *Postman*¹⁹ para la validación de los contratos de comunicación entre servicios. Finalmente, el rendimiento y la escalabilidad funcional de la solución se evaluaron bajo condiciones de estrés mediante *Apache JMeter*²⁰.

¹⁰Unidad de investigación y desarrollo de la Universidad de Santiago de Chile enfocada en la gestión del conocimiento e ingeniería de software.

¹¹*Software* de edición de código fuente con soporte para múltiples lenguajes y depuración.

¹²*Software* de control de versiones para el registro histórico y gestión de cambios.

¹³Plataforma en la nube para el alojamiento y gestión del *repositorio* central del proyecto.

¹⁴Lenguaje interpretado y multiparadigma utilizado para el desarrollo de aplicaciones web.

¹⁵Biblioteca encargada de la creación y renderizado eficiente de interfaces de usuario.

¹⁶Entorno de ejecución de capa lógica basado en el motor de *JavaScript* V8.

¹⁷Lenguaje orientado al análisis masivo de datos y la computación científica.

¹⁸*Framework* para la validación automática y ejecución de pruebas unitarias de código.

¹⁹Herramienta para la depuración y consumo de interfaces *API* y protocolos de red.

²⁰*Software* para la evaluación de métricas de rendimiento y carga crítica del sistema.

1.7. Organización del documento

La organización del documento es de la siguiente manera. El Capítulo 2 presenta el marco teórico y el estado del arte sobre microservicios, ciencia de datos y tecnologías de contenedores. El Capítulo 3 describe el diseño arquitectónico de la propuesta, detallando la estructura modular y la interacción entre servicios. El Capítulo 4 detalla el proceso de implementación del prototipo funcional, el empaquetamiento en Docker y las pruebas de integración. El Capítulo 5 expone los resultados de las evaluaciones de rendimiento, carga y escalabilidad de la plataforma. Finalmente, en el Capítulo 5.5 se presentan las conclusiones del trabajo y las proyecciones futuras.

Capítulo 2. Marco Teórico

En este capítulo se presentan los fundamentos conceptuales y tecnológicos que respaldan el diseño y validación de la plataforma propuesta. La Sección 2.1 aborda los fundamentos de la ciencia de datos, Big Data y los principios FAIR. La Sección 2.1.5 detalla el diseño de arquitecturas basadas en microservicios. La Sección 2.3 explora la integración investigativa y ética de la inteligencia artificial. La Sección 2.4 presenta las prácticas de ingeniería de software y *MLOps*¹ para sistemas escalables. La Sección 2.5 define las estrategias de aseguramiento de calidad del software y evaluación de rendimiento. El capítulo finaliza con una conclusión del marco teórico.

2.1. Ciencia de Datos

La ciencia de datos se ha consolidado como un eje fundamental para la toma de decisiones basada en evidencia, impulsada por el acceso masivo a información y los avances en capacidad de cómputo (Dhar, 2013). Su aplicación es crítica para abordar desafíos complejos, especialmente en salud pública, donde facilita diagnósticos precisos y modelos predictivos (Bates et al., 2014).

2.1.1. Definición

La ciencia de datos se define como la disciplina que estudia la extracción sistemática de conocimiento y valor a partir de conjuntos de datos heterogéneos, integrando estadística, informática, *aprendizaje automático*² (o *Machine Learning*) y la experiencia

¹Paradigma que integra aprendizaje automático, ingeniería de software y de datos.

²Algoritmos que permiten a los sistemas aprender patrones a partir de los datos.

*de dominio*³ (Candela et al., 2020). Este carácter interdisciplinario permite abordar problemas complejos y fundamentar la toma de decisiones al incorporar habilidades computacionales que trascienden la estadística tradicional (Cao, 2017; Cleveland, 2001). Su núcleo operativo se basa en un ciclo de vida iterativo que comprende las etapas de recolección, limpieza, análisis, interpretación y comunicación de resultados. En la práctica, este proceso transforma los *datos crudos*⁴ en conocimiento útil mediante el uso de lenguajes como *Python*⁵ o *R*⁶ y librerías como *Pandas*⁷ y *Scikit-Learn*⁸. Esto incluye la implementación de técnicas de preprocesamiento, tales como *StandardScaler*⁹ y *Label Encoder*¹⁰, junto con algoritmos de aprendizaje automático. La disciplina enfatiza la creación de flujos de trabajo reproducibles a través de *notebooks*¹¹ y *pipelines*¹² que aseguran decisiones informadas, replicables y sostenibles (Provost y Fawcett, 2013).

2.1.2. Big Data

El concepto de Big Data hace referencia al paradigma tecnológico orientado al procesamiento de datos que exceden las capacidades de los sistemas de gestión de bases de datos relacionales y las arquitecturas de almacenamiento centralizadas (Jagadish et al., 2014). Este paradigma se define mediante tres dimensiones críticas que incluyen un volumen masivo de registros junto a una alta velocidad referida al flujo continuo de generación e ingesta de datos en tiempo real. También se integra una amplia variedad de fuentes y formatos que abarcan desde datos estructurados hasta documentos no estructurados como imágenes o señales de sensores (Gandomi y Haider, 2015). En el ámbito de la investigación científica la integración de *datos heterogéneos* permite generar conocimiento *accionable*¹³ (Shilo et al., 2020). Abordar

³Conocimiento especializado sobre el contexto y la problemática específica del área de aplicación.

⁴Información en su estado original que no ha sido procesada o analizada.

⁵Lenguaje de programación de alto nivel con amplia adopción en la ciencia de datos.

⁶Entorno y lenguaje especializado en computación estadística y gráficos.

⁷Biblioteca que proporciona estructuras de datos y herramientas de análisis para Python.

⁸Librería de aprendizaje automático con herramientas para modelado predictivo y minería de datos.

⁹Método de normalización que escala los datos para que tengan media cero y varianza unitaria.

¹⁰Técnica de codificación que convierte etiquetas categóricas en valores numéricos.

¹¹Entornos interactivos que integran código, texto y visualizaciones.

¹²Secuencias automatizadas de pasos para el procesamiento y modelado de datos.

¹³Información procesada que permite fundamentar la toma de decisiones o acciones concretas.

este desafío técnico exige garantizar la *interoperabilidad*¹⁴ entre sistemas mediante el uso de arquitecturas distribuidas (Demchenko et al., 2013). El manejo eficiente de esta complejidad informacional requiere además la implementación de clústeres de procesamiento paralelo (Dash et al., 2019).

2.1.3. Principios FAIR y reproducibilidad

Los principios FAIR: *Findable*, *Accessible*, *Interoperable*, *Reusable* (Encontrable, Accesible, Interoperable y Reutilizable) postulan los lineamientos fundamentales para asegurar que los datos puedan ser localizados, accedidos, integrados y reutilizados tanto por investigadores humanos como por sistemas tecnológicos (Wilkinson et al., 2016).

Un ejemplo de aplicación es el proyecto europeo *FAIR4Health*¹⁵, que demostró la efectividad de implementar estos principios en entornos clínicos mediante un flujo de trabajo FAIR para datos sanitarios y el uso de estándares como *HL7 FHIR*¹⁶. Esta iniciativa permitió desarrollar herramientas para anonimizar, estructurar y reparar datos provenientes de diversas instituciones de salud en Europa, habilitando análisis colaborativos respetuosos de la privacidad mediante algoritmos de aprendizaje automático (Alvarez-Romero et al., 2022). Por otra parte, el proyecto estadounidense *NIH Data Commons*¹⁷ ha promovido el desarrollo de plataformas abiertas de investigación financiadas con fondos públicos, incentivando la interoperabilidad de los datos biomédicos y el cumplimiento de buenas prácticas para garantizar la reproducibilidad científica (NIH Data Commons Pilot Phase Consortium, 2017).

El concepto de reproducibilidad corresponde a la capacidad de obtener resultados consistentes al emplear los mismos datos de entrada junto a los *métodos*

¹⁴Capacidad de diversos sistemas para intercambiar información y utilizarla de forma conjunta.

¹⁵Iniciativa europea que aplica principios FAIR sobre datos de salud para la investigación clínica.

¹⁶Estándar internacional para facilitar el intercambio electrónico de información de salud.

¹⁷Ecosistema para almacenamiento y compartición de datos biomédicos bajo ciencia abierta.

*computacionales*¹⁸ originales (Peng, 2011), lo cual es esencial para asegurar la transparencia y la validación de la investigación científica moderna (National Academies of Sciences, Engineering, and Medicine, 2019). El cumplimiento de este estándar técnico requiere el acceso total al código fuente junto a las condiciones de análisis iniciales (Del Pup y Atzori, 2024). Bajo este enfoque un experimento permite a agentes externos *replicar*¹⁹ el procedimiento de forma exacta (Del Pup y Atzori, 2024). Esta validación técnica depende de la disponibilidad de los datos utilizados durante el estudio (Peng, 2011).

2.1.4. Gobernanza: Identidad y Linaje de Datos

En entornos de investigación y ciencia de datos aplicada, no basta con procesar la información de manera eficiente, es estrictamente necesario garantizar la trazabilidad, la auditoría y la confiabilidad de los resultados. Para lograr esto, las plataformas modernas deben implementar mecanismos robustos de gobernanza, sostenidos principalmente por dos pilares: la identidad de los datos y su linaje (Alhassan et al., 2016).

- **Identidad de datos:** Se refiere a la capacidad de asignar identificadores únicos, inmutables y contextuales a cada conjunto de información que ingresa al sistema. Esto permite asociar inequívocamente un archivo o registro con su propietario (investigador o institución), sus permisos de acceso y su origen (Kahn & Wilensky, 2006). En *arquitecturas distribuidas*²⁰, mantener una identidad clara es fundamental para aplicar políticas de seguridad, como el *Control de Acceso Basado en Roles*²¹ (RBAC), y garantizar el cumplimiento normativo en el manejo de datos sensibles (Plotkin, 2020).
- **Linaje de datos (Data Lineage):** Define el ciclo de vida completo de la información, registrando su origen, sus movimientos y, especialmente, las transformaciones por las que atraviesa a lo largo del tiempo (Bose & Frew,

¹⁸Algoritmos y procedimientos lógicos ejecutados por un sistema informático.

¹⁹Repetición de un proceso experimental para verificar la consistencia de los resultados.

²⁰Arquitectura de nodos interconectados que operan como una única entidad lógica.

²¹Método de seguridad que restringe el acceso al sistema según la función específica del usuario.

2005). En el contexto del *Machine Learning*, el linaje permite documentar qué conjunto de datos crudo generó un *dataset*²² procesado (tras etapas de limpieza o vectorización), y qué versión específica de ese *dataset* fue utilizada para *entrenar un modelo*²³ determinado (Vartak et al., 2016).

La correcta implementación de la identidad y el linaje favorece directamente la reproducibilidad científica (alineándose con los principios FAIR). Si un modelo algorítmico arroja un resultado anómalo en *producción*²⁴, el linaje de datos permite a los investigadores auditar el flujo en retrospectiva, identificando si el sesgo proviene de los *parámetros del modelo*²⁵, del procesamiento intermedio o del documento original subyacente (Bose & Frew, 2005).

2.1.5. Diseño de plataformas de ciencia de datos

El diseño de una plataforma de ciencia de datos busca integrar componentes tecnológicos que gestionen integralmente el ciclo de vida de la información, desde su captura hasta la generación de conocimiento estratégico.

Bajo esta premisa, una plataforma de ciencia de datos no solo debe responder a volúmenes crecientes de datos, sino también democratizar el acceso para investigadores no especializados y potenciar la colaboración interdisciplinaria (Candela et al., 2020). Este enfoque ha sido validado por implementaciones como la plataforma *beHEALTHIER*²⁶, que utiliza microservicios para garantizar la interoperabilidad sanitaria (Mavrogiorgou et al., 2021).

²²Colección estructurada de datos empíricos o sintéticos en formato tabular o matricial.

²³Fase donde el modelo ajusta parámetros con ejemplos para generalizar a datos nuevos.

²⁴Entorno operativo final donde el sistema está disponible para su uso real por los usuarios.

²⁵Variables internas que se ajustan durante el entrenamiento del algoritmo.

²⁶Plataforma de infraestructura Big Data orientada a la investigación médica.

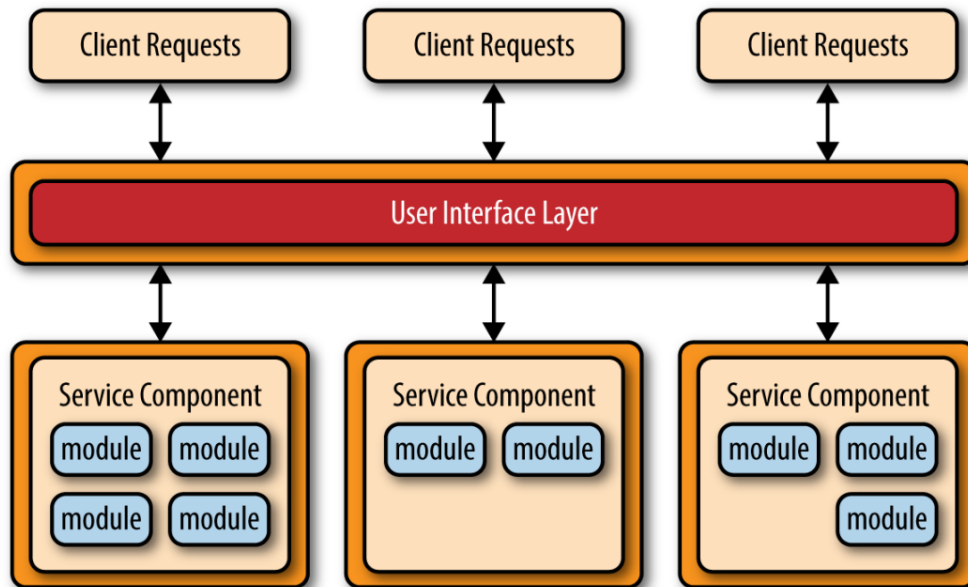


Figura 1. Patrón de arquitectura de microservicios (Richards, 2015).

2.2. Arquitectura de plataformas de ciencia de datos

Una arquitectura basada en microservicios organiza las aplicaciones como un conjunto de *servicios autónomos*²⁷, en contraste con los sistemas monolíticos donde todos los módulos comparten recursos en una única unidad desplegable, lo cual suele limitar la escalabilidad horizontal (Fritzsche et al., 2019). En este paradigma, la modularidad se expresa de manera explícita: cada servicio se comporta como un componente independiente con su propio ciclo de vida, implementando principios fundamentales de la ingeniería de software como el de responsabilidad única (*Single Responsibility Principle*, SRP²⁸) (Dragoni et al., 2017).

Como se ilustra en la Figura 1, este patrón se organiza en componentes de servicio desacoplados que interactúan mediante una capa de interfaz de usuario, lo que facilita el aislamiento funcional de sus módulos internos (Richards, 2015).

²⁷Unidad funcional independiente con lógica, datos y ciclo de vida propios, desplegable y escalable.

²⁸Principio que establece que cada componente debe tener una única responsabilidad.

2.2.1. Comunicación síncrona

En arquitecturas distribuidas, y particularmente durante las fases de validación de nuevos diseños, los microservicios suelen implementar comunicación síncrona mediante interfaces *RESTful*²⁹. Este modelo utiliza *HTTP*³⁰ como protocolo estándar para exponer funcionalidades mediante operaciones definidas semánticamente (Fielding, 2000). La transferencia de información se realiza habitualmente en formatos ligeros y estandarizados como *JSON*³¹ logrando un desacoplamiento explícito entre el cliente y el servidor. Al establecer las especificaciones de interfaz desde la concepción del sistema, la arquitectura queda estructurada para soportar transiciones futuras, asegurando que la lógica interna de un contenedor pueda evolucionar sin alterar a los clientes o servicios que la consumen (Pautasso et al., 2017).

2.2.2. Comunicación asíncrona

Mientras que el estándar *REST*³² resulta altamente eficiente para cargas de investigación controladas, la escala industrial en plataformas de ciencia de datos requiere gestionar la contrapresión (*backpressure*³³) generada por grandes volúmenes de información asegurando la estabilidad del sistema ante fluctuaciones en la tasa de ingesta (Henning & Hasselbring, 2021). En este contexto, el estado del arte sugiere la integración de herramientas como *Apache Kafka*³⁴, las cuales implementan un modelo de publicación-suscripción (*publish-subscribe*³⁵), ideal para gobernar flujos asíncronos masivos (Kreps et al., 2011). Conceptualmente, una arquitectura modular bien diseñada y desacoplada garantiza que un sistema esté preparado para migrar hacia este ecosistema híbrido (orientado a eventos) sin requerir una reingeniería profunda en sus nodos analíticos (Waehner, 2020).

²⁹Estilo arquitectónico basado en HTTP para la comunicación entre sistemas web.

³⁰Protocolo para el intercambio de información en la web basado en el modelo cliente-servidor.

³¹Formato ligero de texto para el intercambio de datos.

³²Arquitectura para sistemas distribuidos con interfaz uniforme y sin estado.

³³Mecanismo de control de flujo donde el consumidor regula la velocidad del productor.

³⁴Plataforma distribuida para la transmisión y procesamiento de eventos en tiempo real.

³⁵Patrón de mensajería asíncrona mediado por un bus de eventos entre servicios.

Considerando lo anterior, la arquitectura requiere de un *Message Broker*³⁶ que gestione el flujo de eventos. Complementariamente, se incorporan tecnologías como *SSE*³⁷ para el monitoreo en tiempo real, permitiendo al usuario observar el estado de las tareas asincrónicas sin sobrecargar la red con peticiones constantes (Shabani et al., 2021; World Wide Web Consortium, 2015).

2.2.3. Patrón API Gateway

Para evitar el acoplamiento directo entre el cliente y una red de microservicios, la literatura arquitectónica recomienda el empleo del patrón *API Gateway*³⁸ como punto de entrada único. Este componente centraliza los *endpoints*³⁹, lo cual mitiga el riesgo de fallos en cascada y optimiza la gestión de errores como el *Gateway Timeout*⁴⁰. Este nodo asume las siguientes responsabilidades transversales para la seguridad y el rendimiento del sistema.

- **Enrutamiento dinámico:** Redirige las solicitudes entrantes al contenedor o servicio especializado correspondiente.
- **Seguridad centralizada:** Gestiona la validación de credenciales, como los *JSON Web Tokens* (JWT⁴¹), y aplica políticas de control de acceso.
- **Validación de Carga Útil:** Verifica la integridad semántica de los datos antes de permitir su paso hacia las capas de procesamiento intensivo.

Delegar estas tareas al Gateway permite preservar la capacidad de cómputo de los contenedores subyacentes, reservándola exclusivamente para la ejecución de tareas analíticas y matemáticas (Newman, 2021).

En la Figura 2 se demuestran dos enfoques para la gestión de accesos en arquitecturas

³⁶Componente intermedio que facilita el intercambio de mensajes entre servicios desacoplados.

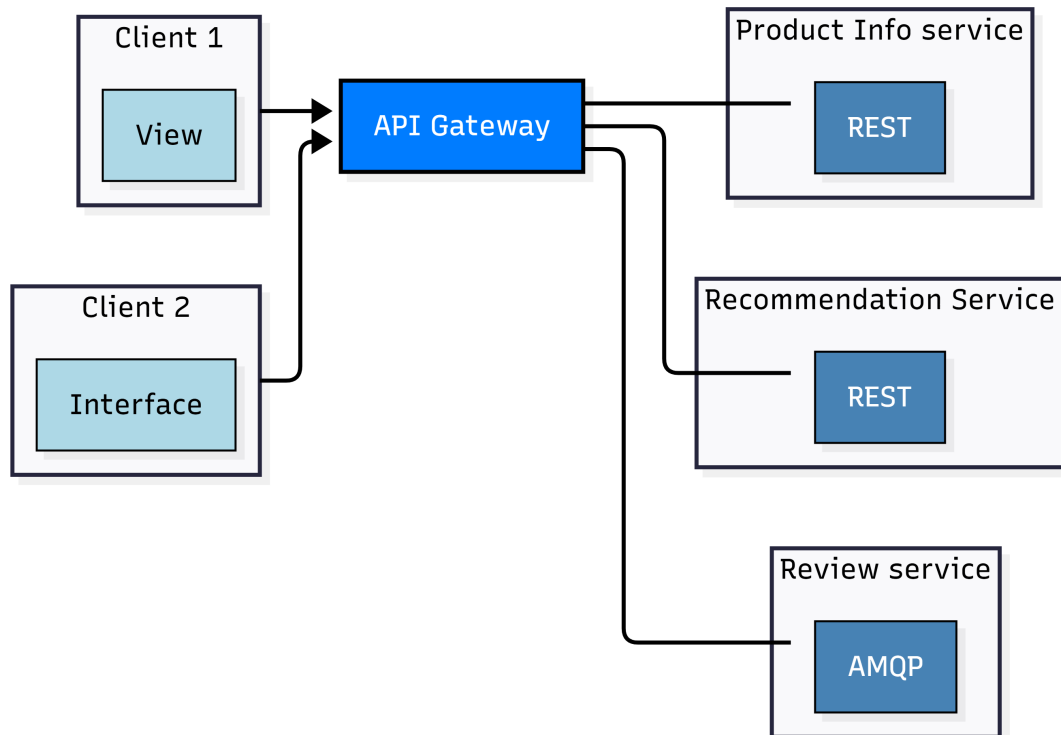
³⁷Server-Sent Events: Permite a un servidor enviar actualizaciones automáticas a un cliente.

³⁸Componente arquitectónico que centraliza las peticiones hacia múltiples microservicios.

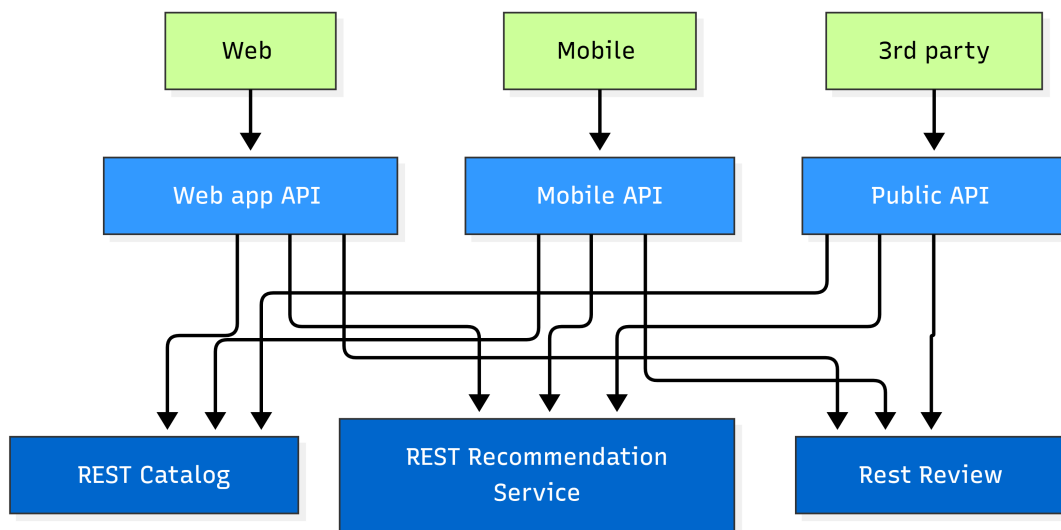
³⁹Puntos de acceso específicos en una API que representan una funcionalidad o recurso.

⁴⁰Error HTTP 504 por tiempo de espera del servidor ascendente.

⁴¹Estándar para la transmisión segura y compacta de información autenticada entre sistemas.



(a) Patrón de API Gateway Único.



(b) Patrón Backend for Frontends (BFF).

Figura 2. Comparativa de patrones de acceso a microservicios. Modificada de (Richardson, 2018).

distribuidas. En la Figura 2a se ilustra el patrón de *API Gateway Único*, el cual centraliza responsabilidades como la seguridad y el ruteo, siendo ideal para sistemas con requerimientos de datos uniformes entre clientes (Richards, 2015). Por el contrario, en la Figura 2b se presenta el patrón *Backend for Frontends (BFF)*, donde se emplean puertas de enlace especializadas para cada tipo de consumidor.

2.2.4. Persistencia Políglota

La naturaleza heterogénea de la información científica hace que depender de un único motor de base de datos resulte ineficiente. Frente a esto, surge el paradigma de la *persistencia políglota*, el cual representa un hito arquitectónico fundamental para sistemas que proyectan integrarse a ecosistemas de *Big Data* (Sadalage & Fowler, 2012). Este enfoque combina estratégicamente distintas tecnologías de almacenamiento:

- **Modelo Relacional (SQL):** Orientado a garantizar propiedades *ACID*⁴², resultando ideal para la gobernanza, gestión de identidades y trazabilidad transaccional (Halili et al., 2025).
- **Modelo Orientado a Documentos (NoSQL):** Proporciona la flexibilidad estructural (*schema-less*) necesaria para ingerir *datasets* variables y artefactos serializados dinámicamente. En este contexto, destaca el uso de formatos de almacenamiento estructurado como *BSON*⁴³, maximizando la eficiencia de lectura y escritura (Eyada et al., 2020). Asimismo, facilita la implementación de patrones de modelado de datos como el *parent-child*⁴⁴, optimizando la consulta de estructuras complejas (Maicha et al., 2022). Este modelo actúa como la base precursora para una futura transición hacia repositorios masivos tipo *Data Lake*⁴⁵ (Kiehn et al., 2022).

2.2.5. Contenedorización y empaquetamiento de software

La contenedorización, liderada por tecnologías como *Docker*, permite empaquetar una aplicación con todas sus dependencias, reduciendo la sobrecarga operativa y garantizando que el código se ejecute de manera determinista sin importar el entorno anfitrión (Merkel, 2014). En la ciencia de datos moderna, esto se alinea directamente

⁴²Propiedades que garantizan la integridad y fiabilidad de las transacciones en bases de datos.

⁴³Representación binaria de documentos JSON, utilizada de forma nativa por MongoDB.

⁴⁴Patrón arquitectónico de datos utilizado para representar relaciones jerárquicas.

⁴⁵Repositorio centralizado para el almacenamiento de datos masivos en su formato nativo.



Figura 3. Arquitectura de tres niveles (S. Kumar, 2019).

con los principios FAIR (Findable, Accessible, Interoperable, Reusable), impulsando la transparencia y auditabilidad del análisis (Wilkinson et al., 2016). Para gestionar múltiples contenedores, la reproducibilidad se consolida mediante el paradigma de Infraestructura como Código (IaC). Utilizando orquestadores locales, como *Docker Compose*⁴⁶, es posible configurar despliegues multicontenedor de forma declarativa, estableciendo un estándar estructurado y escalable para el ciclo de vida de los flujos analíticos (Bodnar et al., 2024).

2.2.6. Arquitectura Cliente-Servidor

El diseño de infraestructuras para el análisis de datos se fundamenta frecuentemente en el modelo de arquitectura cliente-servidor, el cual permite una separación lógica y física entre la interfaz con el usuario y el procesamiento de la información (Pressman & Maxim, 2015). Este paradigma se estructura frecuentemente mediante una distribución de tres niveles que facilita la escalabilidad y el desarrollo de aplicaciones complejas (S. Kumar, 2019). Como se ilustra en la Figura 3 el sistema se divide en capas funcionales independientes para optimizar la gestión de los recursos computacionales.

El *Frontend*⁴⁷ representa el primer nivel de interacción responsable de la captura de parámetros y la visualización de resultados (Sommerville, 2011). Para optimizar la experiencia del usuario y reducir la latencia de navegación, esta capa suele implementarse bajo el modelo de Aplicación de Página Única (*Single Page Application*, SPA). En una SPA, el navegador carga un único documento HTML inicial y actualiza el

⁴⁶Herramienta para la orquestación y gestión simplificada de múltiples contenedores.

⁴⁷Interfaz visual y capa de interacción directa con el usuario.

contenido de forma dinámica mediante JavaScript a medida que el usuario interactúa, evitando la recarga completa de la página y permitiendo una transición fluida entre vistas (Mikowski & Powell, 2013).

Para el *Backend*⁴⁸ el cual constituye el núcleo operativo, siendo en esta capa donde reside la lógica de negocio junto a la gobernanza de los datos (Newman, 2021). Este componente integra el servidor de aplicaciones para la orquestación de servicios y el servidor de datos para asegurar la integridad de la persistencia (S. Kumar, 2019). La implementación de esta arquitectura asegura que cada sección sea escalable de forma independiente ante volúmenes masivos de información (Pressman & Maxim, 2015). Para la gestión de alta concurrencia y la orquestación de red basada en eventos, destaca el uso de Node.js (Chaniotis et al., 2015). A su vez, las operaciones analíticas y de procesamiento intensivo se delegan a *FastAPI*⁴⁹ por su velocidad algorítmica y soporte nativo asíncrono, mientras que la robustez de estas interfaces se refuerza mediante herramientas como *Pydantic*⁵⁰, garantizando la validación semántica y estructural de la información antes de su procesamiento definitivo (Voron, 2021).

2.2.7. Componentes técnicos y ciclo de vida

Una plataforma integral de ciencia de datos debe abarcar conceptualmente desde la captura de la información hasta la explotación y consumo de los modelos predictivos. A continuación, se describen las herramientas y metodologías que fundamentan cada etapa, contrastando los enfoques de validación arquitectónica con las soluciones orientadas a volúmenes masivos.

2.2.7.1. Ingesta, procesamiento distribuido y orquestación

En fases de experimentación o prototipado, la ingesta y el procesamiento algorítmico suelen resolverse eficientemente mediante arquitecturas *REST* y librerías que operan

⁴⁸Capa de lógica, procesamiento y persistencia de datos del sistema.

⁴⁹Marco de trabajo moderno y de alto rendimiento para construir APIs con Python.

⁵⁰Librería de Python para la validación de datos y gestión de configuraciones mediante anotaciones.

en memoria. Sin embargo, el diseño estructural de las plataformas modernas debe contemplar la futura inyección de computación distribuida para soportar cargas masivas. En el estado del arte, esto implica transicionar hacia ecosistemas especializados. Para el procesamiento informático masivo de datos (*batch* en inglés) se recomienda *Apache Spark*, un motor distribuido altamente optimizado para el análisis a gran escala. Por otro lado, para el análisis en *streaming* o procesamiento continuo de eventos en tiempo real, destaca *Apache Flink*, un *framework* distribuido diseñado para operar con baja latencia (Regalado, 2025).

Estos procesos requieren ser coordinados habitualmente por herramientas como *Apache Airflow*, una plataforma diseñada para orquestar y monitorizar flujos de trabajo de datos complejos. Asimismo, a nivel de almacenamiento profundo, la industria proyecta la adopción de arquitecturas *lakehouse*. Este modelo híbrido unifica la flexibilidad y escalabilidad de un *Data Lake*⁵¹ con las capacidades transaccionales de un *Data Warehouse*⁵². Un ejemplo representativo es *Delta Lake*, una capa de almacenamiento que aporta fiabilidad y transacciones *ACID* a estos ecosistemas masivos (Armbrust et al., 2020).

2.2.7.2. Gestión de Modelos: Reproducibilidad Mínima Viable

Para gestionar el ciclo de vida de los algoritmos de aprendizaje automático, el estado del arte en MLOps sugiere la adopción de plataformas dedicadas como *MLflow*⁵³ para el seguimiento de *hiperparámetros*⁵⁴ y *DVC*⁵⁵ para el versionado de conjuntos de datos (Taye, 2023). No obstante, en la literatura orientada a la ingeniería pragmática y la validación arquitectónica temprana (Fairbanks, 2010), el principio de la navaja de Ockham (Blumer et al., 1987) sugiere evitar la sobreingeniería para mitigar la deuda técnica estructural (Sculley et al., 2015). De esta necesidad surge el concepto de

⁵¹ Repositorio centralizado que almacena grandes volúmenes de datos en su formato original.

⁵² Sistema que agrega datos de varias fuentes en un único almacén de datos central y coherente.

⁵³ Plataforma dedicada a la gestión integral del ciclo de vida del aprendizaje automático.

⁵⁴ Valores de configuración establecidos para controlar el proceso de aprendizaje del modelo.

⁵⁵ Sistema de versionado de datos diseñado para garantizar la reproducibilidad científica.

Reproducibilidad Mínima Viable (MVR).

Este enfoque teórico sostiene que acoplando el almacenamiento binario de modelos serializados en volúmenes persistentes de sistema de archivos (locales o en la nube), junto con el registro estricto de su linaje en bases relacionales SQL y la persistencia de metadatos en bases de datos NoSQL es posible garantizar la trazabilidad científica (Vartak et al., 2016). Esto permite cumplir con los estándares de investigación reproducible (Heil et al., 2021) sin requerir infraestructuras sobredimensionadas. En la práctica, este flujo se materializa, por ejemplo, mediante la persistencia de objetos a través de *Joblib*⁵⁶ (Pedregosa et al., 2011), transformando el modelo entrenado en un artefacto estático (Joblib Development Team, 2024) listo para ser consumido por las capas de predicción, clasificación o agrupamiento.

2.2.7.3. Despliegue y Motores de Inferencia

Tras la etapa de modelado, la explotación de los algoritmos requiere su integración en entornos productivos mediante un *motor de inferencia*. En el contexto de la ciencia de datos y MLOps, un motor de inferencia se define como el componente arquitectónico especializado en recibir nuevos conjuntos de datos (información no vista durante el entrenamiento) y aplicar sobre ellos las operaciones matemáticas del modelo serializado para generar predicciones o clasificaciones (Song & Brooks, 2025).

En el paradigma de microservicios, el despliegue de estos motores suele realizarse instanciando el modelo matemático dentro de contenedores orquestados que exponen APIs RESTful. Este diseño modular no solo desacopla la lógica predictiva de la interfaz de usuario, sino que favorece la actualización continua del algoritmo sin interrumpir la disponibilidad del servicio general (Dankwa-Mullan, 2024). Al encapsular la carga de inferencia en un motor dedicado, se cierra el ciclo analítico y se garantiza la interoperabilidad operativa del ecosistema tecnológico (Kreuzberger et al., 2023;

⁵⁶Herramienta para serializar eficientemente objetos Python con grandes datos, como modelos ML.

Newman, 2021).

Para asegurar la estabilidad operativa y un rendimiento de grado industrial, la literatura técnica recomienda la adopción de arquitecturas multicapa que gestionen la concurrencia y la resiliencia del servidor de aplicaciones. En el ecosistema *Python*, el estado del arte sugiere la implementación de gestores de procesos asíncronos usando herramientas como *Gunicorn*⁵⁷ y *Uvicorn*⁵⁸ las cuales operan bajo un modelo de trabajadores (*workers*)⁵⁹ para optimizar el manejo de tareas intensivas y garantizar la recuperación automática ante fallos críticos de procesos individuales (Shabani et al., 2021).

Este esquema se complementa habitualmente con el patrón de *Proxy Inverso*⁶⁰ (usando herramientas como *Nginx*⁶¹), el cual asume responsabilidades transversales de balanceo de carga, terminación *SSL/TLS*⁶² y seguridad perimetral, protegiendo los nodos analíticos de la exposición directa a la red pública (Richards, 2015). Finalmente, la consolidación de estos componentes mediante orquestadores de contenedores permite una gestión declarativa de la infraestructura, asegurando la alta disponibilidad y la capacidad de autorreparación del sistema ante fluctuaciones en la demanda de inferencia (Bodnar et al., 2024; Merkel, 2014).

⁵⁷Servidor HTTP WSGI para Python que emplea un modelo *pre-fork worker*, actuando como un gestor de procesos robusto para aplicaciones web (Grinberg, 2018).

⁵⁸Implementación de servidor ASGI de alto rendimiento basada en *uvloop*, diseñada para soportar la ejecución asíncrona de aplicaciones modernas (Tiangolo, 2024).

⁵⁹Procesos independientes generados por un proceso maestro para manejar peticiones de forma paralela, permitiendo la redundancia y recuperación ante fallos (Richardson, 2018).

⁶⁰Intermediario que centraliza peticiones externas para proteger y balancear servicios internos (Newman, 2021).

⁶¹Motor de red de alto rendimiento diseñado para la gestión eficiente de concurrencia y tráfico (F5, Inc., 2024).

⁶²Estándares de cifrado que garantizan la confidencialidad y autenticidad de la información en tránsito (Rescorla, 2018).

2.3. Inteligencia Artificial y Ciencia de Datos

La inteligencia artificial (IA) se ha consolidado como un área crítica de la informática al permitir la ejecución de tareas cognitivas complejas, como el razonamiento y la toma de decisiones, que tradicionalmente requerían intervención humana (Dankwa-Mullan, 2024). Su integración con la ciencia de datos genera una simbiosis fundamental: mientras la ciencia de datos proporciona la infraestructura metodológica y la organización de los activos de información, la IA actúa en la capa inferencial para transformar grandes volúmenes de datos en conocimiento predictivo y estratégico (Taye, 2023).

Dentro del estado del arte, es imperativo distinguir entre el Aprendizaje Automático (*Machine Learning*, ML) y el Aprendizaje Profundo (*Deep Learning*, DL). El DL utiliza redes neuronales de múltiples capas para aprender representaciones jerárquicas a partir de datos no estructurados, como imágenes o lenguaje natural (LeCun et al., 2015). Sin embargo, su adopción suele estar condicionada a una infraestructura de altos recursos computacionales y grandes volúmenes de datos (Esteva et al., 2017).

En contraste, el Machine Learning se posiciona como el enfoque predominante para el tratamiento de datos estructurados debido a su eficiencia algorítmica y viabilidad operativa (Shwartz-Ziv & Armon, 2022). A diferencia del DL, las técnicas de ML tradicional no requieren necesariamente hardware especializado, permitiendo un rendimiento óptimo en computadores de escritorio convencionales sin sacrificar la capacidad explicativa o la precisión (Jordan & Mitchell, 2015; Obermeyer & Emanuel, 2016). Esta eficiencia se complementa con la metodología MLOps, que estandariza la gestión de experimentos, el control de versiones y el despliegue continuo de modelos, asegurando la robustez teórica y práctica de los sistemas inferenciales (Song & Brooks, 2025; Venkata, 2025).

2.3.1. Machine Learning

Dentro de las capacidades inferenciales de una plataforma de ciencia de datos moderna se sitúan los modelos de aprendizaje automático. Estas técnicas permiten extraer valor predictivo a partir de grandes volúmenes de datos (Beam & Kohane, 2018) y resultan esenciales para tareas de clasificación, detección de anomalías, segmentación de pacientes y predicción de desenlaces clínicos (Obermeyer & Emanuel, 2016).

Entre las tareas que se realizan en el aprendizaje automático están:

- **Clasificación:** Se trata de asignar una etiqueta o categoría a un conjunto de datos basándose en sus características. Entre las técnicas populares de clasificación están las redes neuronales, las máquinas de vector de soporte, y el bosque aleatorio. Entre las tareas destacadas de la clasificación está el análisis de sentimientos, reconocimiento de objetos y la clasificación de texto.
- **Predicción:** Consiste en predecir un valor basado en datos previos. Entre las técnicas populares de predicción está el uso de árboles de decisión, las redes neuronales artificiales y las redes bayesianas. Entre las tareas destacadas de la predicción está la predicción de la siguiente palabra de un texto, la predicción financiera o la predicción del clima.
- **Agrupamiento:** Agrupar un conjunto de datos en grupos o clústeres basados en sus similitudes. Entre las técnicas populares de agrupamiento está el uso de K vecinos más próximos. Entre las tareas destacadas del agrupamiento está el análisis de redes sociales, la organización de documentos y la segmentación de imágenes.

En el ML existen múltiples enfoques de aprendizaje, que son los métodos que utiliza la máquina para obtener conocimiento de la información. Entre los enfoques existentes están:

- **Aprendizaje Supervisado:** En este enfoque, los algoritmos trabajan con datos etiquetados, es decir, clasificados. El objetivo de este aprendizaje es asignar la etiqueta correcta a nuevos datos no etiquetados. Este aprendizaje se utiliza en algoritmos de predicción o clasificación (Cunningham et al., 2008).
- **Aprendizaje No Supervisado:** En este enfoque, los algoritmos trabajan con datos no etiquetados ni categorías predefinidas, analizando únicamente los patrones de los datos y categorizando respecto a las diferencias entre los propios datos. El objetivo de este aprendizaje es descubrir patrones, estructuras o agrupaciones ocultas en los datos. Este aprendizaje se utiliza en algoritmos de agrupación (Ghahramani, 2004).
- **Aprendizaje Semi-Supervisado:** En este enfoque, se combinan los aprendizajes supervisados y no supervisado, utilizando tanto datos etiquetados como no etiquetados. En este aprendizaje se aprovechan los datos no etiquetados para formar grupos de datos que se catalogan respecto a los datos etiquetados. Se utiliza tanto en algoritmos de clasificación y agrupación (Chapelle et al., 2006).
- **Aprendizaje por Refuerzo:** En este enfoque, el agente aprende a comportarse en un entorno, realizando acciones y observando los resultados. Este aprendizaje busca maximizar una recompensa recibida por las acciones. Este enfoque se utiliza en algoritmos de optimización y predicción (Sutton & Barto, 1998).

El aprendizaje automático se utiliza en una amplia variedad de aplicaciones y campos, como el reconocimiento de voz, la visión por computadora, la conducción autónoma, la recomendación de contenido, la detección de fraudes, entre otros. A medida que la cantidad de datos disponibles ha aumentado exponencialmente en los últimos años, el aprendizaje automático se ha convertido en una herramienta fundamental para extraer información valiosa y tomar decisiones basadas en datos en una variedad de industrias y disciplinas (Kubát, 2017).

Existen múltiples técnicas de aprendizaje automático que se implementan respecto a la tarea requerida:

- **Bosque Aleatorio (Random Forest):** Algoritmo de ensamble que combina múltiples árboles de decisión para mejorar la precisión y controlar el sobreajuste (Breiman, 2001).
- **Regresión Ridge (Ridge Regression):** Variante de la regresión lineal que incorpora una regularización L2 para mitigar la colinealidad y el sobreajuste (Hoerl & Kennard, 1970).
- **Regresión Lineal (Linear Regression):** Método fundamental que modela la relación lineal entre variables predictoras y una variable de respuesta continua (Cox, 1958).
- **Clasificación de Vectores de Soporte (SVC):** Técnica que proyecta datos en espacios de alta dimensión para hallar hiperplanos de separación óptimos entre categorías (Cortes & Vapnik, 1995).
- **K-Vecinos Cercanos (KNN):** Método no paramétrico que clasifica instancias basándose en la métrica de proximidad de sus vecinos más cercanos (Cover & Hart, 1967).
- **Regresión Logística (Logistic Regression):** Modelo estadístico utilizado para estimar la probabilidad de pertenencia a una categoría específica (Cox, 1958).
- **K-Medias (K-Means):** Algoritmo de aprendizaje no supervisado que particiona los datos en k grupos minimizando la varianza intraclúster (Lloyd, 1982).
- **DBSCAN:** Algoritmo basado en densidad capaz de identificar agrupaciones de forma arbitraria y detectar ruido en los datos (Ester et al., 1996).
- **Modelos de Mezcla Gaussiana (GMM):** Modelo probabilístico que asume que los datos son generados a partir de una mezcla de varias distribuciones normales (Reynolds, 2009).

Los fundamentos matemáticos de estos métodos se detallan en el Anexo A.

2.3.2. Métricas de Evaluación

De manera funcional, las tareas principales del *ML* se categorizan según el tipo de problema a resolver. Para cuantificar la capacidad de generalización y el error de los algoritmos frente a datos no observados, cada categoría emplea métricas de evaluación específicas (Kuhn & Johnson, 2013):

2.3.2.1. Evaluación de Predicción (Regresión)

Consiste en la estimación de valores numéricos continuos basados en datos históricos (Botchkarev, 2018). Las métricas utilizadas para evaluar estos modelos se definen a continuación:

- **Error Absoluto Medio (MAE):** Promedio de las diferencias absolutas entre las predicciones y los valores reales (Willmott & Matsuura, 2005). Para un conjunto de n muestras, se utiliza la Ecuación (2.1), donde y_i es el valor real y $\hat{y}_i$ es el valor predicho:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (2.1)$$

- **Error Cuadrático Medio (MSE):** Medida del promedio de los errores al cuadrado, que penaliza desviaciones grandes (Chai & Draxler, 2014). Para un conjunto de n muestras, se utiliza la Ecuación (2.2), donde los errores se elevan al cuadrado para evitar compensaciones:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (2.2)$$

- **Raíz del Error Cuadrático Medio (RMSE):** Proporciona una medida del error en las mismas unidades que la variable objetivo. Para su cálculo, se utiliza la Ecuación (2.3), donde se aplica la raíz cuadrada al promedio de los errores

cuadráticos:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2.3)$$

- **Coeficiente de Determinación (R^2):** Indica la proporción de la varianza predecible en la variable dependiente (Chicco et al., 2021). Para determinar este ajuste, se utiliza la Ecuación (2.4), donde $\bar{y}$ representa la media de los valores reales:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (2.4)$$

2.3.2.2. Evaluación de Clasificación

Asignación de etiquetas categóricas a nuevas instancias. Sus métricas se basan en la matriz de confusión (Powers, 2020):

- **Exactitud (Accuracy):** Mide la proporción de predicciones correctas totales. Para obtener este valor, se utiliza la Ecuación (2.5), donde TP , TN , FP y FN representan los verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos respectivamente:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.5)$$

- **Precisión (Precision):** Evalúa la calidad de las predicciones positivas. Para calcular la fiabilidad de estos aciertos, se utiliza la Ecuación (2.6), donde TP se divide por el total de predicciones positivas:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2.6)$$

- **Sensibilidad (Recall / Sensitivity):** Mide la capacidad del modelo para detectar casos positivos. Para identificar la tasa de captura, se utiliza la Ecuación (2.7),

donde FN representa los casos positivos no detectados:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2.7)$$

- **Puntuación F1 (F1-Score):** Representa la media armónica entre la Precisión y la Sensibilidad (Fawcett, 2006). Para hallar este balance, se utiliza la Ecuación (2.8), donde se combinan ambas métricas para penalizar valores extremos:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.8)$$

2.3.2.3. Evaluación de Agrupamiento (Clustering)

Identificación de estructuras y similitudes en datos no etiquetados (Jain, 2010). Para evaluar la calidad de las agrupaciones se utilizan las siguientes métricas:

- **Coeficiente de Silueta (s):** Mide qué tan similar es un objeto a su propio grupo en comparación con otros grupos (Rousseeuw, 1987). Para una muestra i , se utiliza la Ecuación (2.9), donde $a(i)$ es la distancia media intra-cluster y $b(i)$ es la distancia media al cluster más cercano:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (2.9)$$

- **Índice Davies-Bouldin (DB):** Evalúa la separación entre clusters y su cohesión interna (Davies & Bouldin, 1979). Para un conjunto de k clusters, se utiliza la Ecuación (2.10), donde s_i es la dispersión del cluster i y d_{ij} es la distancia entre los centroides de los clusters i y j :

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{s_i + s_j}{d_{ij}} \right) \quad (2.10)$$

- **Inercia:** Suma de las distancias al cuadrado de cada objeto al centroide de su cluster asignado (MacQueen, 1967). Para un conjunto de n muestras, se utiliza

la Ecuación (2.11), donde μ_j representa el centroide del cluster C más cercano:

$$\text{Inercia} = \sum_{i=1}^n \min_{\mu_j \in C} (||x_i - \mu_j||^2) \quad (2.11)$$

2.3.3. Consideraciones éticas e interpretabilidad

A pesar de sus beneficios potenciales, el uso creciente de la IA en salud y otros sectores sensibles plantea desafíos éticos importantes, especialmente en términos de equidad, sesgos en modelos predictivos y transparencia (Char et al., 2018). Un problema recurrente es que los modelos entrenados con conjuntos de datos limitados o sesgados tienden a amplificar desigualdades existentes, afectando negativamente a grupos vulnerables (Dankwa-Mullan, 2024).

La interpretabilidad y la transparencia se han convertido, por tanto, en requisitos fundamentales. Técnicas como *LIME* (Local Interpretable Model-agnostic Explanations) han surgido como métodos para explicar localmente decisiones específicas tomadas por modelos de *ML* y *DL*, facilitando su aceptación y comprensión tanto por profesionales médicos como por pacientes (Ribeiro et al., 2016). Adicionalmente, marcos científicos rigurosos han sido propuestos para evaluar sistemáticamente y comparar la calidad de estas explicaciones, aportando confiabilidad y validez científica al uso de modelos complejos en entornos críticos (Doshi-Velez & Kim, 2017).

Finalmente, el cumplimiento normativo y ético implica adoptar principios de diseño como la privacidad desde la *concepción*⁶³, técnicas avanzadas de *anonimización de datos*⁶⁴ y auditorías continuas del desempeño y *sesgos de los modelos*⁶⁵, asegurando no solo la calidad técnica, sino también la confianza social y la equidad en las aplicaciones reales de estos sistemas.

⁶³Integración de la privacidad como requisito desde el inicio del desarrollo (Cavoukian, 2009).

⁶⁴Procesos técnicos para evitar la reidentificación de sujetos en datos (Dwork, 2008)).

⁶⁵Distorsiones algorítmicas con impacto desigual entre grupos (Raji et al., 2020).

2.4. Ingeniería de software

La ingeniería de software se define como la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software («IEEE Standard Glossary of Software Engineering Terminology», 2008). En el ámbito científico, este paradigma permite transformar procesos experimentales en plataformas robustas, mitigando la deuda técnica y asegurando la escalabilidad del sistema (Sculley et al., 2015). Esta disciplina fomenta un entorno colaborativo ágil que integra equipos multidisciplinarios en ciclos iterativos. Para optimizar la gestión del código fuente en arquitecturas distribuidas, se adopta frecuentemente el uso de un *Monorepo*⁶⁶, facilitando la adaptación ante la incertidumbre propia de la investigación (Sommerville, 2011).

Mediante el uso de patrones de diseño, se garantiza una arquitectura modular y desacoplada capaz de sostener flujos analíticos de alta complejidad (Gamma et al., 1994). Finalmente, las prácticas de MLOps permiten cerrar el ciclo de vida del software al automatizar el despliegue y monitoreo, resguardando la integridad operativa desde la fase de experimentación hasta la puesta en producción definitiva (Kreuzberger et al., 2023).

2.4.1. Metodologías ágiles: Scrum y UML

Las metodologías ágiles surgieron como respuesta a la rigidez de modelos clásicos, por ejemplo, el modelo en cascada (Sommerville, 2011) o el modelo en V (Pressman & Maxim, 2015). Se orientan a ciclos cortos de entrega (*sprints*) que permiten incorporar retroalimentación temprana y adaptarse a requisitos cambiantes. El Manifiesto Ágil destaca la colaboración con el cliente y el software funcional sobre la documentación extensa (Beck et al., 2001).

⁶⁶Estrategia de repositorio único para múltiples proyectos o microservicios.

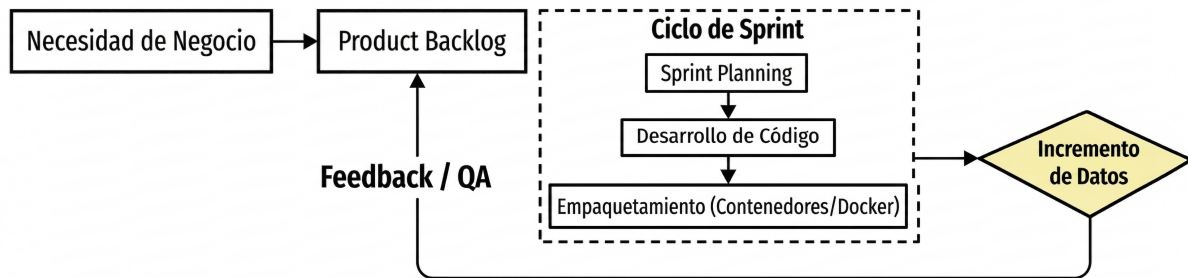


Figura 4. Flujo de trabajo Scrum. (Elaboración propia)

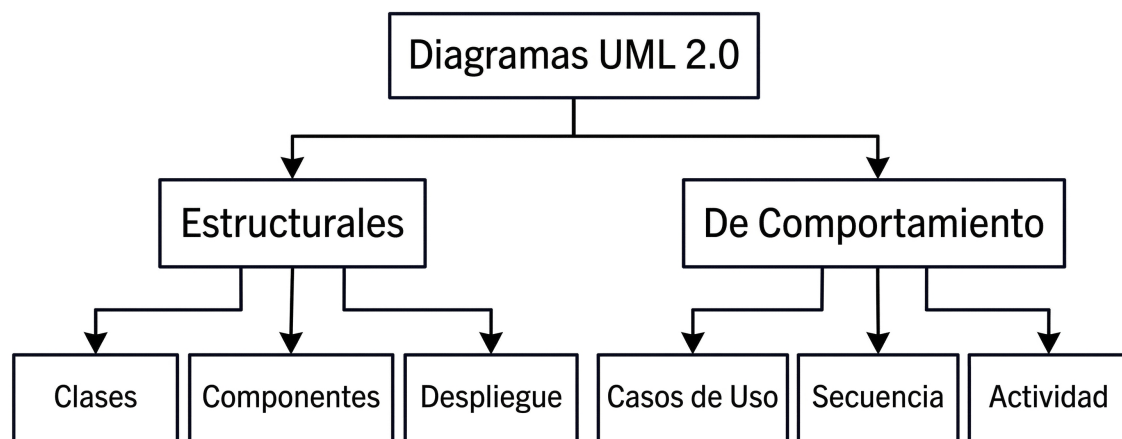


Figura 5. Diagramas UML 2.0. Modificado de Sommerville (2011)

Scrum define roles de Product Owner, Scrum Master y equipo de desarrollo. Promueve transparencia, inspección y adaptación mediante *daily scrums*, revisiones y retrospectivas iterativas (Schwaber & Sutherland, 2020). Resulta especialmente útil cuando varios equipos desarrollan componentes independientes de una plataforma modular, ya que reduce tiempos de integración y mejora la comunicación (Dikert et al., 2016). El flujo de trabajo iterativo y los hitos del proceso adoptado se presentan en la Figura 4.

La documentación visual de estos sistemas se realiza mediante el *Lenguaje de Modelado Unificado* (UML), consolidado como el estándar industrial para el diseño de software orientado a objetos (Booch et al., 2005). Tal como expone la Figura 5, sus notaciones gráficas se dividen en categorías estructurales y de comportamiento,

destacando dos enfoques fundamentales para el modelado de la plataforma:

- **Diagramas de Casos de Uso:** Pertenecientes a la categoría de modelado de comportamiento, estos diagramas especifican las funcionalidades del sistema desde la perspectiva de los usuarios externos (actores). Permiten definir claramente los límites del sistema y los privilegios de interacción, siendo una herramienta crucial para derivar requisitos funcionales y políticas de control de acceso (Sommerville, 2011).
- **Diagramas de Secuencia:** Ubicados dentro de la categoría de modelado de interacción, ilustran cómo los componentes internos del sistema se comunican entre sí a lo largo del tiempo. En arquitecturas basadas en microservicios, estos diagramas son esenciales para mapear el ciclo de vida transaccional, detallando las llamadas a APIs, el flujo asíncrono de datos y las operaciones de persistencia en bases de datos (Booch et al., 2005).

La aplicación conjunta de estas herramientas de modelado facilita la validación de la coherencia lógica de los componentes analíticos y de gestión antes de su implementación, reduciendo la incertidumbre durante los ciclos de desarrollo (Sommerville, 2011).

2.4.2. Patrones de arquitectura: MVC y EDA

Los patrones arquitectónicos establecen cómo se estructuran y comunican los componentes internos de una plataforma. El patrón *Modelo-Vista-Controlador* (MVC) separa datos, lógica de negocio y capa de presentación, lo que facilita las pruebas unitarias y el desarrollo paralelo (Haduth et al., 2022; M. Kumar & Choppella, 2024; Necula, 2024). Como se ilustra en la Figura 6, esta estructura permite que el usuario interactúe con la vista mientras el controlador gestiona la lógica de entrada para actualizar el modelo.

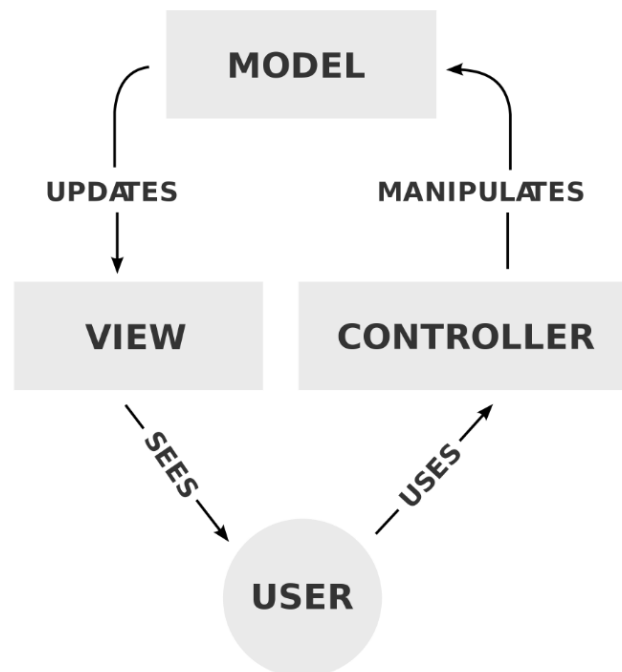


Figura 6. Arquitectura Modelo-Vista-Controlador (MVC) (M. Kumar & Choppella, 2024).

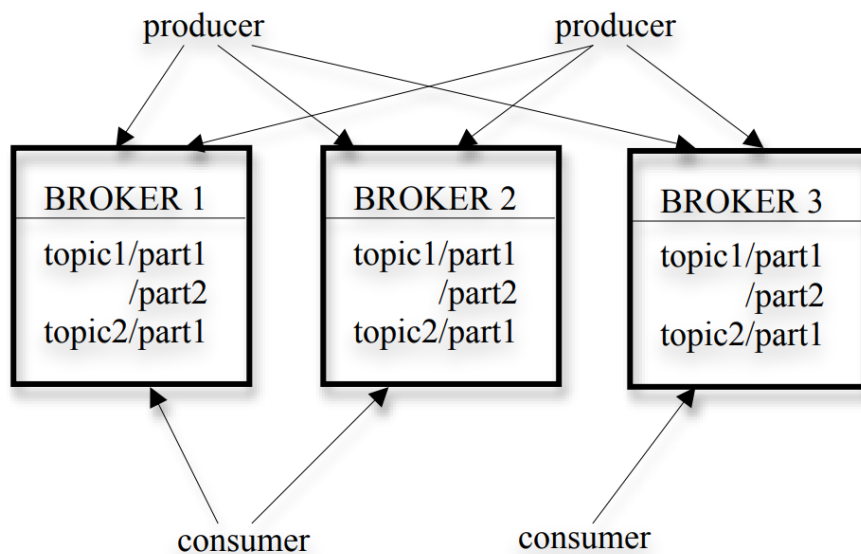


Figura 7. Arquitectura dirigida por eventos (Kreps et al., 2011).

Por otro lado, la *Arquitectura Dirigida por Eventos* (EDA) introduce un bus de mensajería que desacopla productores y consumidores dentro de una plataforma. Este esquema de comunicación asíncrona, representado en la Figura 7, incrementa la *resiliencia ante fallos*⁶⁷ debido a que si un servicio se interrumpe el resto del

⁶⁷Capacidad del sistema para mantener disponibilidad y niveles operativos ante fallos de sus

sistema puede continuar funcionando sin dependencia directa. Asimismo, al reducir el acoplamiento, es posible escalar o integrar nuevos componentes de forma incremental, lo que favorece la evolución progresiva de la arquitectura (Kreps et al., 2011; Waehner, 2020).

En conjunto, estos patrones aportan la flexibilidad necesaria para integrar microservicios, procesar flujos asíncronos y mantener la coherencia del sistema en entornos distribuidos (Newman, 2021).

2.4.3. MLOps y prácticas de despliegue continuo

El concepto de MLOps constituye el marco operativo para la gestión sistemática del ciclo de vida de los modelos de aprendizaje automático, abarcando desde el versionado de datos hasta el monitoreo en entornos productivos (Song & Brooks, 2025). Esta disciplina permite la transición de procesos experimentales aislados hacia sistemas de software confiables y escalables mediante la integración de principios de ingeniería de software y operaciones de datos (Kreuzberger et al., 2023).

La garantía de reproducibilidad en estos flujos de trabajo se sustenta en la tecnología de contenedores Docker, la cual permite empaquetar los modelos junto a sus dependencias exactas y librerías específicas (Boettiger, 2015). Este enfoque asegura un comportamiento determinista de las aplicaciones independientemente de la infraestructura anfitriona, facilitando el aislamiento de procesos y la portabilidad del sistema (Merkel, 2014).

En este contexto, la orquestación de la arquitectura modular se puede realizar mediante Docker Compose, herramienta que permite el despliegue coordinado de múltiples microservicios a través de archivos de configuración declarativos (List, 2017). El uso de esta plataforma de orquestación garantiza la consistencia operativa y la

componentes internos sin colapsar la plataforma.

rápida replicación de la infraestructura completa en diversos entornos de investigación y desarrollo (Mäkinen et al., 2021).

2.5. Calidad de software y evaluación de rendimiento

El valor de una arquitectura escalable se materializa cuando la plataforma demuestra solidez funcional y estabilidad bajo condiciones de carga reales (Bondi, 2000). En entornos de investigación aplicada es habitual que coexistan procesos analíticos de alto cómputo con consultas interactivas constantes que exigen respuestas inmediatas (Candela et al., 2020). En este escenario resulta imperativo validar que la modularidad y la contenedorización sustenten los flujos de trabajo sin comprometer la integridad del sistema ni generar latencias inaceptables (Luettgau et al., 2023; Merkel, 2014).

A continuación se presentan las estrategias de QA, se definen las métricas críticas de rendimiento como latencia, *throughput* y escalabilidad y se describen las herramientas especializadas para la ejecución de pruebas funcionales y de carga en ecosistemas modernos.

2.5.1. Estrategias de QA

Para estructurar la validación funcional del software, la literatura destaca el modelo de la *Pirámide de Pruebas* (Test Pyramid) (Cohn, 2009). Este marco conceptual propone una organización jerárquica de los esfuerzos de prueba:

- **Base (Pruebas Unitarias):** Constituyen el nivel más numeroso y rápido, enfocado en validar de forma aislada la lógica de funciones o clases específicas.
- **Medio (Pruebas de Integración):** Verifican la comunicación y el intercambio de datos entre servicios, bases de datos y APIs, lo cual resulta crítico en arquitecturas de microservicios.
- **Cúspide (Pruebas E2E):** Simulan el comportamiento integral del usuario final

Tabla 1. Comparativa de modelos de estrategias de aseguramiento de calidad.

Modelo	Foco	Ventaja
Pirámide de Pruebas	Aislamiento lógico y velocidad de ejecución.	Valida la precisión matemática de los algoritmos de forma aislada antes de orquestar la infraestructura.
Testing Trophy	Interoperabilidad de servicios.	Maximiza la fiabilidad en la comunicación de APIs distribuidas asegurando la consistencia transaccional.
Agile Quadrants	Propósito técnico y valor de negocio.	Garantiza una trazabilidad directa entre los requisitos del investigador y la funcionalidad entregada.

y son más costosas y lentas, por lo que se reservan habitualmente para validar flujos críticos.

Existen otros enfoques relevantes adaptados a las exigencias de arquitecturas modernas y ciclos de desarrollo ágiles:

- **Testing Trophy:** Otorga mayor relevancia a las pruebas de integración para asegurar la interoperabilidad en entornos web y sistemas distribuidos, buscando un equilibrio entre velocidad y seguridad operativa (Dodds, 2018).
- **Agile Testing Quadrants:** Clasifican las validaciones en cuatro dimensiones según su orientación técnica o de negocio para equilibrar el apoyo al equipo de desarrollo con la evaluación de los requisitos funcionales (Crispin & Gregory, 2009).

En la Tabla 1 se presenta una síntesis de las principales características y ventajas de estos modelos de aseguramiento de calidad.

2.5.2. Métricas de rendimiento

Una vez garantizada la funcionalidad del sistema resulta necesario evaluar su comportamiento bajo condiciones de estrés concurrente (Bondi, 2000). Para ello la literatura especializada define métricas estandarizadas que permiten cuantificar el

rendimiento en arquitecturas de microservicios y sistemas orientados al procesamiento masivo de información (Luettgau et al., 2023).

- **Latencia (L):** Corresponde al tiempo transcurrido desde que un cliente emite una petición hasta que recibe el primer byte de la respuesta (Luettgau et al., 2023). Un valor alto en esta métrica impacta negativamente en la experiencia de uso dentro de plataformas de datos aplicadas (Pressman & Maxim, 2015). Se calcula mediante la Ecuación 2.12, que mide la diferencia entre el tiempo de recepción (t_{rx}) y el de emisión (t_{tx}) (Bondi, 2000).

$$L = t_{rx} - t_{tx} \quad (2.12)$$

Para mitigar el sesgo generado por valores atípicos se emplean percentiles como P_{95} o P_{99} (Bondi, 2000). Estas medidas representan el umbral de latencia bajo el cual se encuentra un porcentaje específico de las solicitudes totales procesadas y permiten identificar la degradación del servicio en escenarios críticos (Luettgau et al., 2023).

- **Rendimiento (*Throughput* - X):** Representa la cantidad de transacciones o unidades de información procesadas con éxito por el sistema en un intervalo de tiempo determinado (Bondi, 2000). Esta métrica refleja la capacidad operativa real del servidor bajo condiciones de carga sostenida (Sommerville, 2011). El cálculo se rige por la Ecuación 2.13.

$$X = \frac{N}{T} \quad (2.13)$$

En esta expresión N representa el número de peticiones completadas de forma satisfactoria mientras que T corresponde al tiempo total de observación (Bondi, 2000).

- **Tasa de Error (E):** Se obtiene con la Ecuación 2.14 que cuantifica la proporción de transacciones que fallan debido a la saturación de recursos respecto al volumen

total de peticiones intentadas (Sommerville, 2011). Las fallas suelen manifestarse mediante códigos de error HTTP o el agotamiento de tiempos de espera en la comunicación entre microservicios (Luettgau et al., 2023).

$$E = \left(\frac{N_{fallos}}{N_{totales}} \right) \times 100 \quad (2.14)$$

Este indicador permite determinar los límites de estabilidad de la arquitectura frente a ráfagas de tráfico inesperadas (Bondi, 2000).

- **Escalabilidad:** Describe la capacidad de la plataforma para manejar un incremento en la demanda de trabajo mediante la adición de recursos computacionales (Bondi, 2000). Un diseño escalable permite mantener niveles aceptables de *throughput* y controlar la latencia a medida que aumenta la concurrencia de usuarios en el sistema (Luettgau et al., 2023).
- **Disponibilidad (A):** Indica la probabilidad de que un sistema se encuentre operativo y accesible para cumplir su función en un instante de tiempo dado (Sommerville, 2011). Su cálculo se fundamenta en la robustez de los componentes y su capacidad de recuperación ante fallos parciales (Pressman & Maxim, 2015). Para obtener este valor se utiliza la Ecuación 2.15.

$$A = \frac{MTBF}{MTBF + MTTR} \quad (2.15)$$

El tiempo medio entre fallos (MTBF) y el tiempo medio de recuperación (MTTR) definen la continuidad del servicio en entornos de producción (Sommerville, 2011).

2.5.3. Herramientas de evaluación y observabilidad

Para implementar las estrategias de validación en una arquitectura desacoplada, se requiere un conjunto de herramientas especializadas según el nivel de prueba y la tecnología empleada:

- **Nivel Funcional:** En el ecosistema del *Frontend* (React), se utiliza *Jest* como

estándar para pruebas unitarias debido a su capacidad de *mocking*⁶⁸ (Meta Open Source, 2023). Para el *Backend* (FastAPI), se emplea *Pytest*⁶⁹, permitiendo validar la lógica de los servicios analíticos.

- **Integración de APIs:** La validación de contratos de datos y códigos de respuesta HTTP se realiza mediante bibliotecas como *Supertest* o plataformas como *Postman*, asegurando la integridad de los objetos JSON intercambiados entre microservicios (Meta Open Source, 2023).
- **Nivel de Carga:** Para simular concurrencia y medir la resiliencia del sistema, se utiliza Apache JMeter, herramienta empleada para ejecutar escenarios de prueba concurrentes y evaluar latencia, throughput y tasa de error bajo demanda masiva (Apache Software Foundation, 2023).

Complementariamente, la observabilidad del sistema se gestiona mediante análisis estático de código utilizando *linters*⁷⁰, junto con el monitoreo centralizado de registros (*logs*) provenientes de los contenedores, lo que contribuye a garantizar la mantenibilidad y la transparencia operativa de la plataforma.

2.6. Conclusiones del capítulo

En el capítulo se presentaron los fundamentos necesarios para el análisis, diseño y modelado de arquitecturas escalables destinadas a plataformas de ciencia de datos colaborativas. La convergencia entre los principios de gobernanza (FAIR y linaje) y el diseño modular basado en microservicios establece los pilares teóricos para asegurar la trazabilidad de la información. Bajo este enfoque, la descripción de los paradigmas de aprendizaje automático, operacionalizados mediante microservicios de entrenamiento e inferencia, constituye el soporte técnico necesario para obtener resultados analíticos consistentes y reproducibles en entornos de investigación.

⁶⁸Técnica de implementaciones simuladas para aislar el comportamiento del código.

⁶⁹Framework de pruebas para escribir tests pequeños, legibles y escalables en Python.

⁷⁰Herramientas automatizadas que detectan errores de sintaxis, estilo y posibles vulnerabilidades sin ejecutar el código.

Asimismo, la aplicación de metodologías de ingeniería de software con pasos específicos y claros disminuye el riesgo implícito en el desarrollo, evitando el incumplimiento de requerimientos funcionales o técnicos. En términos informáticos, el uso de contenedores Docker permite la implementación de los sistemas en entornos aislados, lo cual garantiza la paridad entre entornos de ejecución y facilita significativamente su empaquetamiento, levantamiento y despliegue. Finalmente, las prácticas de MLOps y la aplicación de los criterios de aseguramiento de calidad consolidan una base teórica de evaluación capaz de responder a las exigencias operativas de eficiencia en entornos de investigación colaborativa.

Por tanto, con lo descrito en este marco teórico se dispone de una base teórica, técnica y de evaluación para el diseño e implementación de una propuesta de plataforma de ciencia de datos para investigación colaborativa.

Capítulo 3. Diseño de propuesta de software

En este capítulo se presenta el diseño de la propuesta de software, integrando las especificaciones técnicas y arquitectónicas necesarias para la construcción de la plataforma. El contenido se estructura de la siguiente manera.

La Sección 3.1 presenta el análisis de requerimientos de software. En la Sección 3.2 se presenta el diseño del software. En la Sección 3.3 se presenta el plan de pruebas para evaluar la calidad de la propuesta de software. En la Sección 3.4 se expone el modelo de ingeniería de software. Finalmente, en la Sección 3.5 se presentan las conclusiones del capítulo.

3.1. Análisis de requerimientos

El análisis de requerimientos define el alcance funcional y operativo de la plataforma, asegurando que el sistema cumpla las necesidades analíticas del entorno de investigación. Para estructurar formalmente este proceso, los requerimientos se clasifican en funcionales y no funcionales. Los primeros describen las capacidades y servicios que debe ofrecer el sistema, mientras que los segundos establecen atributos de calidad asociados a rendimiento, seguridad, escalabilidad y mantenibilidad. Asimismo, se identifican los perfiles de usuario que interactúan con la plataforma.

Perfiles de Actores del Sistema

- **Investigador de datos:** Usuario técnico responsable de la configuración, entrenamiento iterativo y gestión del ciclo de vida de los modelos de *machine learning*, incluyendo su versionado y validación experimental.

- **Usuario:** Usuario final que emplea las herramientas predictivas para la ingesta de datos y la interpretación visual de resultados, apoyando la toma de decisiones basadas en evidencia.
- **Administrador:** Responsable de la orquestación del despliegue y la gestión de accesos mediante políticas de control basado en roles.

Requerimientos Funcionales

- El sistema debe proporcionar interfaces web diferenciadas que separen el entorno de experimentación algorítmica del entorno de carga de datos para inferencia.
- La arquitectura debe implementar módulos analíticos funcionalmente independientes para tareas de predicción, clasificación, agrupamiento e inferencia, promoviendo alta cohesión y bajo acoplamiento.
- El motor de inferencia debe operar de forma desacoplada y no bloqueante, permitiendo la ejecución de modelos previamente entrenados para el procesamiento de conjuntos de datos tabulares en *formato CSV*.
- La plataforma debe restringir el acceso y las funcionalidades de acuerdo con un modelo de control de acceso basado en roles (RBAC).
- El sistema debe permitir la visualización interactiva de metadatos y la generación de reportes exportables en formato *PDF*.
- El diseño arquitectónico debe facilitar la incorporación de nuevos modelos o servicios de inferencia sin requerir modificaciones estructurales en la infraestructura existente.

Requerimientos No Funcionales

- **Usabilidad:** La interfaz gráfica debe abstraer la complejidad matemática subyacente, minimizando la carga cognitiva del usuario y favoreciendo una experiencia intuitiva.

- **Rendimiento:** Las operaciones de inferencia individual deben retornar resultados en un intervalo de latencia acotado entre 1 y 5 segundos, asegurando viabilidad operativa en entornos reales.
- **Portabilidad y Escalabilidad:** El sistema debe ser agnóstico a la infraestructura subyacente (local o en la nube) y desplegable mediante contenedorización con Docker, garantizando reproducibilidad y escalabilidad horizontal.
- **Mantenibilidad y Modularidad:** El código debe estructurarse bajo principios de bajo acoplamiento y separación de responsabilidades, permitiendo la evolución independiente de los servicios analíticos.
- **Seguridad:** El tratamiento de datos sensibles debe incorporar autenticación centralizada, control de accesos y mecanismos de cifrado, alineándose con el principio de privacidad desde el diseño.

3.2. Diseño de software

En esta sección se detalla el diseño de la plataforma, traduciendo los requisitos técnicos en una arquitectura formal que garantiza funcionalidad y rendimiento. El contenido aborda la definición de los **Casos de Uso** y el alcance del sistema, para luego describir la **Arquitectura de Microservicios** bajo el estándar RESTful. Finalmente, se presenta la maquetación del **Frontend** junto con la lógica del **Backend** y la integración de los módulos de *Machine Learning* destinados al procesamiento analítico.

3.2.1. Casos de uso

El alcance del sistema se definió en UML con una estricta separación de responsabilidades. Mientras el usuario administra la carga de datos e inferencias, el investigador controla las tareas experimentación y el ciclo de vida de los modelos (MLOps). La Figura 8 detalla la interacción entre estos actores y los módulos funcionales.

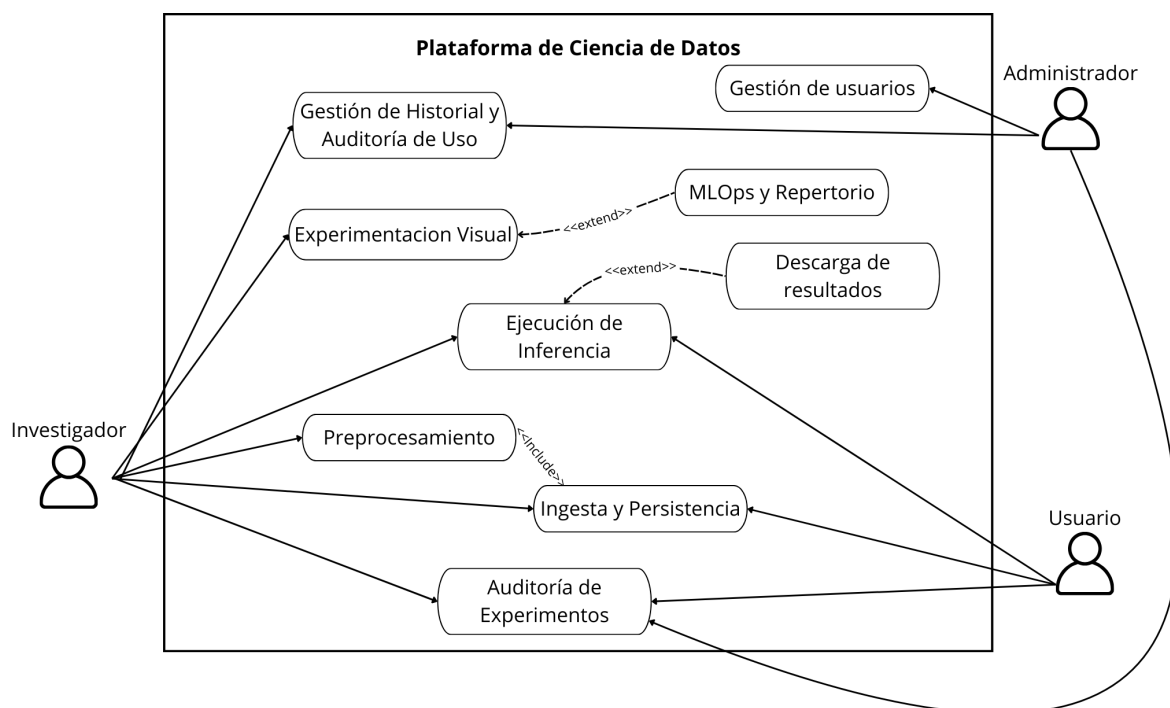


Figura 8. Diagrama de casos de uso del sistema.

La escalabilidad funcional, entendida como la capacidad de expandir las prestaciones del sistema mediante la agregación incremental de nuevos servicios especializados. Bajo este paradigma, la modularidad permite incorporar capacidades analíticas adicionales sin requerir una reingeniería del núcleo, diferenciando con precisión las operaciones transaccionales del almacenamiento documental masivo. Esta visión de diseño, que responde a una gestión de datos híbrida y extensible, se ve reflejada en los nueve casos de uso definidos en la Tabla 2, donde se sintetizan sus objetivos, los actores involucrados y los resultados esperados. La especificación técnica exhaustiva de cada interacción se encuentra disponible en el Anexo B.

La definición de estos nueve casos de uso permite concluir que la plataforma no solo cumple con la función operativa de procesar algoritmos de aprendizaje automático, sino que establece un marco robusto de gobernanza de datos. Al separar la ejecución inmediata (CU-06) de la exportación opcional (CU-07) y de la gestión histórica (CU-08 y CU-09), el sistema garantiza la trazabilidad técnica exigida en entornos de investigación colaborativa.

Tabla 2. Casos de uso.

ID	Caso de Uso	Actores	Resultado
CU-01	Gestión de Usuarios y Roles	Administrador	Registro RBAC en PostgreSQL.
CU-02	Ingesta de Datos	Usuario, Investigador	Metadata en PostgreSQL y documentos BSON en MongoDB.
CU-03	Preprocesamiento y Vectorización	Investigador	Generación de <i>mappings</i> y persistencia en MongoDB.
CU-04	Experimentación y Streaming Visual	Investigador	Streaming SSE hacia el frontend (sin persistencia inmediata).
CU-05	MLOps y Repertorio de Modelos	Investigador	Serialización de modelos y registro de versión en PostgreSQL.
CU-06	Ejecución de Inferencia	Usuario, Investigador	Cálculo matricial y persistencia de resultados en MongoDB.
CU-07	Exportación de Reportes	Usuario, Investigador	Generación de archivos binarios (CSV/PDF) en local.
CU-08	Historial y Auditoría de Uso	Usuario, Investigador, Admin	Recuperación de registros históricos desde MongoDB.
CU-09	Auditoría de Experimentos	Investigador, Administrador	Recuperación de logs técnicos e hiperparámetros desde MongoDB.

3.2.2. Arquitectura de Plataforma

La plataforma emplea una arquitectura modular de microservicios desplegada en un *Servidor Privado Virtual* (VPS). El diseño se rige por decisiones de ingeniería orientadas a la viabilidad operativa y la estabilidad analítica del prototipo:

- **Despliegue Centralizado:** Se opta por una orquestación en un nodo único mediante contenedores. Esta configuración permite validar la cohesión lógica y el enrutamiento a través de un *API Gateway* sin incurrir en la complejidad de red de un entorno distribuido.
- **Comunicación y Resiliencia:** La interacción se basa en el estándar REST con un modelo de E/S no bloqueante. Se prescinde de gestores de mensajería asíncrona para evitar sobreingeniería.
- **Persistencia y MVR:** Bajo el concepto de Reproducibilidad Mínima Viable (MVR), se asegura la trazabilidad científica mediante la persistencia binaria de modelos en un volumen persistente.

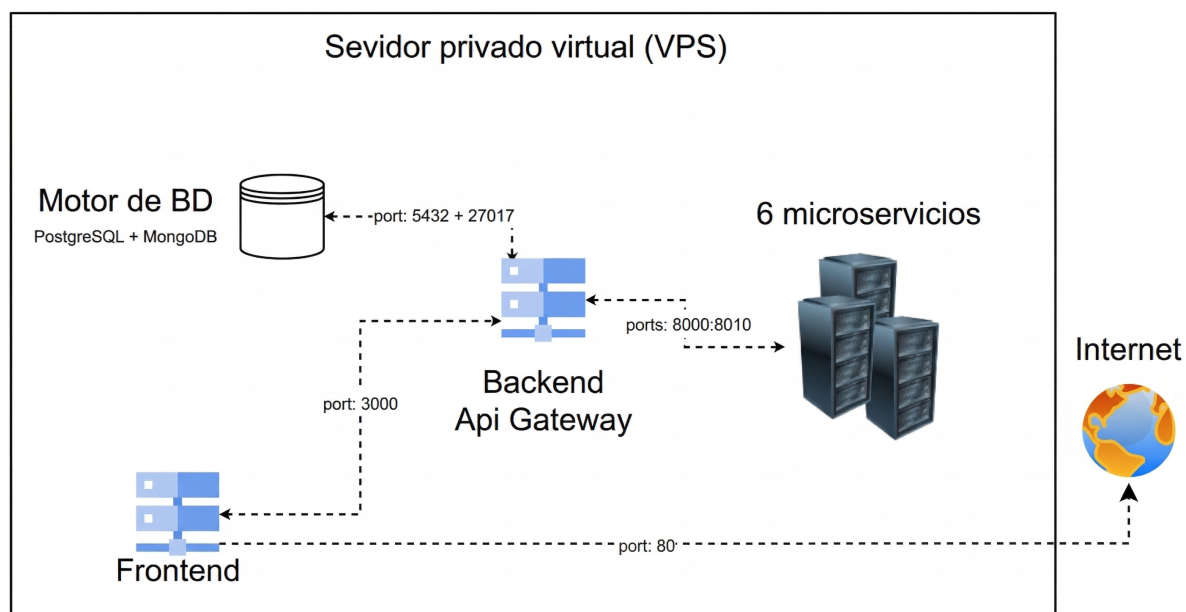


Figura 9. Diagrama general de Arquitectura de plataforma.

Establecidos estos lineamientos de diseño, la arquitectura general fue estructurada para garantizar el desacoplamiento funcional. Como se detalla en la Figura 9, el sistema se organiza en cuatro capas lógicas:

- **Interfaz (Frontend):** Punto de interacción con el usuario. Gestiona la captura de parámetros y la visualización de resultados.
- **Orquestación (API Gateway):** Núcleo coordinador que centraliza las peticiones, asegurando la integridad transaccional y el control de acceso.
- **Capa Analítica (Microservicios):** Seis servicios independientes especializados en las tareas de preprocesamiento, entrenamiento e inferencia.
- **Persistencia Políglota:** Esquema híbrido que utiliza PostgreSQL (puerto 5432) para metadatos y roles, y MongoDB para almacenamiento masivo de datos no estructurados.

La arquitectura de componentes y sus relaciones en el entorno virtualizado se ilustran en la Figura 10.

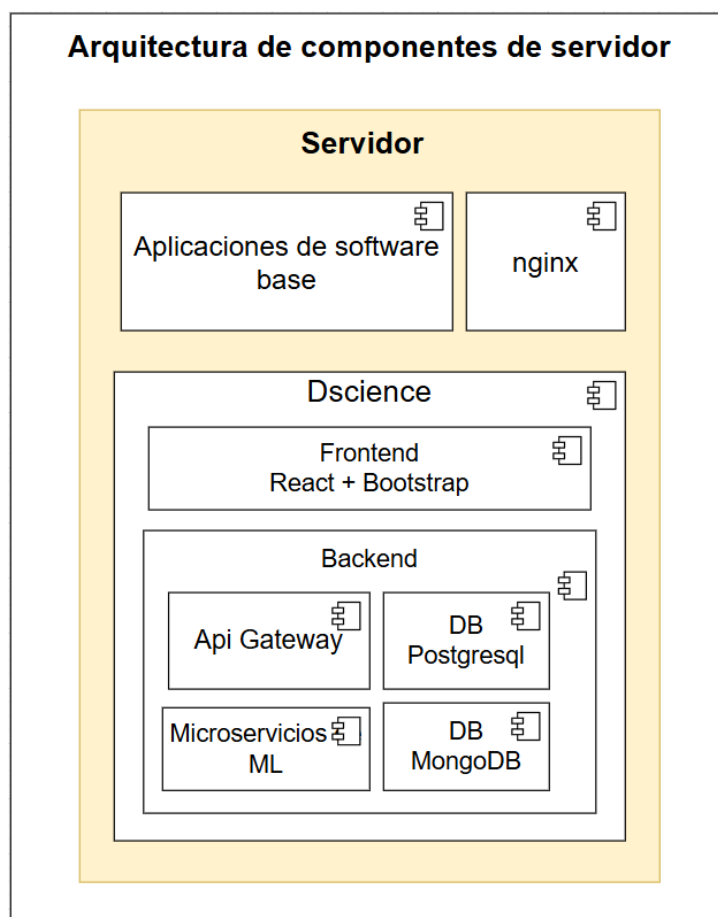


Figura 10. Arquitectura de componentes de servidor.

- **Software Base y Nginx:** Capa de infraestructura que utiliza Nginx como servidor web y proxy inverso para enrutar el tráfico externo hacia los contenedores internos de forma segura.
- **Frontend:** Componente basado en React encargado de renderizar la interfaz de usuario y consumir los servicios expuestos por el API Gateway.
- **Backend:** El API Gateway centraliza la lógica de negocio, coordinando las peticiones hacia los microservicios de Machine Learning y gestionando las transacciones hacia los motores de base de datos.
- **Bases de Datos:** Módulos de persistencia integrados. PostgreSQL asegura la consistencia de metadatos relacionales, mientras que MongoDB provee la flexibilidad necesaria para la trazabilidad y persistencia de datasets masivos.

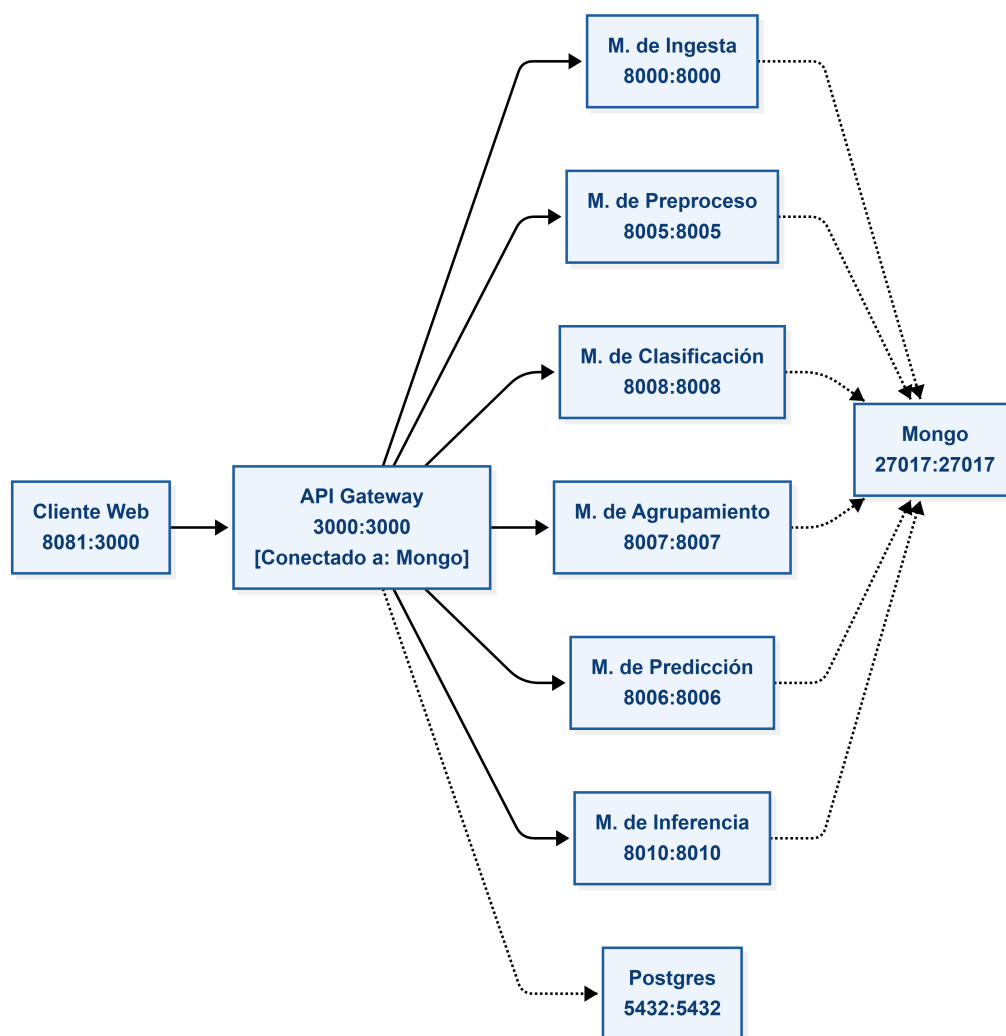


Figura 11. Arquitectura de distribución de microservicios.

En síntesis, esta distribución funcional permite que cada componente evolucione de forma independiente sin comprometer la integridad global. Esta modularidad establece una base técnica robusta para asegurar la trazabilidad y reproducibilidad de los experimentos científicos que se detallarán en las secciones posteriores.

3.2.3. Arquitectura de microservicios

Atendiendo a los requisitos de escalabilidad y resiliencia, se opta por una arquitectura de microservicios donde los servicios analíticos operan independientemente mediante un núcleo coordinador y una interfaz desacoplada. Esta organización se define a través de dos arquitecturas, la Figura 11 presenta la distribución física y el mapeo de puertos, mientras que la Figura 12 detalla la arquitectura funcional de los microservicios.

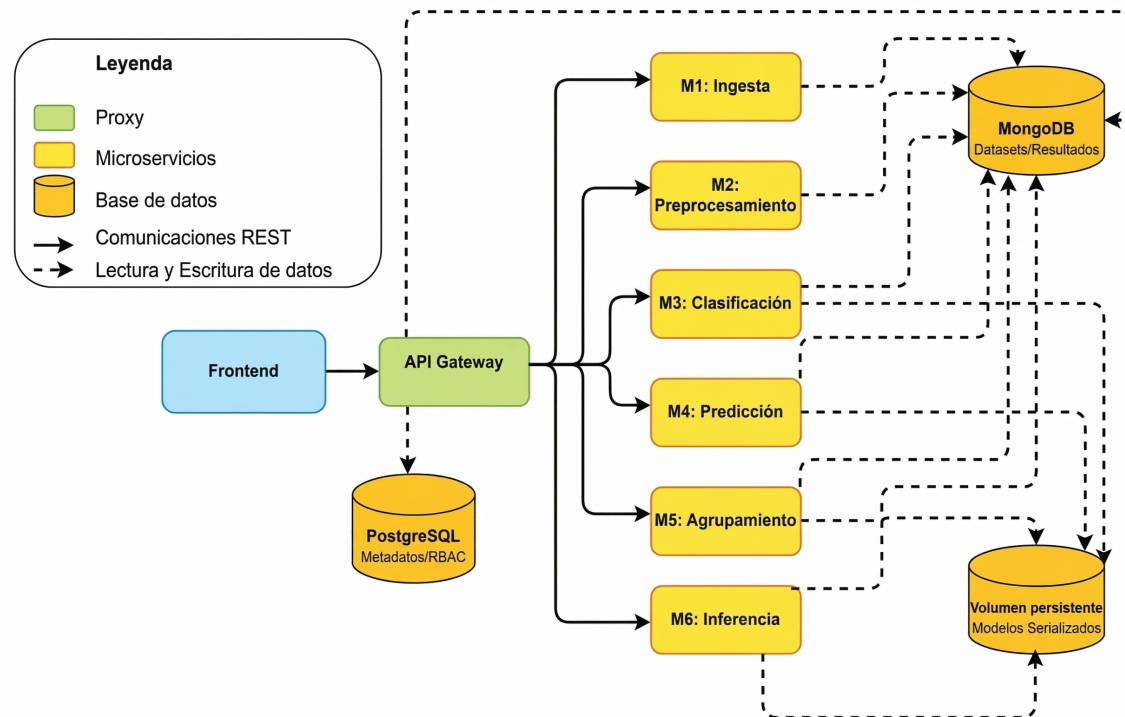


Figura 12. Arquitectura funcional de microservicios.

3.2.3.1. Diseño de Frontend

El diseño de frontend se concibe como una aplicación de página única (SPA) desarrollada en React, lo que permite una interfaz dinámica y reactiva mediante una arquitectura de componentes. El sistema se organiza bajo un contenedor principal de diseño (*Layout*) que distribuye la interfaz en dos paneles base, una *Barra de Navegación Lateral* (Sidebar), encargada de la gestión de rutas y persistencia del menú, y un *Área de Trabajo central*, un contenedor dinámico donde se renderizan los módulos funcionales según el estado de la navegación. tal como se ilustra en la Figura 13.

Esta distribución permite que los componentes base se mantengan estables mientras el área central se actualiza, facilitando el trabajo de los distintos perfiles de usuario y reduciendo la carga cognitiva. A continuación, se detallan los tres núcleos funcionales del sistema: la Interfaz de Entrenamiento (F1), la Interfaz de Inferencia (F2) y el Seguimiento de Experimentos (F3).

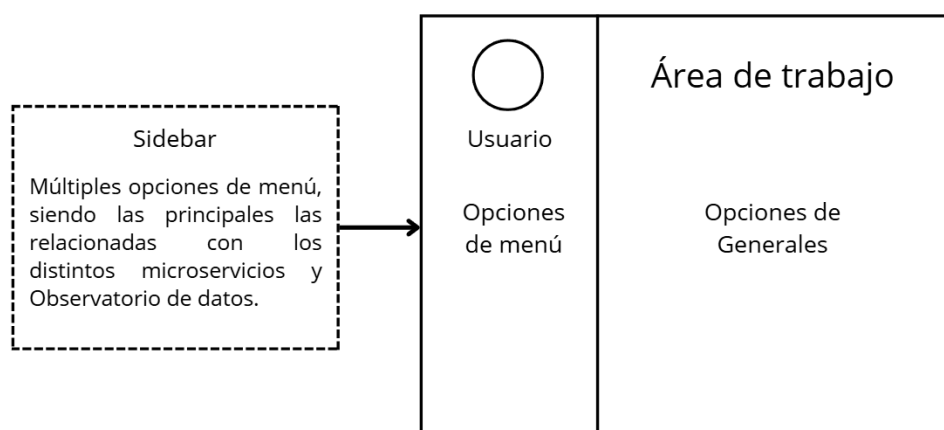


Figura 13. Diseño de Layout.

Plataforma de Ciencia de Datos - Entrenamiento				
Menú Módulos <input checked="" type="button" value="Ingesta **"/> Preprocesado Entrenamiento MLOps/Catálogo	Vista: Datasets Disponibles (MongoDB)			
	Nombre	Registros	Fecha	Acciones
	Dataset_Clinico_v1	15.400	04/03/26	Ver [Eliminar]
	Dataset_Test_v2	2.100	01/03/26	Ver [Eliminar]
	+ Nueva Ingesta de Datos			

Figura 14. Diseño del panel principal del investigador.

F1: Interfaz de Entrenamiento

Este componente de interfaz es para guiar al usuario en la ejecución de algoritmos de aprendizaje automático, desde que se cargan datos en sistema, hasta el proceso de generación y almacenamiento de modelos de ML (machine learning). A continuación se describen los componentes principales de la interfaz de entrenamiento de ML.

- **Panel de entrenamiento:** En esta vista el investigador puede administrar los datos guardados. Como se observa en la Figura 14, el diseño incluye un panel central con tablas que muestran la información básica (metadata) de los archivos almacenados en la base de datos.
- **Módulo de Ingesta:** Esta interfaz permite subir nuevos conjuntos de datos. En la Figura 15 se muestra el diseño de este módulo, presentando una zona central para seleccionar archivos y entrega mensajes al usuario para confirmar la subida.

Módulo Seleccionado: Ingesta y Persistencia Híbrida

Arrastre aquí el archivo

Seleccionar Archivo

archivo_crudo.csv (15.4 MB)

P

Estado: Atomizando registros en MongoDB...

[=====] 45%

Cancelar Iniciar Ingesta

Figura 15. Diseño del módulo de ingesta de datos.

Plataforma de Ciencia de Datos

Módulo de Entrenamiento

Seleccionar Dataset: Dataset Clínico_01

Seleccionar Algoritmo: Predicción: Random Forest

Parámetros

N° Estimadores: 100

Test Size: 0.2

Cancelar Entrenar

Estado (SSE)

Progreso: [#####.....] 85%

Métrica: MSE 0.124

Figura 16. Diseño del módulo de Entrenamiento.

- **Módulo de Entrenamiento de modelos:** Esta interfaz es para la ejecución de entrenamiento de los modelos. Como se ilustra en la Figura 16, el diseño organiza el área de trabajo en tres bloques lógicos: selección de activos (dataset y algoritmo), ajuste de hiperparámetros y un monitor de estado, permitiendo al investigador supervisar el progreso del entrenamiento de forma visual.

F2: Interfaz de Inferencia

A diferencia de los entornos de entrenamiento, orientados a la experimentación técnica, la interfaz de inferencia está optimizada para la explotación operativa de los modelos por parte del usuario final. Se prioriza un diseño minimalista, centrando la interacción exclusivamente en la carga de nuevos conjuntos de datos y la interpretación inmediata de las predicciones generadas por el motor analítico.

Módulo de Inferencia	
Opciones Nueva Inferencia Modelos Disponibles	Selección de Modelo** Catálogo de Modelos: Clasificador_Riesgo_v1.2 v Resumen del Modelo Algoritmo: Random Forest Precisión (Accuracy): 94.2% Cargar Datos para Predecir

Figura 17. Diseño de ejecución de inferencias.

- **Módulo de Inferencia:** Aquí se ejecutan los modelos ya entrenados. Tal como se aprecia en la Figura 17, la navegación muestra las opciones de ejecución de análisis y selección de modelos.
- **Resultados de Predicción:** Una vez que el sistema procesa los datos, el área central muestra los resultados en una tabla. El diseño permite que cada predicción aparezca junto a su porcentaje de confianza, facilitando que el usuario pueda interpretar la información.

F3: Interfaz de seguimiento de ML

Como parte del diseño de la interfaz y la gestión de la información del sistema, se definieron dos esquemas conceptuales encargados de asegurar la trazabilidad de los procesos realizados.

- **Diseño del Módulo de Historial de Entrenamientos:** Esta interfaz, vinculada al **CU-09**, se proyectó como el registro principal de las ejecuciones del sistema. Como se observa en la Figura 18, la vista organiza los registros mediante un panel de filtros superior (algoritmo, estado y rango de fechas) para facilitar la localización de ejecuciones específicas.

La tabla de resultados destaca el identificador único (*ID*), el estado de la tarea y la duración del procesamiento.

Historial de Entrenamientos (Modelos Persistidos)						
Buscar por ID o Modelo...			^Algoritmo^		Desde: 01/03/2026 , Hasta: 10/03/2026	
ID	Algoritmo	Estado	Fecha de Inicio	Duración	Usuario	Modelo
108	RandomForest	COMPLETED	2026-03-10	14m 22s	V. Santos	Ver Artefacto
107	XGBoost	COMPLETED	2026-03-09	08m 10s	P. Gomez	Ver Artefacto
106	LinearReg	COMPLETED	2026-03-08	01m 45s	V. Santos	Ver Artefacto
105	KMeans	COMPLETED	2026-03-07	05m 30s	Admin	Ver Artefacto

Figura 18. Diseño del historial de entrenamientos y búsqueda.

Historial de Inferencias			
Buscar por modelo o usuario...		Search	
Modelo	Fecha Ejecución	Usuario	Acciones
Model_v2_Health	2026-03-10 14:00	⊙ V. Santos	Descargar CSV
Cancer_Pred_01	2026-03-09 11:20	⊙ P. Gomez	Descargar CSV
X_Ray_Analysis	2026-03-08 09:15	⊙ Admin	Descargar CSV

Figura 19. Diseño del historial de inferencias.

- **Diseño del Módulo de Historial de Inferencias:** Esta propuesta de interfaz, permite gestionar el registro de las predicciones ejecutadas. Se planificó la inclusión de un buscador dinámico para encontrar registros por nombre de modelo o usuario responsable.

Tal como se ilustra en el esquema de la Figura 19, el diseño considera el control de acceso basado en roles (RBAC): la vista para el administrador muestra la identidad del usuario que ejecutó la inferencia. Asimismo, se define la ubicación del botón de Descarga de Resultados, permitiendo al investigador recuperar los datasets procesados junto con sus predicciones.

Esta planificación de la interfaz asegura que el flujo de trabajo sea funcional y permita el seguimiento de los datos antes de proceder con la implementación definitiva del software.

3.2.3.2. Diseño de Backend

Una vez definidos los componentes de la capa de presentación y la infraestructura general, es necesario detallar la lógica que gobierna la interacción entre el punto de entrada y los microservicios especializados. En esta arquitectura, el backend se organiza en dos pilares operativos que garantizan la integridad, seguridad y el procesamiento de la información.

B1: API Gateway (Orquestador y Router)

Como núcleo lógico y punto de entrada único bajo un modelo de E/S no bloqueante, el *API Gateway* integra dos funciones críticas:

- **Seguridad y Enrutamiento:** Centraliza la validación de identidad, el control de acceso (RBAC) y la inspección de *payloads* para dirigir eficientemente las peticiones hacia los microservicios correspondientes.
- **Orquestación y Control Transaccional:** Coordina el ciclo de vida de las solicitudes analíticas y centraliza exclusivamente la persistencia en **PostgreSQL**. Al aislar a los microservicios del motor de base de datos, el Gateway normaliza las respuestas para el *Frontend* y asegura la trazabilidad de los experimentos.

Este diseño desacopla la complejidad del aprendizaje automático de la lógica transaccional, facilitando el mantenimiento y garantizando la integridad de la infraestructura.

B2: Persistencia Políglota y Almacén de Artefactos

El éxito de la gestión de información radica en una estrategia de almacenamiento híbrida que separa la administración transaccional, el manejo masivo de datos y el resguardo de los activos analíticos:

- **Modelo Relacional (PostgreSQL):** Gestiona la consistencia de metadatos, la configuración del RBAC y el registro de versiones de los modelos de ML.

- **Modelo Documental (MongoDB):** Almacena los datos no estructurados, tales como *datasets* originales, matrices vectorizadas, métricas de desempeño y resultados de inferencia en formato BSON.
- **Almacén de Artefactos (Volumen Persistente):** Sistema de archivos compartido dedicado a la conservación binaria de los modelos serializados (*.joblib*). Este componente actúa como puente físico, permitiendo que un modelo generado por los servicios de entrenamiento sea cargado dinámicamente por el microservicio de inferencia sin necesidad de re-entrenamiento.

Esta estrategia de persistencia distribuida permite que el sistema maneje flujos de trabajo asíncronos y datos heterogéneos sin comprometer la integridad del estado global de la aplicación.

3.2.3.3. Diseño de servicios de ML

El diseño de la capa analítica se fundamenta en el *Principio de Responsabilidad Única (SRP)* y el aislamiento de procesos mediante microservicios. Esta arquitectura prioriza la *escalabilidad funcional*, facilitando la adición de nuevos modelos predictivos. Además, impone un estricto aislamiento de red frente al motor relacional, ningún microservicio posee conexión directa con PostgreSQL, delegando el registro de metadatos al API Gateway para preservar la integridad transaccional.

A continuación, se describen los seis módulos independientes que componen la capa analítica de la plataforma.

M1: Ingestión

Actúa como punto de entrada, encargándose de la captura y atomización de registros. Asegura la disponibilidad de datos crudos en la base de datos NoSQL (MongoDB) y delega al API Gateway el registro de la referencia y los metadatos del *dataset* en el historial SQL.

M2: Preprocesamiento

Módulo de refinamiento técnico que ejecuta la limpieza, normalización matricial y codificación de variables. Genera vectores numéricos almacenados en NoSQL y notifica al API Gateway para actualizar la referencia y el estado del conjunto de datos procesado en la base relacional.

M3: Clasificación

Servicio de aprendizaje supervisado para tareas de taxonomía. Al finalizar el entrenamiento, persiste las métricas de desempeño en NoSQL, almacena el artefacto serializado (*joblib*) en el volumen persistente y notifica al API Gateway para que actualice el estado del modelo en el registro relacional.

M4: Predicción

Microservicio orientado al modelado de variables continuas y proyecciones de tendencias. Garantiza la reproducibilidad almacenando los resultados técnicos en NoSQL y el modelo físico en el almacenamiento persistente, delegando la actualización del historial al orquestador.

M5: Agrupamiento

Implementa algoritmos de aprendizaje no supervisado para el descubrimiento de estructuras ocultas (*clusters*). Guarda los resultados en NoSQL y el modelo serializado en el volumen persistente, reportando el término de la tarea al API Gateway.

M6: Inferencia

Componente operativo desacoplado que carga modelos serializados para procesar nuevas solicitudes analíticas sin interferir con los entrenamientos activos. Al finalizar la predicción, el servicio retorna el *payload* de resultados al API Gateway, el cual consolida la operación y registra el historial en SQL.

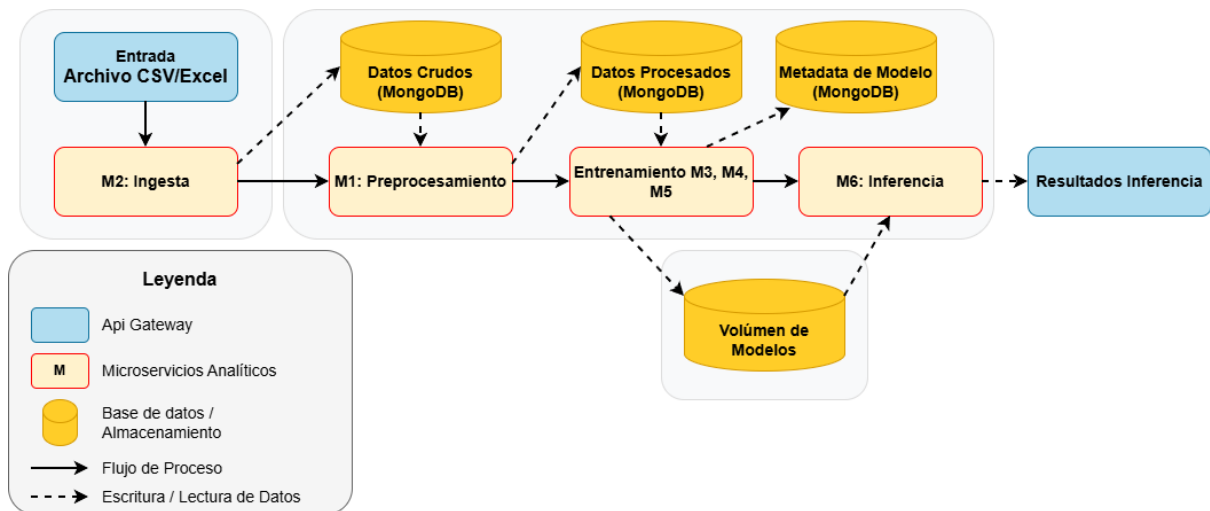


Figura 20. Pipeline de servicios de procesamiento, entrenamiento e inferencia.

Como síntesis de las interacciones descritas, la Figura 20 presenta el *pipeline* detallado de los microservicios, ilustrando el flujo secuencial de los activos de información y su persistencia entre los distintos servicios analíticos.

3.2.4. Arquitectura de Despliegue

La arquitectura de despliegue define la materialización física de los componentes diseñados, estableciendo la integración de los microservicios y motores de persistencia en el entorno operativo. Como se ilustra en la Figura 21, se presenta como un esquema centralizado mediante contenedorización en un *Servidor Privado Virtual* (VPS), garantizando la portabilidad.

Esta configuración va acompañada de las siguientes decisiones de ingeniería:

- **Virtualización y Orquestación:** Se emplea *Docker* y *Docker Compose* para definir la infraestructura, permitiendo gestionar el ciclo de vida de los servicios (M1 a M6 y el orquestador) de forma independiente. El uso de contenedores independientes evita conflictos de dependencias y asegura que el sistema sea reproducible en cualquier hardware compatible.

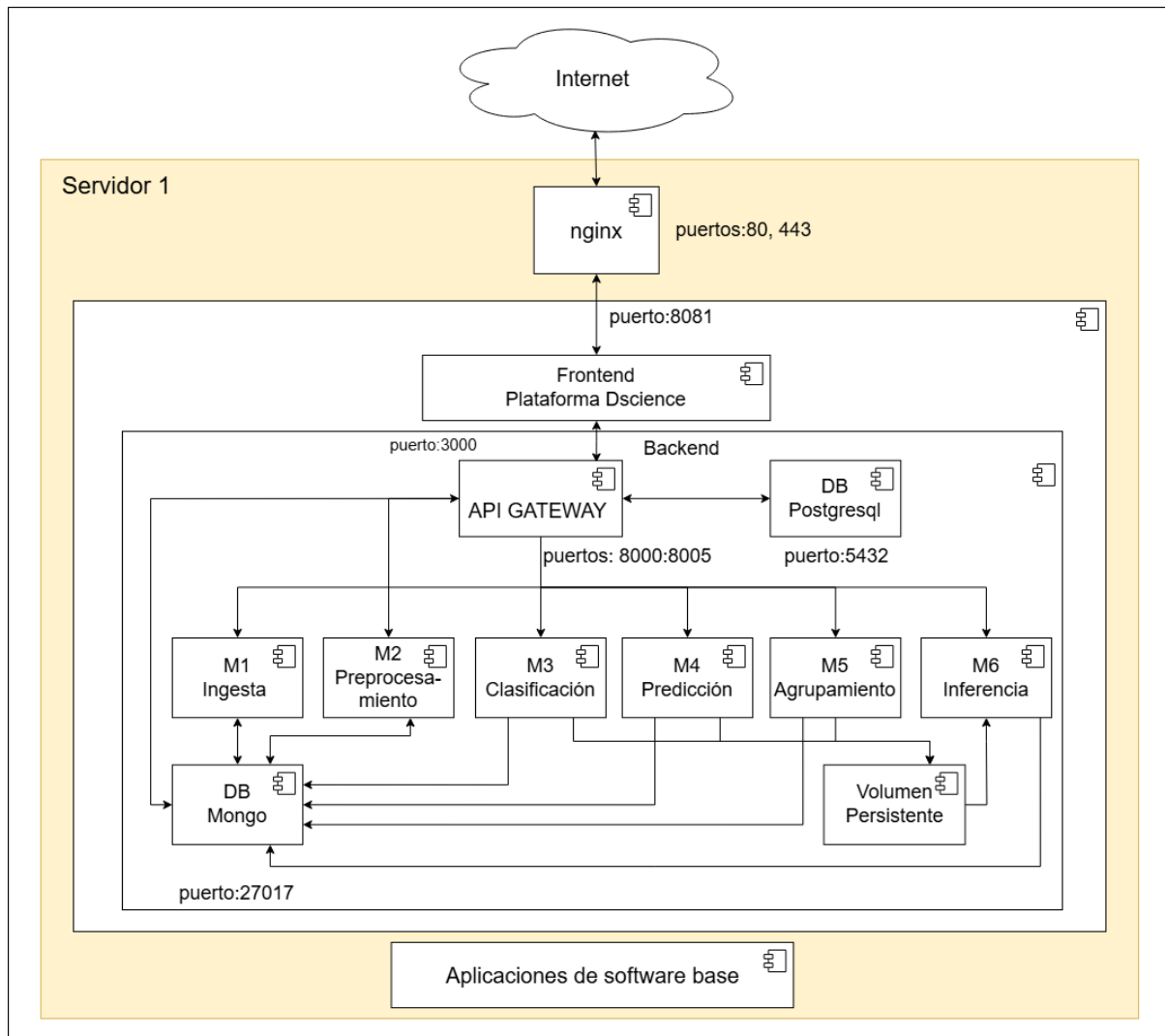


Figura 21. Arquitectura de despliegue de la plataforma.

- **Seguridad y Enrutamiento:** Un servidor **Nginx** actúa como proxy inverso gestionando el tráfico externo y el cifrado SSL/TLS. Este aísla los microservicios y bases de datos, manteniéndolos protegidos en una red interna privada de Docker, sin acceso desde el internet público.
- **Gestión de Persistencia y Recursos:** Al desplegarse en un nodo único, los servicios comparten la CPU y RAM del VPS. Para mitigar la pérdida de datos, se emplean **Volúmenes de Docker** que desacoplan la información del ciclo de vida de los contenedores. Esto asegura la persistencia en SQL y NoSQL, habilitando un almacén compartido para que los modelos serializados (*.joblib*) fluyan entre entrenamiento e inferencia.

En síntesis, esta arquitectura física proporciona un entorno aislado y resiliente, optimizando los recursos disponibles del servidor único y estableciendo la base técnica para una futura transición hacia clústeres de orquestación distribuida.

3.3. Plan de Pruebas y QA

Para certificar la fiabilidad de la plataforma, la estrategia de Aseguramiento de la Calidad (QA) se estructura bajo el paradigma de la pirámide de pruebas, permitiendo una validación sistemática en tres niveles:

1. **Nivel Base (Pruebas Unitarias):** Validación de algoritmos y lógica matemática mediante **Jest** y **Pytest**. Asegura la integridad de los procesos de ciencia de datos antes de su empaquetado.
2. **Nivel Medio (Pruebas de Integración):** Auditoría de la interoperabilidad entre el *API Gateway*, servicios analíticos y persistencia. Se emplea **Postman** para validar esquemas JSON y códigos de respuesta HTTP.
3. **Cúspide (Pruebas E2E y de Carga):** Emulación integral del usuario y pruebas de estrés mediante **Apache JMeter**. Evalúa la tolerancia a fallos, la latencia de inferencia y la escalabilidad del clúster ante ráfagas de tráfico.

Para validar la viabilidad de este prototipo, el diseño experimental se rige por tres dimensiones métricas de éxito: **eficacia**, **eficiencia** y **trazabilidad** bajo condiciones controladas. Estos criterios conforman las tres dimensiones de evaluación que se detallan a continuación.

3.3.1. Pruebas de eficacia

Estas pruebas verifican que el sistema cumpla sus objetivos funcionales de procesamiento y persistencia bajo dos criterios fundamentales:

- **Flexibilidad Estructural:** Se valida la capacidad de la arquitectura para almacenar conjuntos de datos con esquemas variables y metadatos dinámicos

(como hiperparámetros) sin requerir migraciones manuales o cambios en la estructura del motor relacional.

- **Integridad de la Operación:** Asegura que los módulos de *machine learning* retornen resultados consistentes y coherentes con los datos de entrada, garantizando que el flujo de procesamiento analítico sea fiable y preciso.

3.3.2. Pruebas de eficiencia

Estas pruebas validan el rendimiento técnico y la capacidad de respuesta de la arquitectura bajo carga. Los criterios de éxito definidos son:

- **Latencia de Enrutamiento:** El tiempo de interceptación, validación (RBAC) y redirección por el *API Gateway* no debe superar los 500 ms en el percentil 95 (P_{95}), incluyendo la recuperación de metadatos.
- **Respuesta de Módulos ML:** Para tareas de inferencia o procesamiento intensivo, se establece un rango de 1 a 5 segundos. Este margen absorbe la complejidad computacional del modelo y la carga de datos en memoria secundaria.
- **Eficiencia de Consultas:** La reconstrucción de información entre motores (relacional y documental) debe mantener una latencia total inferior a 500 ms para asegurar la interactividad del sistema.
- **Resistencia a la Carga Concurrente:** Mediante pruebas de estrés, el sistema debe soportar ráfagas de al menos 50 peticiones simultáneas sin pérdida de datos ni errores de servidor (HTTP 500/503). Esto valida la efectividad del desacoplamiento frente a picos de demanda.

3.3.3. Pruebas de trazabilidad

Constituyen el pilar de la reproducibilidad científica. Dado que una decisión central del diseño es la segmentación de datos en modelos relacionales y no relacionales, la

validación de este componente consiste en una auditoría funcional capaz de reconstruir el linaje de datos a través de ambos motores:

1. **Origen:** Identificación unívoca del usuario responsable mediante el *token* de sesión (SQL).
2. **Configuración:** Recuperación exacta de los hiperparámetros y la versión del modelo utilizado (NoSQL/SQL).
3. **Linaje:** Rastreo del flujo desde el resultado final de la inferencia hasta el registro original en el motor documental.

Si la arquitectura logra reconstruir este flujo sin pérdida de información, se justificará que la separación políglota aporta la flexibilidad necesaria y se considerará exitosa en su objetivo de asegurar la integridad de los procesos de investigación.

3.4. Modelo de ingeniería de software

El marco de trabajo para el desarrollo tecnológico se fundamentará en la metodología ágil Scrum, tal como se evaluó en la Sección 2.4.1. Este enfoque iterativo es mandatorio para sistemas modulares y plataformas de ciencia de datos, ya que el ciclo continuo de entregas minimiza los riesgos de integración algorítmica y facilita la adaptación ante requerimientos investigativos emergentes.

La planificación del proyecto se organiza en 11 *Sprints* secuenciales, utilizando la herramienta de gestión colaborativa OpenProject para administrar incidencias, rastrear el progreso y documentar decisiones de diseño. Esta metodología permite el avance de las múltiples capas del sistema, desde la orquestación hasta la consolidación del *pipeline* de aprendizaje automático. El desarrollo cronológico se representa visualmente en la Figura 22.

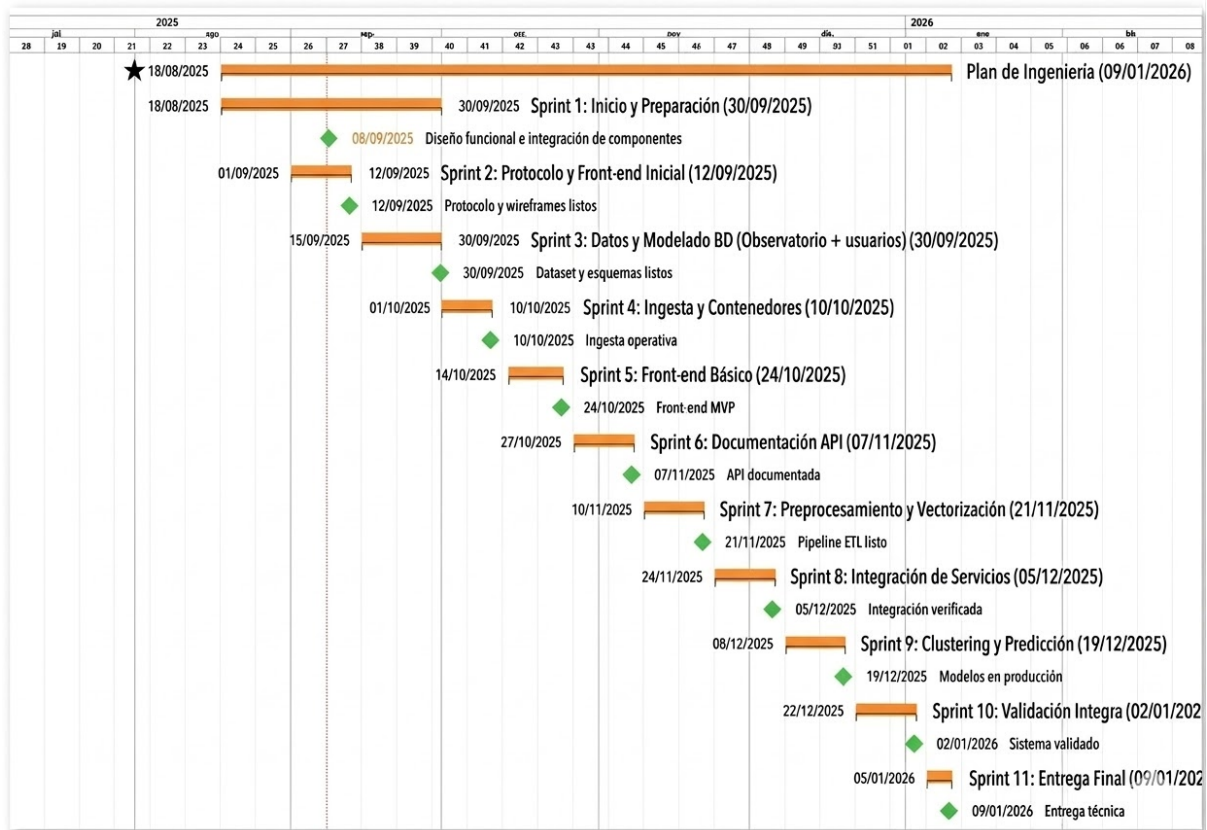


Figura 22. Carta Gantt.

3.5. Conclusiones del capítulo

En este capítulo se presentaron los fundamentos para una arquitectura escalable de ciencia de datos. El diseño propuesto funciona como el plano maestro que permite pasar de requerimientos detallados a una arquitectura de microservicios y un esquema de persistencia políglota. El modelado de casos de uso y la estructura funcional de los microservicios permiten que los experimentos se puedan rastrear y repetir fácilmente, lo cual es relevante en la investigación colaborativa e interdisciplinar.

Asimismo, el plan de pruebas y los criterios de éxito garantizan que el prototipo funcione bien ante distintos tipos de datos y tareas. Finalmente, el uso de Scrum y la planificación en 11 sprints aseguran que el proyecto sea viable metodológicamente de desarrollar e implementar.

Capítulo 4. Desarrollo, Implementación y Despliegue

En este capítulo se detalla el proceso de desarrollo, implementación y despliegue del sistema propuesto, implementando el diseño establecido en el Capítulo 3.

La Sección 4.1 describe las herramientas de desarrollo. En la Sección 4.2 se aborda el desarrollo e implementación. En la Sección 4.3 se explica el empaquetamiento y despliegue. Finalmente, en la Sección 4.4 se presentan las conclusiones del capítulo.

4.1. Herramientas de desarrollo

Para la gestión del código fuente se adoptó una estrategia de repositorio unificado o Monorepo. Este enfoque centraliza la orquestación de la infraestructura y facilita la integración continua bajo la metodología ágil (*Scrum*) detallada en la Sección 3.4.

Su implementación garantiza la atomicidad de los despliegues a través de todas las capas del sistema (Frontend, Backend y Microservicios). Específicamente en el contexto MLOps, el Monorepo asegura que cualquier modificación en los modelos de machine learning se sincronice simultáneamente con el orquestador y la interfaz, manteniendo la consistencia global de la plataforma en cada iteración.

El desarrollo del software se realizó bajo el marco institucional del Centro de Desarrollo, Experimentación y Transferencia de Tecnología Educativa (CEDETEC), unidad de investigación y desarrollo de la Universidad de Santiago de Chile enfocada en la gestión del conocimiento y la ingeniería de software científico. Alineado con

los objetivos de innovación tecnológica, el código fuente íntegro, la configuración de contenedores y los artefactos de despliegue se encuentran alojados y disponibles en el repositorio oficial de CEDETEC: <https://github.com/cedetec/dscience-platform>.

4.2. Desarrollo e implementación

La implementación del sistema se abordó mediante una estrategia de desarrollo desacoplado, permitiendo que cada capa lógica evolucione de forma independiente antes de su integración final en la red de contenedores.

4.2.1. Desarrollo de Frontend

El desarrollo de la capa de presentación se basa en una arquitectura de aplicación de página única (*SPA*) construida con la biblioteca *React*. La distribución general se organiza mediante un componente de diseño base (*Layout*) que gestiona un menú lateral persistente y un área de trabajo central dinámica. Esta estructura permite que el estado de la aplicación se mantenga fluido mientras los distintos módulos funcionales se renderizan según la interacción del usuario.

4.2.1.1. Panel de control de acceso (RBAC)

El acceso a la plataforma está mediado por una interfaz de autenticación centralizada, presentada en la Figura 23. Esta vista representa el punto de entrada al flujo de seguridad, donde el API Gateway valida la identidad del usuario y gestiona la persistencia de la sesión mediante tokens JWT, tal como se explicó en la Sección 3.2.3 en que se presentó la arquitectura funcional de microservicios, en la Figura 12.

4.2.1.2. Panel de gestión de experimentos

Como se describió en la Sección 3.2.3.1, el investigador requiere un control exhaustivo sobre los activos de datos para garantizar la trazabilidad de los experimentos. En la Figura 24 se presenta la interfaz del panel de gestión, la cual actúa como el centro de

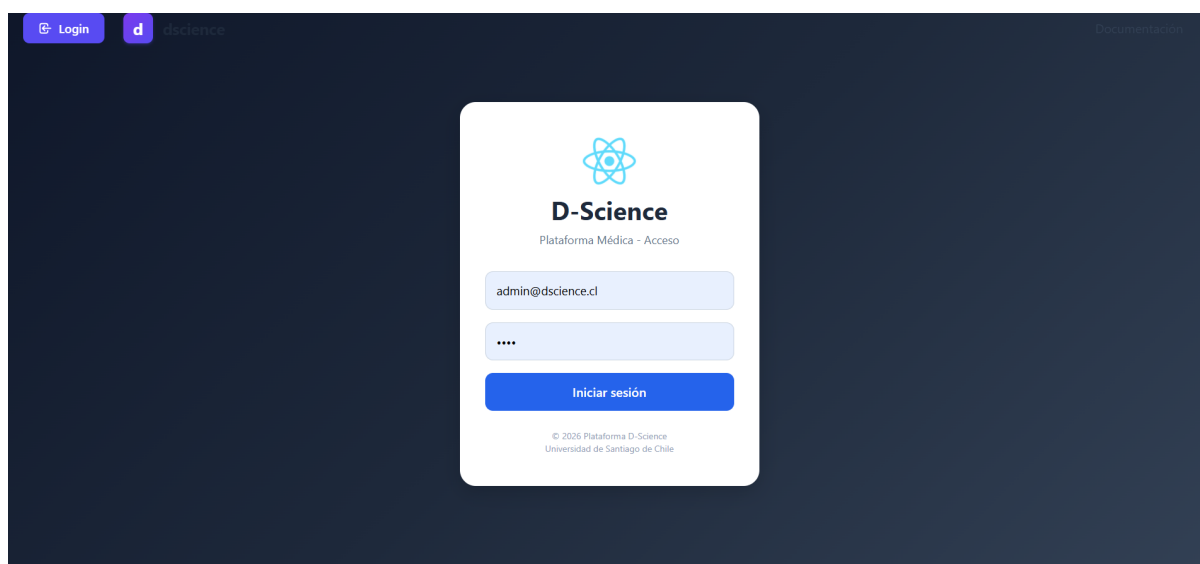


Figura 23. Interfaz de autenticación y control de acceso (RBAC).

monitoreo principal. Esta vista renderiza el inventario de archivos disponibles antes de iniciar un flujo analítico.

Complementariamente, el componente de ingesta (Figura 25) permite la carga de datos definida en el microservicio **M1**. Para asegurar una experiencia de usuario fluida durante el procesamiento de archivos, se implementó una lógica de retroalimentación visual mediante estados de carga y notificaciones reactivas. Estos componentes informan al usuario sobre el éxito de la persistencia en **MongoDB** tras la ejecución del proceso y la creación de las referencias en el esquema relacional.

Finalmente, el módulo de preprocesamiento, ilustrado en la Figura 26, permite la manipulación directa de los *datasets* crudos para su adecuación a las exigencias de los microservicios de aprendizaje automático. Esta interfaz facilita la ejecución de tareas de limpieza y transformación de variables, interactuando con el microservicio **M2** para generar colecciones de datos procesados que aseguren la compatibilidad técnica con los algoritmos de clasificación, predicción o agrupamiento.

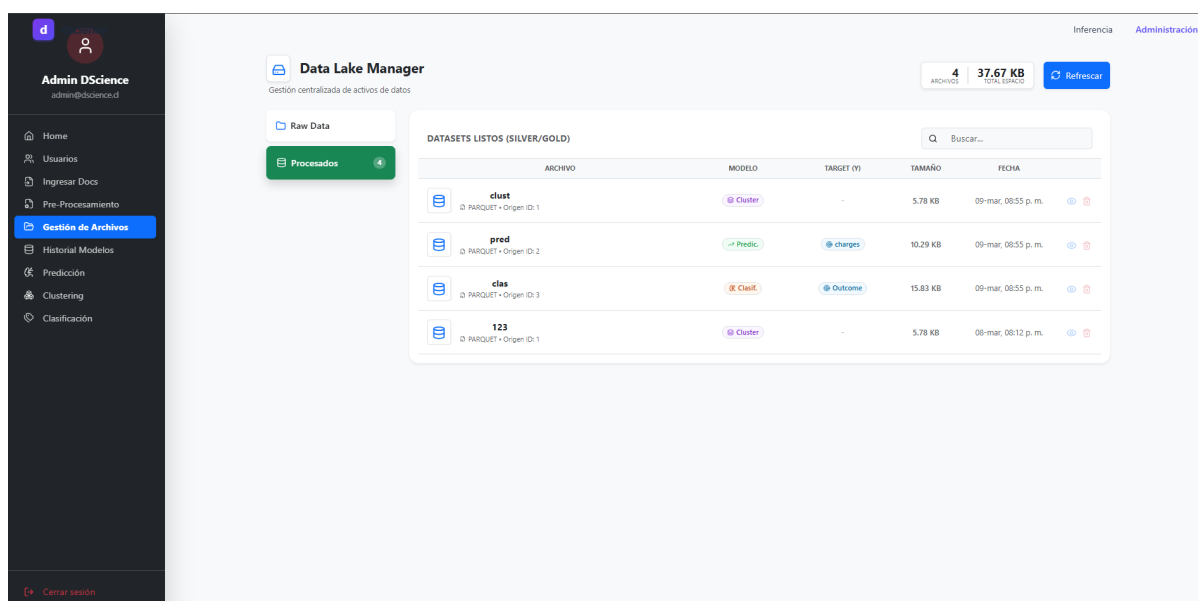


Figura 24. Panel de control principal del investigador.

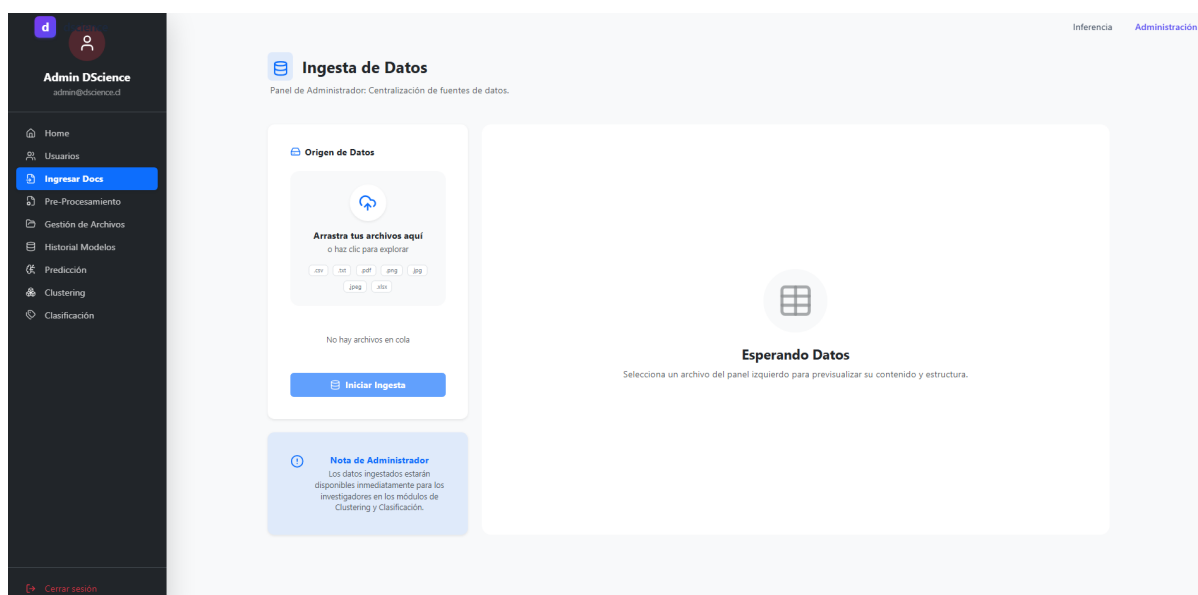


Figura 25. Interfaz funcional de ingesta de datos.

4.2.1.3. Interfaz de entrenamiento de ML

Para ejecutar los procesos de ciencia de datos, se desarrollaron tres componentes independientes para tareas de Predicción, Clasificación y Agrupamiento. Aunque se trata de componentes desacoplados a nivel de código para responder a sus respectivos microservicios, comparten una estructura visual homóloga. Este diseño busca estandarizar la curva de aprendizaje del investigador, permitiéndole operar distintas técnicas de *machine learning* bajo un mismo flujo de trabajo.

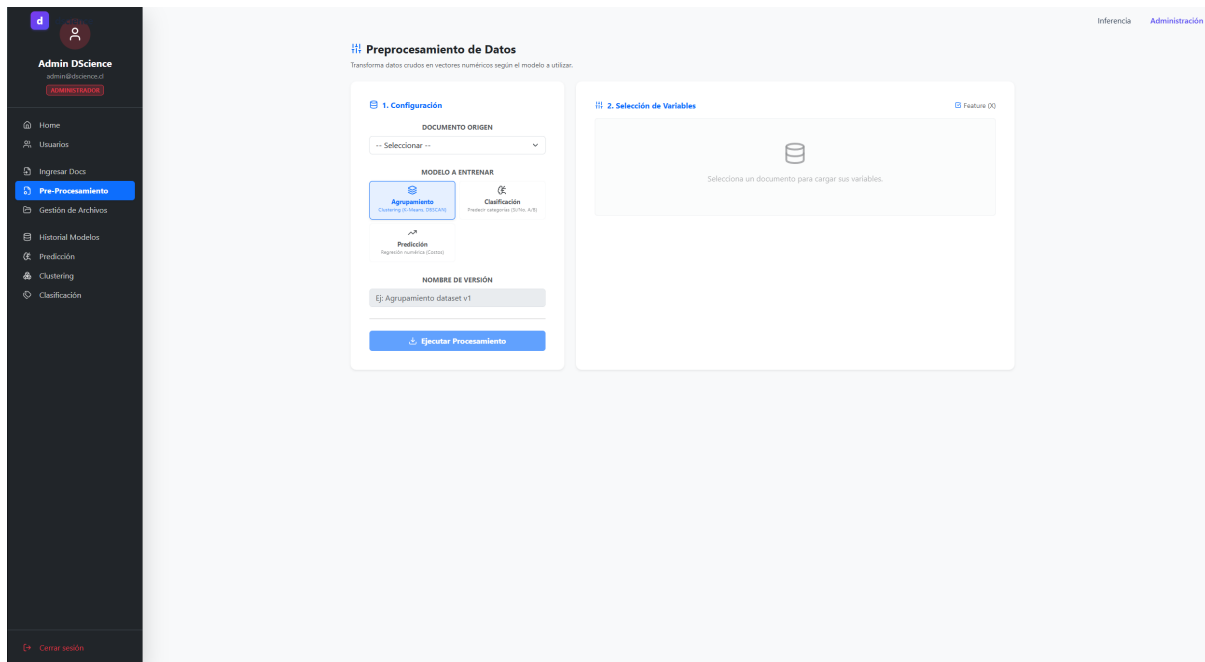


Figura 26. Interfaz funcional de preprocesamiento de datos.

En la **Figura 27** se ilustran las tres variantes de la interfaz: (a) Predicción, (b) Clasificación y (c) Agrupamiento. En ellas se organizan las siguientes funciones comunes:

- **Configuración de Variables e Hiperparámetros:** Espacio dedicado a la selección de columnas del preprocesado y el ajuste manual de los parámetros técnicos del algoritmo seleccionado (como $n_estimators$ o $epochs$).
- **Control de Ejecución y Persistencia:** Botones de acción diferenciados para invocar el entrenamiento y guardar el artefacto (*.joblib*) en el sistema de gestión de modelos.
- **Monitorización en Tiempo Real:** Panel de streaming visual que consume eventos del servidor (SSE), permitiendo observar métricas como el error o la inercia durante el proceso.

Inferencia Administración

Predicción Numérica

Entrenamiento de modelos de regresión (valores continuos).

Guardar Modelo

1. Dataset

CostoMaquinaria
3.89 KB

california_housing
745.51 KB

Columna Objetivo
costo_mantenimiento

2. Configuración

ALGORITMO
Random Forest Regressor

TRAIN/TEST SPLIT
80%

EPOCHS
50

Iniciar Entrenamiento

Pérdida (Loss)

0.0000

MSE ACTUAL

Esperando datos...

(a) Interfaz de entrenamiento para Predicción.

Inferencia Administración

Clasificación Supervisada

Entrenamiento de modelos predictivos categóricos.

Guardar Modelo

1. Dataset

load_iris
4.03 KB

b2
72.38 KB

Columna Objetivo (Target)
target

2. Configuración

ALGORITMO
Random Forest (Ensamble)

SPLIT TRAIN/TEST
80%

Iniciar Entrenamiento

Monitor de Entrenamiento

Esperando inicio...

0.0%

ACCURACY ACTUAL

Inicia el entrenamiento para ver métricas en tiempo real

(b) Interfaz de entrenamiento para Clasificación.

Inferencia Administración

Clustering en Tiempo Real

Entrenamiento y etiquetado de datos no supervisado.

1. Dataset

grupoGenesZ (75.47 KB)

Selecciona un dataset procesado sin variable Target.

2. Configuración

Algoritmo
K-Means Clustering

Clusters (K)
3

Max Iter
100

Iniciar Entrenamiento

Monitor de Entrenamiento

Esperando inicio...

0.00

INERCIAS / RUIDO

Inicia el análisis para ver métricas

Mapa de Clusters (PCA)

Esperando resultados...

(c) Interfaz de entrenamiento para Agrupamiento.

Figura 27. Componentes de experimentación y entrenamiento.

4.2.1.4. Interfaz de inferencia

Esta interfaz constituye el punto de interacción final para el usuario no técnico, diseñada para la explotación operativa de los modelos alojados en el catálogo del sistema. Se priorizó un diseño de abstracción técnica que permite cargar nuevos conjuntos de datos y obtener resultados de los modelos de machine learning previamente entrenados de manera simple e intuitiva.

Dependiendo del microservicio analítico invocado, la interfaz adapta la visualización de los resultados para responder a la naturaleza del problema. En la **Figura 28** se presentan las tres variantes funcionales.

- **Inferencia de Clasificación (M3):** Como se observa en la Figura 28a, el sistema renderiza la etiqueta categórica asignada a cada registro junto con su respectiva probabilidad de pertenencia o porcentaje de confianza, facilitando la toma de decisiones basada en clases discretas.
- **Inferencia de Predicción (M4):** Esta variante, ilustrada en la Figura 28b, se especializa en la exposición de valores numéricos continuos. La interfaz presenta las estimaciones resultantes del modelo de regresión, permitiendo la comparación visual entre las tendencias proyectadas.
- **Inferencia de Agrupamiento (M5):** A diferencia de los módulos anteriores, el resultado del agrupamiento no supervisado consiste en la asignación de cada instancia a un segmento específico. La Figura 28c muestra cómo la plataforma identifica y etiqueta los clústeres a los que pertenece cada dato, revelando estructuras ocultas en el dataset de entrada.

The screenshot shows the inference interface for the 'Clasificación: Clasificador de diabetes' model. At the top right, there are tabs for 'Inferencia' and 'Administración'. A link 'Volver a mis modelos' is at the top left. The model name is 'Clasificación: Clasificador de diabetes' with a version number 'v.1773940927267'. Below the name, it says 'Sube nuevos datos para que el modelo asigne etiquetas automáticamente.' There is a section for 'Archivo de Entrada (CSV/Excel)' with a button 'Seleccionar archivo' and a text field 'Ningún archivo seleccionado'. A yellow 'Clasificar' button is on the right.

(a) Interfaz de ejecución de inferencias para Clasificación.

The screenshot shows the inference interface for the 'Predicción: Costo Maquinaria' model. At the top right, there are tabs for 'Inferencia' and 'Administración'. A link 'Volver a mis modelos' is at the top left. The model name is 'Predicción: Costo Maquinaria' with a version number 'v.1773942171030'. Below the name, it says 'Sube un archivo CSV/Excel con las mismas columnas usadas en el entrenamiento.' There is a section for 'Archivo de Entrada' with a button 'Seleccionar archivo' and a text field 'Ningún archivo seleccionado'. A blue 'Predecir Valor' button is on the right. Below the text field, it says 'Máximo 20MB. Requiere cabecera con columnas.'

(b) Interfaz de ejecución de inferencias para Predicción.

The screenshot shows the inference interface for the 'Segmentación: kmediasGenes' model. At the top right, there are tabs for 'Inferencia' and 'Administración'. A link 'Volver a mis modelos' is at the top left. The model name is 'Segmentación: kmediasGenes' with a version number 'v.1773942171030'. Below the name, it says 'Asigna nuevos datos a los grupos (clusters) existentes.' There is a section for 'Archivo de Nuevos Clientes/Datos (CSV/Excel)' with a button 'Seleccionar archivo' and a text field 'Ningún archivo seleccionado'. A green 'Asignar Grupos' button is on the right.

(c) Interfaz de ejecución de inferencias para Agrupamiento.

Figura 28. Interfaces de ejecución de inferencias.

Historial de Entrenamientos / Modelos

Filtro por tipo y fecha, y revisa hiperparámetros y métricas.

Buscar: Nombre, versión o Run ID... Tipo de modelo: Todos Estado: Todos Desde: dd/mm/aaaa Hasta: dd/mm/aaaa

Mostrando 4

MODELO	TIPO	VERSIÓN	INICIO	ACCIÓN
test3 Run ID: 4	CLASSIFICATION	v.1773100543982	9/3/2026, 20:55:43	Ver
test22 Run ID: 3	CLUSTERING	v.1773100536973	9/3/2026, 20:55:36	Ver
test Run ID: 2	PREDICTION	v.1773100527540	9/3/2026, 20:55:27	Ver
123 Run ID: 1	CLUSTERING	v.1773011532881	8/3/2026, 20:12:32	Ver

test3

CLASSIFICATION v.1773100543982 Estado: SUCCESS Cerrar

Inicio: 9/3/2026, 20:55:43 Fin: 9/3/2026, 20:55:43

Dataset: clas Files / Features: 768 / 8

Target: target_data Duración: 0s

Hiperparámetros

```
{
  "testsize": 0.2,
  "algorithm": "rft",
  "targetcolumn": "outcome"
}
```

Métricas / Resultados

```
{
  "accuracy": 0.7597482097482097,
  "f1_macro": 0.7274597824597825,
  "f1_micro": 0.7274597824597825,
  "precision_macro": 0.7274597824597825,
  "precision_micro": 0.7274597824597825,
  "recall_macro": 0.7274597824597825,
  "recall_micro": 0.7274597824597825,
  "confusion_matrix": {
    "labels": 1
  }
}
```

Figura 29. Interfaz del historial de entrenamientos.

Historial de Inferencias

Panel de Administración Visualizando actividad global.

Buscar modelo o usuario...

FECHA	USUARIO	MODELO UTILIZADO	TIPO DE TAREA	ACCIONES
9/3/2026 21:18:34	Admin DScience admin@ds-science.cl	test Versión v.1773100527540	PREDICTION	Descargar Resultados
9/3/2026 20:56:49	Admin DScience admin@ds-science.cl	test3 Versión v.1773100543982	CLASSIFICATION	Descargar Resultados
9/3/2026 20:56:24	Admin DScience admin@ds-science.cl	test22 Versión v.1773100536973	CLUSTERING	Descargar Resultados
9/3/2026 20:56:10	Admin DScience admin@ds-science.cl	test3 Versión v.1773100543982	CLASSIFICATION	Descargar Resultados
9/3/2026 20:55:59	Admin DScience admin@ds-science.cl	test Versión v.1773100527540	PREDICTION	Descargar Resultados

Mostrando 5 registros de inferencia.

Figura 30. Interfaz de auditoría de inferencias.

4.2.1.5. Interfaz de seguimiento

Materializa la trazabilidad del sistema mediante los componentes de historial, presentados en la Figura 29 y Figura 30. Estos componentes permiten la auditoría técnica de los experimentos y la recuperación de resultados históricos almacenados en la base de datos documental.

4.2.2. Desarrollo de Backend

El backend se desarrolló como un ecosistema coordinado por un API Gateway centralizado, cuya responsabilidad es la gobernanza de las transacciones y la seguridad de la información, así como la orquestación de los microservicios.

4.2.2.1. Desarrollo del Orquestador (API Gateway)

Este componente central unifica las capacidades de enrutamiento y orquestación del ecosistema. Su responsabilidad principal es la resolución de peticiones HTTP, la validación de tokens JWT para el cumplimiento del RBAC y el enrutamiento inteligente hacia los microservicios de machine learning. Se desarrolló utilizando Node.js, asumiendo la tarea de exponer los *endpoints* técnicos del sistema, los cuales se presentan en la Tabla 3. También se encarga de validar la carga útil (*payload*) de cada petición y redirigir el tráfico hacia los contenedores especializados según el tipo de tarea solicitada, como fue diseñado en la Sección 3.2.3.2.

Tabla 3. Contratos de interfaz RESTful del sistema.

Método	Ruta (Endpoint)	Responsabilidad Arquitectónica
POST	/api/auth/login	Validación de credenciales (PostgreSQL) y generación de JWT para RBAC.
GET	/api/usuarios	Listado de usuarios y privilegios para gestión administrativa.
POST	/api/ingesta/upload	Ingesta de archivos y delegación de atomización a Python/MongoDB.
GET	/api/ingesta/documentosprocesados	Recuperación de metadatos de datasets vectorizados para IA.
DELETE	/api/ingesta/documentos/:id	Eliminación coordinada de registros en SQL y colecciones en MongoDB.
POST	/api/preprocessing/procesar	Limpieza matricial y generación de mappings en motor NoSQL.

Método	Ruta (Endpoint)	Responsabilidad Arquitectónica
POST	/api/modelos/guardar	Flujo MLOps: serialización (.joblib), autoría y versionado en SQL.
POST	/api/modelos/inferir	Ejecución de predicción y persistencia de resultados para auditoría.
GET	/api/infra/docker-stats	Telemetría en tiempo real (CPU/RAM) vía socket de Docker.
GET	/api/modelos/training-runs	Auditoría técnica de experimentos (métricas, duración y autoría).
GET	/api/modelos/inferencias	Historial de uso con filtrado dinámico según <i>rol_id</i> .
GET	/api/modelos/inferencias/:id/detalle	Recuperación de datos volumétricos (MongoDB) mediante referencia SQL.
POST	/api/modelos/prediction	Proxy SSE para métricas de regresión en tiempo real.
POST	/api/modelos/classification	Proxy SSE para métricas de entrenamiento.
POST	/api/modelos/clustering	Proxy SSE para flujo asíncrono de modelos KMeans/DBSCAN.

En esta capa también se implementó la lógica de agregación para los historiales de auditoría. El orquestador coordina consultas para recuperar los metadatos transaccionales de PostgreSQL y los combina con los datos volumétricos almacenados en MongoDB, entregando una respuesta JSON unificada que alimenta a las interfaces de presentación del sistema.

4.2.2.2. Desarrollo de Bases de Datos

La estrategia políglota diseñada conceptualmente en la Sección 3.2.3.2 fue materializada mediante la implementación conjunta de PostgreSQL y MongoDB.

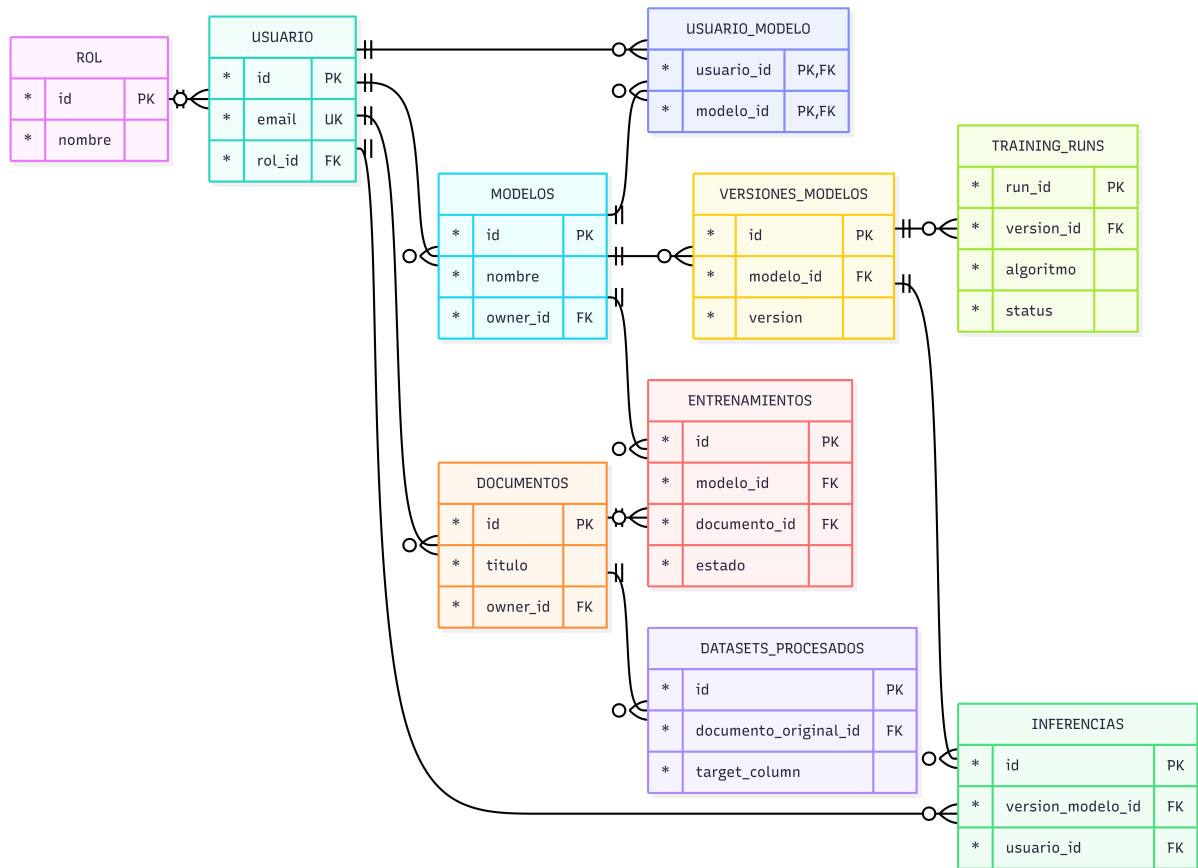


Figura 31. Esquema físico de la base de datos relacional.

En PostgreSQL, se construye el esquema relacional físico que soporta el núcleo administrativo, la gestión de usuarios y la trazabilidad de los artefactos del API Gateway. Para lograr esto, se implementaron ocho tablas principales. El diseño de este esquema aplica restricciones de llaves foráneas para garantizar la integridad referencial y prevenir la existencia de registros huérfanos durante el ciclo de vida de la plataforma. En la Figura 31 se presenta una representación simplificada del esquema relacional físico que soporta el núcleo administrativo del sistema. La especificación técnica completa, incluyendo el script de creación de tablas (DDL) y el detalle exhaustivo de cada campo, se encuentra disponible en el **Anexo G**. En la Tabla 4 se describe el propósito y la responsabilidad técnica de cada entidad implementada en la base de datos relacional.

Por otro lado, la organización de los datos no estructurados se visualiza en la Figura 32, donde se detalla la estructura de colecciones implementada usando el motor NoSQL MongoDB. El detalle de las colecciones se presenta en la Tabla 5.

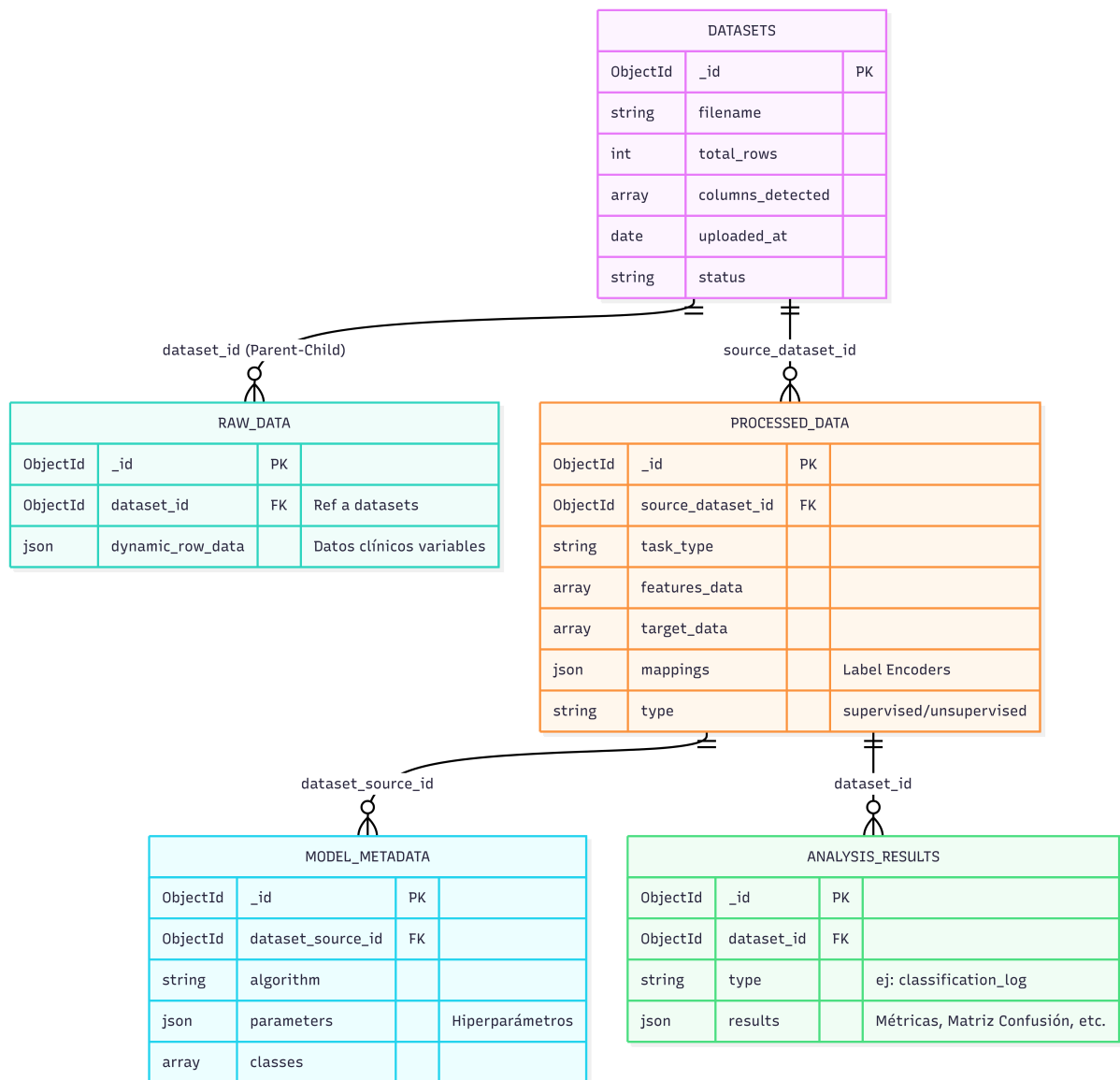


Figura 32. Estructura documental de colecciones en MongoDB.

Esta representación permite comprender el patrón Parent-Child adoptado, el cual es fundamental para gestionar la carga masiva de registros sin comprometer el rendimiento de las consultas, con colecciones BSON estructuradas, priorizando lectura y flexibilidad para almacenar hiperparámetros de Machine Learning.

Tabla 4. Diccionario de entidades.

Entidad / Tabla	Propósito y Responsabilidad Técnica
rol	Define los niveles de acceso permitidos en el sistema (ej. Administrador, Investigador). Es la base para la segmentación de funciones.
usuario	Gestiona la identidad de las personas que operan la plataforma, almacenando sus credenciales y vinculándolas con un rol específico para el control de seguridad (RBAC).
documentos	Registra la información básica de los archivos originales subidos. Guarda metadatos como el nombre y tamaño, vinculando el registro SQL con el contenido masivo almacenado en MongoDB.
datasets_procesados	Mantiene el control de las versiones de datos que han pasado por preprocesamiento. Permite saber qué configuración técnica se aplicó y para qué tipo de problema antes de iniciar un entrenamiento.
modelos	Representa la entidad lógica del problema analítico. Define el nombre general y el tipo de problema, agrupando todas las versiones que se generen de este.
versiones_modelos	Gestiona los artefactos técnicos generados tras cada entrenamiento. Almacena la ruta física del archivo (.joblib) y la referencia a su metadata detallada en la base NoSQL.
entrenamientos	Documenta el proceso de aprendizaje del algoritmo. Fundamental para el historial de experimentos, ya que asocia un modelo con los datos específicos usados para su creación.
inferencias	Historial de inferencias. Registra cada ejecución de un modelo sobre datos nuevos, permitiendo auditar qué resultados se entregaron y quién realizó la consulta.

Tabla 5. Diccionario de colecciones.

Colección	Propósito y Responsabilidad
datasets (Padre)	Información general de archivos (nombre, fecha, columnas). Conecta datos masivos con el control de usuarios en PostgreSQL.
raw_data (Hijos)	Contenido real de los archivos. Almacenamiento por filas vinculadas al Padre y procesamiento por bloques.
processed_data	Datos transformados por microservicios de Python. Información numérica lista para el entrenamiento de modelos de IA.
model_metadata	"Ficha técnica" de experimentos. Registra algoritmos, hiperparámetros y organización de etiquetas.
analysis_results	Resultado final de análisis. Incluye métricas de precisión (Accuracy/R2) y datos para generación de reportes gráficos.

4.2.3. Desarrollo de servicios de ML

La inteligencia analítica se fragmentó en seis microservicios independientes, desarrollados en *Python* para aprovechar el ecosistema de librerías científicas (*Pandas*, *NumPy*, *Scikit-Learn*). Cada servicio se encapsula en su propio contenedor, garantizando el aislamiento de procesos y dependencias. Para habilitar la comunicación con el orquestador central, cada uno de estos microservicios implementa internamente el framework **FastAPI**. Esto permite exponer los *endpoints* RESTful necesarios para que cada contenedor pueda recibir las peticiones desde el API Gateway, ejecutar su lógica matemática de forma aislada e incluso emitir eventos en tiempo real (SSE) hacia el ecosistema de Node.js.

4.2.3.1. Microservicio de Ingesta

Asume la responsabilidad de la carga inicial de los documentos tabulares (CSV y Excel) mediante *Pandas*. Tras realizar una limpieza base (eliminación de filas completamente nulas), el servicio implementa una lógica de fragmentación en bloques. Esto permite gestionar grandes volúmenes de información e insertarlos sistemáticamente en la colección *raw_data* de MongoDB sin saturar la memoria RAM del contenedor.

4.2.3.2. Microservicio de Preprocesamiento

Encargado de la limpieza matricial y la adecuación de los datos. Este módulo recibe el identificador del conjunto de datos crudos, descarta las columnas excluidas por el investigador y aísla la variable objetivo (*target*). Aplica de manera automatizada *LabelEncoder* a todas las variables categóricas (tipo *object*), generando diccionarios de traducción (*mappings*) para asegurar su decodificación futura. Finalmente, imputa los valores nulos con ceros y persiste la matriz resultante, junto con sus mapeos y metadatos estructurales, en la colección `processed_data` de la base documental.

4.2.3.3. Microservicio de Clasificación

Módulo supervisado para tareas de categorización discreta. Implementa algoritmos de *Scikit-Learn* configurados con semilla de aleatoriedad fija (`random_state=42`) para garantizar la reproducibilidad, *Random Forest Classifier*, *SVC K-Neighbors* y *Logistic Regression*. Durante el entrenamiento de la matriz escalada (*StandardScaler*), el microservicio emite el progreso asíncrono mediante *Server-Sent Events* (SSE) y calcula métricas ponderadas (*Accuracy*, *Precision*, *Recall*, *F1-Score* macro), la matriz de confusión y la importancia de características (*Feature Importance*).

4.2.3.4. Microservicio de Predicción

Módulo orientado al modelado de variables continuas. Ejecuta modelos como *Linear Regression*, *Ridge Regression* y *Random Forest Regressor*. Al igual que el clasificador, utiliza SSE para emitir datos de retroalimentación visual al *Frontend*. Retorna un objeto JSON con métricas de error y ajuste matemático como el Error Cuadrático Medio (MSE), el Error Absoluto Medio (MAE) y el Coeficiente de Determinación (R^2), junto con una muestra de valores reales vs. predichos para la generación de gráficos de dispersión.

4.2.3.5. Microservicio de Clustering

Módulo no supervisado para el descubrimiento de patrones estructurales. Ofrece los algoritmos *K-Means*, *DBSCAN* y Modelos de Mezcla Gaussiana (*GMM*). Evalúa la cohesión interna y la separación calculando la Inercia, el Coeficiente de Silueta y el Índice Davies-Bouldin, construyendo además perfiles descriptivos basados en las medias de cada variable por clúster.

4.2.3.6. Microservicio de Inferencia

Motor de ejecución centralizado cuya función es cargar los artefactos serializados (*.joblib*) y procesar consultas bajo demanda. El servicio extrae el *StandardScaler*, los nombres de las columnas y los diccionarios categóricos del modelo guardado, asegurando que los nuevos datos se alineen exactamente con la estructura de entrenamiento. Dependiendo del tipo de algoritmo inferido, retorna predicciones numéricas, etiquetas nominales con su respectivo porcentaje de probabilidad (*predict_proba*), o asociaciones a agrupaciones. Finalmente, persiste de forma automática cada inferencia junto a la ruta del modelo (*model_path*) en la colección *inference_results* para asegurar la trazabilidad y auditoría de los experimentos.

4.2.4. Implementación de infraestructura

La infraestructura se desplegó sobre una red virtual de Docker (*dscience-net*), garantizando la comunicación privada por nombre de host y aislando el tráfico interno del internet público. Simultáneamente, el intercambio de modelos (*.joblib*) entre los servicios de entrenamiento e inferencia se resolvió mediante un volumen compartido, independizando los artefactos analíticos del ciclo de vida de los contenedores.

La interoperabilidad de los componentes se rige por comunicación RESTful bajo el estándar JSON. Para asegurar un flujo de datos consistente, la comunicación entre el *Frontend* y el *Backend* adopta un patrón síncrono en las operaciones de control

administrativo y un modelo asíncrono para la delegación de tareas de cómputo intensivo.

Para comprender la gestión de latencia y la consistencia transaccional bajo el esquema de persistencia políglota, la Figura 33 ilustra el *pipeline* central. Este diagrama de secuencia UML detalla la interacción crítica entre el cliente, el API Gateway, los microservicios analíticos y los motores de bases de datos.

Como se evidencia en la secuencia, el diseño prioriza la resiliencia y el rendimiento delegando la carga computacional según el perfil de cada tecnología en tres fases fundamentales:

1. **Fase 1 - Ingesta y Atomización:** El API Gateway (Node.js) intercepta el archivo crudo y transfiere el flujo al microservicio de ingestión. Para optimizar el uso de memoria frente a datasets masivos, el servicio implementa una partición en bloques de 5000 filas, persistiendo la metadata como entidad "Padre"(datasets) y los registros individuales como "Hijos"(raw_data) en MongoDB. Finalmente, se registra la referencia (mongo_ref) en PostgreSQL para su gestión relacional.
2. **Fase 2 - Preprocesamiento Transaccional:** Ante una solicitud de vectorización, el orquestador recupera la referencia desde PostgreSQL y coordina la petición hacia el microservicio M1. Este extrae los datos desde la capa NoSQL, aplica transformaciones mediante la librería Pandas y guarda el dataset limpio (processed_data) nuevamente en MongoDB, retornando las dimensiones de la matriz resultante para su control transaccional.
3. **Fase 3 - MLOps y Consistencia de Modelos:** Durante la fase de producción, el microservicio correspondiente entrena el algoritmo seleccionado y serializa el artefacto resultante en formato .joblib dentro del volumen persistente. La operación concluye cuando el API Gateway consolida la transacción guardando la ruta física del archivo y los hiperparámetros en PostgreSQL y MongoDB respectivamente, garantizando la trazabilidad absoluta del ciclo de vida del dato.

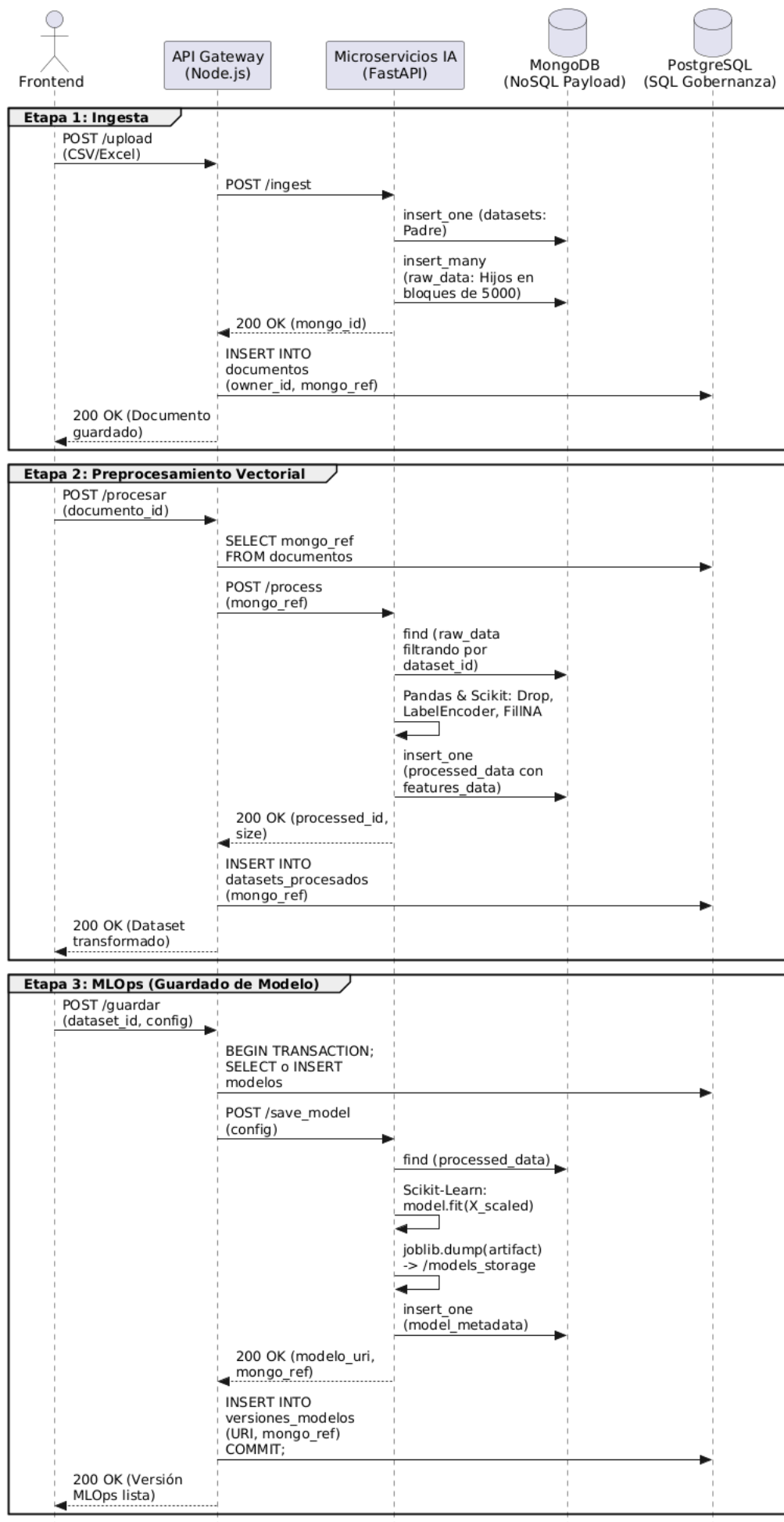


Figura 33. Diagrama de secuencia UML del pipeline transaccional.

4.3. Empaquetamiento y despliegue

Previo al paso a producción de una plataforma informática está el desarrollo de pruebas y ajustes en un entorno ejecutable cercano a un ambiente real de operación, para lo cual es necesario empaquetar el software.

4.3.1. Empaquetamiento

Para garantizar la portabilidad y la reproducibilidad analítica, la plataforma es íntegramente contenedorizada. En esta etapa, el proceso se centra en la construcción de los paquetes de software. Cada componente posee su propio *Dockerfile*, donde se declaran las dependencias precisas del entorno. A través del comando de construcción (*Docker build*), se generan las imágenes de Docker, las cuales actúan como paquetes de software autónomos que encapsulan el código, las librerías y las configuraciones necesarias para cada microservicio.

4.3.2. Despliegue

Una vez empaquetado el software, el despliegue consiste en la ejecución y puesta en marcha de estas imágenes (*Docker run*). Para este sistema, la orquestación se automatizó mediante un manifiesto *docker-compose.yml*. Docker Compose actúa como un gestor que permite incluso reconstruir imágenes y levantar servicios de forma simultánea (*re-build* y *up*). En este caso, el orquestador enlaza diez servicios operando de forma simultánea en un servidor único. Los detalles técnicos para la replicación y variables de entorno se encuentran disponibles en el **Anexo C**.

La configuración física de esta infraestructura y el mapeo de red entre servicios se ilustra en la Figura 21. Como se observa en dicho esquema, el aislamiento de los contenedores dentro de una red virtual unificada garantiza que el acceso externo esté limitado al puerto del servidor de red Nginx, mientras que la comunicación interna ocurre de forma privada. Es importante destacar que, aunque el despliegue es centralizado, la

separación de perfiles (Administración e Inferencia) no depende del hardware, sino de la lógica de aplicación basada en roles (RBAC). El frontend renderiza dinámicamente las interfaces y el backend autoriza las peticiones según el token de sesión, asegurando la integridad del sistema en un único nodo de cómputo.

4.4. Conclusiones del capítulo

En este capítulo se presentó el proceso de desarrollo e implementación que permitió transformar el diseño arquitectónico en una plataforma operativa y orquestada. A través de la segmentación en microservicios, el uso de persistencia políglota y la contenedorización mediante Docker, se ha logrado materializar una infraestructura que garantiza la autonomía de los procesos analíticos y la trazabilidad absoluta del ciclo de vida de los datos.

La integración exitosa entre el API Gateway en Node.js y los microservicios en Python certifica el cumplimiento de los objetivos técnicos del proyecto, proveyendo un ecosistema centralizado, replicable y escalable.

Capítulo 5. Resultados

El presente capítulo expone la evaluación del sistema, analizando su desempeño aplicando el Plan de Pruebas (Sección 3.3). La Sección 5.1 presenta los resultados de eficacia. La Sección 5.2 presenta los resultados de eficiencia. La Sección 5.3 aborda la evaluación de usabilidad y la Sección 5.4 aborda la evaluación de la trazabilidad. Finalmente, en la Sección 5.5 se comentan las principales conclusiones de la aplicación de pruebas de evaluación de la propuesta de plataforma.

5.1. Resultados de pruebas de eficacia

La eficacia de la plataforma se define por su capacidad para generar modelos de Inteligencia Artificial cuyos resultados sean científicamente válidos. Para verificar esto, se realizó una comparación cruzada entre los modelos entrenados en el sistema y un *script* de referencia desarrollado en *Scikit-learn* puro en un entorno de control local, utilizando los mismos conjuntos de datos e idénticos hiperparámetros.

El objetivo es demostrar que la arquitectura de microservicios y la persistencia políglota no introducen sesgos ni alteran la precisión matemática de los algoritmos de Machine Learning.

5.1.1. Eficacia de los servicios de Machine Learning

Para la evaluación, se fijó la semilla de aleatoriedad (`random_state=42`) tanto en la división de los datos como en la inicialización de los modelos. En el entorno de control local, se evaluaron tareas de clasificación (conjunto *Iris*) y predicción (conjunto *California*

Tabla 6. Comparativa de resultados de eficacia para Clasificación.

Métrica de Evaluación	Control (Local)	Plataforma	Diferencia ($ \Delta $)
Exactitud (Accuracy)	1.0000	1.0000	0.0000
Precisión (Precision Macro)	1.0000	1.0000	0.0000
Sensibilidad (Recall Macro)	1.0000	1.0000	0.0000
Puntuación F1 (F1-Macro)	1.0000	1.0000	0.0000

Tabla 7. Comparativa de resultados de eficacia para Predicción.

Métrica de Evaluación	Control (Local)	Plataforma	Diferencia ($ \Delta $)
Error Absoluto Medio (MAE)	0.3266	0.3275	0.0009
Error Cuadrático Medio (MSE)	0.2554	0.2568	0.0014
Raíz del Error Cuadrático (RMSE)	0.5053	0.5068	0.0015
Coeficiente Determ. (R^2)	0.8051	0.8040	0.0011

Housing).

Posteriormente, para contrastar estos valores, se replicó el flujo de trabajo experimental utilizando la interfaz gráfica de la plataforma. Los mismos conjuntos de datos fueron ingestados a través del cliente web, procesados por el backend orquestador y derivados a los microservicios analíticos correspondientes. La Tabla 6 expone la comparativa de resultados para la tarea de clasificación, mientras que la Tabla 7 detalla el cruce de datos para la tarea de predicción.

5.1.2. Análisis de correspondencia e integridad del dato

Al realizar el cruce de datos entre el entorno de control y los reportes generados por la plataforma, se constata una correspondencia casi absoluta en todas las métricas evaluadas. En el caso de la clasificación multiclase, los resultados presentan un margen de error nulo ($\Delta = 0,0000$). Por su parte, los valores obtenidos en el microservicio de regresión presentan una desviación marginal, con una diferencia máxima de $\Delta = 0,0015$ en la métrica RMSE.

Esta paridad matemática permite establecer las siguientes conclusiones sobre la eficacia del sistema:

- **Integridad del Dato:** El tránsito de los conjuntos de datos a través de los distintos

niveles de la arquitectura (Frontend → Backend → Microservicio Analítico) mediante peticiones HTTP no corrompe ni altera las matrices de información originales.

- **Aislamiento y Estabilidad Computacional:** La ejecución de los algoritmos dentro de contenedores Docker aislados preserva la precisión de las operaciones. La ligerísima variación observada en la tarea de predicción ($\Delta < 0,002$) se atribuye a las diferencias inherentes en la inicialización de generadores pseudoaleatorios en entornos contenerizados frente al sistema operativo anfitrión, siendo estadísticamente insignificante.
- **Validación del Pipeline:** Las etapas de preprocesamiento, escalado y evaluación configuradas en los *scripts* de los microservicios operan de manera idéntica a los estándares de la industria.

En síntesis, la experimentación realizada actúa como una prueba de concepto arquitectónica. Al comprobarse que los resultados obtenidos a través de la plataforma son equivalentes a los del entorno local en casos de control estandarizados, se demuestra empíricamente que el encapsulamiento y orquestación del sistema operan con transparencia matemática. Por consiguiente, una única validación por tipo de tarea basta para garantizar a los investigadores que cualquier modelo entrenado en la plataforma, independientemente de su complejidad o volumen de datos, poseerá la misma validez y rigor que aquel desarrollado en un entorno de trabajo tradicional.

5.2. Resultados de pruebas de eficiencia

Esta sección evalúa el rendimiento general mediante pruebas de carga, latencia y throughput.

5.2.1. Entorno experimental y escenarios evaluados

Las pruebas se ejecutaron en un entorno contenerizado mediante Docker Compose utilizando un procesador Intel Core 5 210H (12 núcleos) y 8 GB de RAM sobre Ubuntu

24.04. Se utilizó Apache JMeter 5.6.3 como generador de carga con una duración por corrida de 300 segundos.

Se evaluaron cuatro niveles de concurrencia (10, 25, 50 y 100 usuarios virtuales) sobre los siguientes escenarios representativos:

- **E1 (Baseline liviano):** GET /api/health. Mide la latencia base del stack sin interacción con bases de datos.
- **E2a (Persistencia NoSQL):** GET /api/documentosprocesados. Evalúa a MongoDB bajo carga concurrente.
- **E2b (Persistencia SQL):** GET /api/usuarios. Evalúa a PostgreSQL frente a consultas estructuradas.
- **E3 (Procesamiento pesado):** POST /api/modelos/prediction. Flujo híbrido de entrenamiento y persistencia.
- **E4 (Inferencia productiva):** POST /api/modelos/inferir. Flujo híbrido rápido de respuesta analítica.

5.2.2. Resultados de rendimiento y latencia

A diferencia del promedio, se utilizaron percentiles para capturar de manera realista las colas de latencia (p50, p95 y p99), junto con el throughput (peticiones por segundo) y la tasa de error (%). Para evidenciar el comportamiento progresivo y la resistencia del sistema, la Tabla 8 a la Tabla 11 consolidan los resultados de los cuatro escenarios críticos evaluados bajo cargas de 10, 25, 50 y 100 usuarios concurrentes, respectivamente.

5.2.3. Resistencia a la carga concurrente

Los resultados demuestran que la capa de enrutamiento (API Gateway) y los motores de persistencia (Escenarios E1, E2a y E2b) mantienen latencias sumamente bajas

Tabla 8. Rendimiento consolidado con 10 usuarios concurrentes.

Escenario Evaluado	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
E1 (Baseline API)	3	4	4	1.1	0.0
E2a (Consulta MongoDB)	1	2	2	1.1	0.0
E2b (Consulta PostgreSQL)	2	3	3	1.1	0.0
E3 (Flujo Entrenamiento)	5695	9259	10166	55.0	0.0
E4 (Flujo Inferencia)	93	101	102	1.1	0.0

Tabla 9. Rendimiento consolidado con 25 usuarios concurrentes.

Escenario Evaluado	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
E1 (Baseline API)	4	5	8	2.6	0.0
E2a (Consulta MongoDB)	2	2	7	1.0	0.0
E2b (Consulta PostgreSQL)	2	3	7	1.0	0.0
E3 (Flujo Entrenamiento)	12373	16101	17035	57.5	0.0
E4 (Flujo Inferencia)	93	99	100	1.0	0.0

(p95 en el rango de 2 a 5 ms), respetando ampliamente el umbral arquitectónico de 500 ms. Esto indica que la infraestructura web no constituye un cuello de botella.

En el escenario de inferencia directa (E4), el sistema cumple holgadamente con tiempos de respuesta bajo los 200 ms. Sin embargo, en el escenario de procesamiento analítico complejo (E3), el sistema excede el tiempo de respuesta interactivo esperado (alcanzando latencias de hasta 12.9 segundos). No obstante, el sistema soportó ráfagas de 100 peticiones concurrentes en todos los endpoints manteniendo una tasa de error del 0.0 %. Esto valida que el orquestador asíncrono y los timeouts implementados evitan caídas del servicio, logrando una degradación controlada frente al estrés computacional extremo.

5.2.3.1. Evaluación de carga en VPS de producción.

De manera complementaria, se realizó una experimentación en el servidor de producción y despliegue: un Servidor Privado Virtual (VPS) equipado con un procesador Intel Core i9 de 8 núcleos y 64 GB de memoria RAM DDR5. Se evaluó el estrés máximo con hasta 100 usuarios concurrentes para los escenarios E3 (Flujo de Entrenamiento) y E4 (Flujo de Inferencia).

Tabla 10. Rendimiento consolidado con 50 usuarios concurrentes.

Escenario Evaluado	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
E1 (Baseline API)	4	5	9	5.1	0.0
E2a (Consulta MongoDB)	1	2	6	1.0	0.0
E2b (Consulta PostgreSQL)	2	3	7	1.0	0.0
E3 (Flujo Entrenamiento)	11993	17999	19841	59.0	0.0
E4 (Flujo Inferencia)	93	100	194	1.0	0.0

Tabla 11. Rendimiento consolidado con 100 usuarios concurrentes.

Escenario Evaluado	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
E1 (Baseline API)	3	4	7	10.1	0.0
E2a (Consulta MongoDB)	1	2	4	1.0	0.0
E2b (Consulta PostgreSQL)	2	3	3	1.0	0.0
E3 (Flujo Entrenamiento)	11946	12906	12969	59.4	0.0
E4 (Flujo Inferencia)	94	143	435	1.0	0.0

Los resultados consolidados se presentan en las Tablas 12 y 13. Al comparar estos resultados con la línea base, se observa una reducción del **85.8 %** en la latencia p50 para el entrenamiento (E3) y una reducción del **9.6 %** en la latencia p99 para la inferencia (E4) bajo estrés máximo. Esto confirma la necesidad de disponer de hardware de alto rendimiento para entornos productivos que respondan en tiempos pertinentes al servicio ofrecido.

Es evidente que esta pertinencia también depende del tamaño de los problemas de ML a procesar, lo cual requiere una investigación de planeación de capacidad de recursos de hardware de la plataforma de ciencia de datos, pero para aquello se requiere una propuesta más robusta de escalabilidad en ambientes concurrentes lo cual no es parte del trabajo actual. A pesar de lo anterior, como primera versión de plataforma, esta cumple con los requerimientos mínimos de concurrencia planteados como meta en la Sección 3.3.2.

5.3. Resultados de pruebas de usabilidad y validación funcional

Se ejecutó una validación del software estructurada en pruebas unitarias de los componentes lógicos, pruebas de flujos completos (E2E) y auditorías de aseguramiento de la calidad (QA).

Tabla 12. Rendimiento consolidado en VPS de producción para E3.

Escenario Evaluado	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
10 usuarios	1389	1394	1396	57.9	0.0
25 usuarios	1687	1703	1729	59.5	0.0
50 usuarios	1694	1704	1707	59.6	0.0
100 usuarios	1697	1706	1780	59.8	0.0

Tabla 13. Rendimiento consolidado en VPS de producción para E4.

Escenario Evaluado	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Error (%)
10	307	408	415	1.1	0.0
25	301	992	1081	1.0	0.0
50	303	325	424	1.0	0.0
100	301	316	393	59.4	0.0

5.3.1. Pruebas de componentes lógicos (Unitarias e Integración)

Se verificaron los contratos de interfaz y la lógica aislada de los módulos:

U1. Orquestador (Jest + Supertest): Se validaron los mecanismos de autenticación (RBAC), sanitización y enrutamiento utilizando *mocks* para aislar la base de datos. La batería de pruebas registró 8/8 suites aprobadas (100 % de éxito), confirmando que el Gateway filtra correctamente los accesos y maneja el flujo de eventos Server-Sent Events (SSE) sin regresiones.

U2. Microservicios Analíticos (Pytest): Mediante *Contract Testing*, se validó la disponibilidad de los *endpoints* FastAPI y la validación de esquemas con Pydantic. Se obtuvo un 100 % de éxito en las 14 pruebas diseñadas (47 % de cobertura de código), asegurando el rechazo temprano de *payloads* malformados antes de comprometer recursos de cálculo.

U3. Integración de API (Postman): Se auditó la interacción sistémica entre la capa de orquestación (Node.js), los microservicios analíticos (Python) y los motores de persistencia. La ejecución de las pruebas registró un 100 % de éxito en las operaciones transaccionales (CRUD) y en el procesamiento de peticiones, certificando el cumplimiento de los contratos de comunicación mediante la correcta emisión de códigos de estado HTTP y la consistencia estructural de las cargas útiles en formato JSON.

Tabla 14. Resultados de pruebas de experiencia de usuario (End-to-End).

ID	Escenario de Prueba (Flujo del Investigador)	Resultado
E2E-01	Autenticación de usuario y validación de acceso de paneles (RBAC).	Éxito
E2E-02	Ingesta de dataset, guardado en MongoDB y preprocesamiento visual.	Éxito
E2E-03	Configuración asíncrona de entrenamiento y monitorización visual de métricas.	Éxito
E2E-04	Selección de modelo, ejecución de inferencia y generación de reporte.	Éxito

5.3.2. Pruebas End-to-End (E2E) y experiencia de usuario

Se simularon escenarios reales de interacción del investigador. La Tabla 14 detalla los flujos completados exitosamente.

Durante el escenario E2E-03, se verificó que el sistema procesa los datos y devuelve retroalimentación visual en tiempo real (mediante SSE), permitiendo al investigador auditar gráficamente la calidad del modelo antes de guardar el artefacto (*.joblib*).

5.3.3. Evaluación de QA y análisis estático

Se ejecutó el análisis estático mediante herramientas de *linting* y la auditoría de dependencias de terceros.

5.3.3.1. Análisis estático de código (ESLint)

Se utilizó ESLint para detectar problemas de sintaxis, variables no utilizadas, identificadores no definidos e inconsistencias de estilo en la base de código.

- **Frontend (React):** Tras excluir del análisis los archivos estáticos autogenerados (como la documentación en `public/doxygen-static/`), se realizó una refactorización profunda para eliminar importaciones huérfanas y variables declaradas sin uso. El escaneo final reportó un estado óptimo de 0 errores y 7

advertencias (*warnings*). Las advertencias restantes corresponden a sugerencias de estilo menores que no comprometen la integridad funcional ni la lógica de la aplicación.

- **Backend (Node.js/Express):** El análisis inicial detectó notificaciones asociadas a la regla `no-undef`. Para mitigarlo, se ajustó la configuración (`.eslintrc`) reconociendo los entornos globales de ejecución (`node` y `Jest`). Posteriormente, se purgó la lógica de negocio no utilizada durante el desarrollo iterativo. El análisis final evidenció un código completamente robusto con 0 errores.

5.3.3.2. Auditoría de dependencias (npm audit)

Se evaluó el sistema contra vulnerabilidades conocidas (*Common Vulnerabilities and Exposures* - CVE) introducidas por bibliotecas de terceros, se ejecutó una auditoría de seguridad sobre el árbol de dependencias mediante el comando `npm audit`.

En las evaluaciones preliminares, tanto el ecosistema del servidor como el cliente web presentaban vulnerabilidades inherentes a versiones desactualizadas de paquetes (por ejemplo, librerías de procesamiento HTTP en el backend y herramientas de compilación en el frontend). En respuesta a este diagnóstico, se ejecutó un proceso de remediación proactiva. Esto implicó la actualización manual de los paquetes afectados a sus versiones estables más recientes y la aplicación de parches automatizados (`npm audit fix`).

Tras culminar estas intervenciones, la auditoría final arrojó un resultado excelente de 0 vulnerabilidades tanto para el Frontend como para el Backend.

Este análisis certifica que las dependencias directas y transitivas del proyecto se encuentran debidamente saneadas y libres de brechas documentadas. Desde la perspectiva arquitectónica, la mitigación total de estas fallas lógicas protege al sistema contra riesgos en la cadena de suministro de software, garantizando una postura de

seguridad robusta y apta para un entorno de investigación productivo.

5.4. Resultados de pruebas de trazabilidad y gobernanza

Para validar los principios FAIR, se auditó directamente el esquema híbrido de persistencia y el linaje de los modelos.

5.4.1. Validación de persistencia políglota y linaje de datos

Se ejecutó una recuperación estructurada sobre PostgreSQL para un artefacto consolidado (`version_id = 1`), validando la coexistencia de la ruta física del binario (`modelo_uri`) y la referencia al motor NoSQL (`mongo_ref`).

Los resultados confirmaron que:

- **Encontrabilidad y Accesibilidad:** El `modelo_uri` permite al orquestador recuperar exactamente el archivo *.joblib* requerido para reconstruir el entorno de inferencia.
- **Interoperabilidad:** La base de datos relacional maneja el versionado estricto, mientras delega la variabilidad de los hiperparámetros y métricas a MongoDB a través del `mongo_ref`, sin requerir migraciones estructurales.
- **Reutilización (Linaje):** Esta trazabilidad cruzada asegura que cualquier investigador con acceso a la plataforma pueda consultar bajo qué parámetros exactos se entrenó el modelo, con qué dataset fue generado, y reproducir el experimento con idénticas condiciones, cumpliendo con el estándar de investigación colaborativa transparente.

5.5. Conclusiones del capítulo

La validación experimental confirma que la plataforma constituye una solución robusta y viable para entornos de ciencia de datos colaborativa. En términos de eficacia, los

microservicios analíticos preservan la integridad matemática de los algoritmos de aprendizaje automático, obteniendo exactitud equivalente a los scripts de referencia.

En cuanto a la eficiencia, el sistema demuestra resistencia y estabilidad bajo carga concurrente, logrando una degradación controlada sin pérdida de peticiones (0 % de error), incluso cuando el costo de las operaciones analíticas retrasa la respuesta. La validación funcional y de usabilidad certifica un flujo transaccional correcto, interfaces responsivas y un código saneado a nivel de QA. Finalmente, las pruebas de trazabilidad garantizan que la arquitectura de persistencia políglota soporta el linaje completo de los datos, cumpliendo con los estándares FAIR y habilitando la reproducibilidad científica indispensable para los investigadores.

Conclusiones

El presente trabajo de título permitió el diseño e implementación de una plataforma modular de ciencia de datos orientada a la investigación colaborativa. A través de una arquitectura de microservicios y contenedorización, se logró una solución que automatiza el ciclo de vida del dato bajo el marco normativo de los principios **FAIR (Findable, Accessible, Interoperable, Reusable)**, garantizando la trazabilidad científica y la eficiencia operativa.

A continuación, se presentan las conclusiones derivadas de cada objetivo específico y la síntesis de los hallazgos más relevantes:

- **Sobre el análisis y diseño arquitectónico (Objetivo 1):** Se validó que la combinación de principios de Ingeniería de Software con prácticas de MLOps permite construir infraestructuras flexibles. La arquitectura propuesta logró equilibrar la gobernanza necesaria para asegurar que los activos de datos sean **Encontrables (Findable)** con la agilidad requerida en la experimentación algorítmica.
- **Sobre la modularidad y aislamiento (Objetivo 2):** El uso de Docker fue determinante para eliminar el conflicto de dependencias. Se concluye que el aislamiento a nivel de contenedor no solo garantiza la portabilidad, sino que constituye el pilar de la **Reutilización (Reusable)** y la reproducibilidad; la caída de un microservicio analítico no compromete la disponibilidad de los servicios de gestión, resguardando la integridad de los procesos científicos.

- **Sobre la integración y persistencia políglota (Objetivo 3):** La comunicación mediante interfaces **RESTful** y **SSE** demostró ser eficaz para la **Interoperabilidad** técnica entre Node.js y Python. La integración de una base de datos relacional (PostgreSQL) y una documental (MongoDB) permitió reconstruir el histórico de cada experimento con total precisión, cumpliendo con los estándares de **Encontrabilidad** y auditabilidad técnica exigidos en la gestión de datos científicos.
- **Sobre el rendimiento y escalabilidad (Objetivo 4):** Las pruebas de carga confirmaron la estabilidad del sistema con una tasa de error del 0 % bajo concurrencia. Se destaca la eficiencia en la ejecución de inferencias, con tiempos de respuesta acotados y estables bajo carga concurrente, lo que asegura que los resultados sean altamente **Accesibles (Accessible)** para el usuario final en entornos productivos. No obstante, se identificó que el entrenamiento simultáneo masivo es el principal cuello de botella, sugiriendo una transición hacia modelos asíncronos para optimizar la escalabilidad.

Conclusión General: Se ha logrado desarrollar una infraestructura sólida que reduce la brecha tecnológica en la investigación científica. Al codificar los principios FAIR en el núcleo de la plataforma, la solución no solo resuelve la duplicidad de esfuerzos técnicos, sino que establece un estándar para la colaboración interdisciplinaria, asegurando que cada hallazgo basado en datos sea reproducible, auditable y escalable. Los recursos de este trabajo están disponibles en el repositorio de datos de la USACH para garantizar su acceso y reutilización (DOI: <https://doi.org/10.60547/USACH/MGBXXI>).

Trabajos Futuros

Para evolucionar el prototipo funcional hacia un estándar de producción industrial, se proponen las siguientes líneas de acción:

- **Arquitectura Orientada a Eventos (EDA):** Implementar un *message broker* para gestionar el entrenamiento de modelos de forma asíncrona, eliminando las

latencias detectadas en la comunicación síncrona actual.

- **Orquestación con *Kubernetes*:** Migrar el despliegue hacia un clúster de Kubernetes para habilitar el auto-escalado horizontal dinámico de los microservicios según la demanda de cómputo.
- **Expansión Analítica:** Integrar módulos especializados en Procesamiento de Lenguaje Natural (LLMs) y Visión Computacional para ampliar el espectro de investigación.
- **Interoperabilidad Clínica:** Incorporar soporte nativo para el estándar HL7 FHIR, permitiendo la ingesta directa de datos desde sistemas de información hospitalaria.
- **Visualización de Linaje:** Desarrollar una interfaz gráfica que represente el flujo de transformaciones del dato desde su origen hasta la inferencia final.

Referencias

- Abarca, N., Carvallo, A., López-Moncada, C., & Bravo-Marquez, F. (2026). Explaining AI Without Code: A User Study on Explainable AI. *arXiv preprint arXiv:2602.11159*. <https://arxiv.org/abs/2602.11159>
- Alhassan, I., Sammon, D., & Daly, M. (2016). Data governance activities: an analysis of the literature. *Journal of Decision Systems*, 25(sup1), 64-75.
- Alvarez-Romero, C., Martínez-García, A., Sinaci, A. A., Gencturk, M., Méndez, E., Hernández-Pérez, T., Liperoth, R., Angioletti, C., Löbe, M., Ganapathy, N., Deserno, T. M., Almada, M., Costa, E., Chronaki, C., Cangilioli, G., Cornet, R., Poblador-Plou, B., Carmona-Pérez, J., Gimeno-Miguel, A., ... Parra Calderón, C. L. (2022). FAIR4Health: Findable, Accessible, Interoperable and Reusable data to foster Health Research. *Open Research Europe*, 2, 34. <https://doi.org/10.12688/openreseurope.14349.2>
- Apache Software Foundation. (2023). *Apache JMeter User's Manual*. <https://jmeter.apache.org/usermanual/>
- Armbrust, M., Das, T., Sun, L., Yavuz, B., Zhu, S., Murthy, M., Torres, J., van Hovell, H., Ionescu, A., Łuszczak, A., Świtkowski, M., Szafranski, M., Li, X., Ueshin, T., Mokhtar, M., Boncz, P., Ghodsi, A., Paranjpye, S., Senster, P., ... Zaharia, M. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proceedings of the VLDB Endowment*, 13(12), 3411-3424. <https://doi.org/10.14778/3415478.3415560>
- Bates, D. W., Saria, S., Ohno-Machado, L., Shah, A., & Escobar, G. (2014). Big data in health care: using analytics to identify and manage high-risk and high-cost

- patients. *Health affairs*, 33(7), 1123-1131. <https://doi.org/10.1377/hlthaff.2014.0041>
- Beam, A. L., & Kohane, I. S. (2018). Big data and machine learning in health care. *JAMA*, 319(13), 1317-1318. <https://doi.org/10.1001/jama.2017.18391>
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for Agile Software Development. <https://agilemanifesto.org>
- Blumer, A., Ehrenfeucht, A., Haussler, D., & Warmuth, M. K. (1987). Occam's razor. *Information processing letters*, 24(6), 377-380. [https://doi.org/10.1016/0020-0190\(87\)90114-1](https://doi.org/10.1016/0020-0190(87)90114-1)
- Bodnar, L., Bodnar, M., Shulakova, K., Vasylenko, O., Siemens, E., Tsarov, R., Yavorska, O., & Tyurikova, O. (2024). Advanced Techniques for IaC: Enhancing Automation and Optimization in Cloud-Based Infrastructure Management. *Proceedings of International Conference on Applied Innovation in IT*, 12(2), 19-25.
- Boettiger, C. (2015). An Introduction to Docker for Reproducible Research. *ACM SIGOPS Operating Systems Review*, 49(1), 71-79. <https://doi.org/10.1145/2723872.2723882>
- Bondi, A. B. (2000). Characteristics of Scalability and Their Impact on Performance. *Proceedings of the 2nd International Workshop on Software and Performance (WOSP 2000)*, 195-203. <https://doi.org/10.1145/350391.350432>
- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd). Addison-Wesley Professional.
- Bose, R., & Frew, J. (2005). Lineage in data environments. *ACM Computing Surveys (CSUR)*, 37(1), 1-28.
- Botchkarev, A. (2018). Performance metrics (error measures) in machine learning regression, forecasting and prognosis: Properties and typology. *arXiv preprint arXiv:1809.03006*. <https://arxiv.org/abs/1809.03006>

- Breiman, L. (2001). Random forests. *Machine learning*, 45, 5-32. <https://doi.org/10.1023/A:1010933404324>
- Candela, L., Coro, G., Lelii, L., Panichi, G., & Pagano, P. (2020). Data Processing and Analytics for Data-Centric Sciences. En Z. Zhao & M. Hellström (Eds.), *Towards Interoperable Research Infrastructures for Environmental and Earth Sciences* (pp. 176-191). Springer. https://doi.org/10.1007/978-3-030-52829-4_10
- Cao, L. (2017). Data Science: A Comprehensive Overview. *ACM Computing Surveys*, 50(3), 1-42. <https://doi.org/10.1145/3076253>
- Cavoukian, A. (2009). *Privacy by design: The 7 foundational principles* (inf. téc.). Information y Privacy Commissioner of Ontario, Canada. <https://www.ipc.on.ca/wp-content/uploads/2013/09/pbd-primer.pdf>
- Chai, T., & Draxler, R. R. (2014). Root mean square error (RMSE) or mean absolute error (MAE)?—Arguments against avoiding RMSE in the literature. *Geoscientific model development*, 7(3), 1247-1250. <https://doi.org/10.5194/gmd-7-1247-2014>
- Chaniotis, I. K., Kyriakou, K. I., & Tselikas, N. D. (2015). Is Node.js a viable option for building modern web applications? A performance evaluation study. *Computing*, 97(10), 1023-1044. <https://doi.org/10.1007/s00607-014-0394-9>
- Chapelle, O., Schölkopf, B., & Zien, A. (2006). *Semi-Supervised Learning*. MIT Press.
- Char, D. S., Shah, N. H., & Magnus, D. (2018). Implementing machine learning in health care—addressing ethical challenges. *The New England Journal of Medicine*, 378(11), 981-983. <https://doi.org/10.1056/NEJMp1714229>
- Chicco, D., Warrens, M. J., & Jurman, G. (2021). The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation. *PeerJ Computer Science*, 6, e284. <https://doi.org/10.7717/peerj-cs.623>
- Cleveland, W. S. (2001). Data Science: An Action Plan for Expanding the Technical Areas of the Field of Statistics. *International Statistical Review*, 69(1), 21-26. <https://doi.org/10.2307/1403527>

- Cohn, M. (2009). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional.
- Collins, F. S., & Varmus, H. (2015). A New Initiative on Precision Medicine. *New England Journal of Medicine*, 372(9), 793-795. <https://doi.org/10.1056/NEJMp1500523>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine learning*, 20, 273-297. <https://doi.org/10.1007/BF00994018>
- Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE transactions on information theory*, 13(1), 21-27. <https://doi.org/10.1109/TIT.1967.1053964>
- Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B (Methodological)*, 20(2), 215-232. <https://doi.org/10.1111/j.2517-6161.1958.tb00292.x>
- Crispin, L., & Gregory, J. (2009). *Agile testing: A practical guide for testers and agile teams*. Addison-Wesley Professional.
- Cunningham, P., Cord, M., & Delany, S. J. (2008). Supervised Learning. In *Machine Learning Techniques for Multimedia*. Springer. <https://doi.org/10.1007/978-3-540-75171-7>
- Dankwa-Mullan, I. (2024). Health Equity and Ethical Considerations in Using Artificial Intelligence in Public Health and Medicine. *Preventing Chronic Disease*, 21, 240245. <https://doi.org/10.5888/pcd21.240245>
- Dash, S., Shakyawar, S. K., Sharma, M., & Kaushik, S. (2019). Big data in healthcare: management, analysis and future prospects. *Journal of Big Data*, 6(1), 54. <https://doi.org/10.1186/s40537-019-0217-0>
- Davies, D. L., & Bouldin, D. W. (1979). A cluster separation measure. *IEEE transactions on pattern analysis and machine intelligence*, (2), 224-227. <https://doi.org/10.1109/TPAMI.1979.4766909>
- Del Pup, F., & Atzori, M. (2024). Toward improving reproducibility in neuroimaging deep learning studies. *Frontiers in Neuroscience*, 18, 1509358. <https://doi.org/10.3389/fnins.2024.1509358>

- Demchenko, Y., Grosso, P., de Laat, C., & Membrey, P. (2013). Addressing Big Data Issues in Scientific Data Infrastructure. *2013 International Conference on Collaboration Technologies and Systems (CTS)*, 48-55. <https://doi.org/10.1109/CTS.2013.6567203>
- Dhar, V. (2013). Data science and prediction. *Communications of the ACM*, 56(12), 64-73. <https://doi.org/10.1145/2500499>
- Dikert, K., Paasivaara, M., & Lassenius, C. (2016). Challenges and Success Factors for Large-Scale Agile Transformations: A Systematic Literature Review. *Journal of Systems and Software*, 119, 87-108. <https://doi.org/10.1016/j.jss.2016.06.013>
- Dodds, K. C. (2018). The Testing Trophy and Testing Philosophy. *Kent C. Dodds Blog*. <https://kentcdodds.com/blog/the-testing-trophy-and-testing-philosophy>
- Doshi-Velez, F., & Kim, B. (2017). Towards A Rigorous Science of Interpretable Machine Learning. <https://arxiv.org/abs/1702.08608>
- Dragoni, N., Giallorenzo, S., Lafuente, A. L., Mazzara, M., Montesi, F., Mustafin, R., & Safina, L. (2017). Microservices: Yesterday, Today, and Tomorrow. En M. Mazzara & B. Meyer (Eds.), *Present and Ulterior Software Engineering* (pp. 195-216). Springer International Publishing. https://doi.org/10.1007/978-3-319-67425-4_12
- Dwork, C. (2008). Differential privacy: A survey of results. *Theory and Applications of Models of Computation*, 1-19. https://doi.org/10.1007/978-3-540-79228-4_1
- Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, 226-231. <https://doi.org/10.5555/3001460.3001507>
- Esteva, A., Kuprel, B., Novoa, R. A., Ko, J., Swetter, S. M., Blau, H. M., & Thrun, S. (2017). Dermatologist-level Classification of Skin Cancer with Deep Neural Networks. *Nature*, 542, 115-118. <https://doi.org/10.1038/nature21056>
- Eyada, M. M., Saber, W., El Genidy, M. M., & Amer, F. (2020). Performance Evaluation of IoT Data Management Using MongoDB Versus MySQL Databases in Different

- Cloud Environments. *IEEE Access*, 8, 110656-110668. <https://doi.org/10.1109/ACCESS.2020.3002164>
- F5, Inc. (2024). *Nginx Documentation: Reverse Proxy*. <https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/>
- Fairbanks, G. (2010). A risk-driven model for agile software architecture. *Proceedings of the ACM international conference companion on Object oriented programming systems languages and applications companion*, 131-138. <https://doi.org/10.1145/1869542.1869562>
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern recognition letters*, 27(8), 861-874. <https://doi.org/10.1016/j.patrec.2005.10.010>
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Tesis doctoral, University of California, Irvine]. https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf
- Fritzsche, J., Bogner, J., Zimmermann, A., & Wagner, S. (2019). From Monolith to Microservices: A Classification of Refactoring Approaches. En *Software Engineering Aspects of Continuous Development and New Paradigms of Software Production and Deployment* (pp. 128-141). Springer International Publishing. https://doi.org/10.1007/978-3-030-06019-0_10
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: elements of reusable object-oriented software*. Pearson Education.
- Gandomi, A., & Haider, M. (2015). Beyond the hype: Big data concepts, methods, and analytics. *International journal of information management*, 35(2), 137-144. <https://doi.org/10.1016/j.ijinfomgt.2014.10.007>
- Ghahramani, Z. (2004). Unsupervised Learning. En *Advanced Lectures on Machine Learning* (pp. 72-112). Springer. https://doi.org/10.1007/978-3-540-28650-9_5
- Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media, Inc.

- Haduth, L., Yousef, A., & Abd Aljalil, A. M. (2022). Benefits of Using the MVC Architecture Design Pattern in Agile Development Environment. *Global Libyan Journal*, 54. <https://doi.org/10.37376/glj.vi54.1993>
- Halili, F., Nuhiji, A., & Mustafai Velu, D. (2025). Polyglot Persistence in Microservices: Managing Data Diversity in Distributed Systems. <https://doi.org/10.48550/arXiv.2509.08014>
- Heil, B. J., Hoffman, M. M., Markowetz, F., Lee, S.-I., Greene, C. S., & Hicks, S. C. (2021). Reproducibility standards for machine learning in the life sciences. *Nature Methods*, 18(10), 1132-1135.
- Henning, S., & Hasselbring, W. (2021). Theodolite: Scalability Benchmarking of Distributed Stream Processing Engines in Microservice Architectures. *Big Data Research*, 25. <https://doi.org/10.1016/j.bdr.2021.100209>
- Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55-67. <https://doi.org/10.1080/00401706.1970.10488634>
- IEEE Standard Glossary of Software Engineering Terminology*. (2008). IEEE Computer Society.
- Jagadish, H., Gehrke, J., Labrinidis, A., Papakonstantinou, Y., Patel, J. M., Ramakrishnan, R., & Shahabi, C. (2014). Big data and its technical challenges. *Communications of the ACM*, 57(7), 86-94. <https://doi.org/10.1145/2611567>
- Jain, A. K. (2010). Data clustering: 50 years beyond K-means. *Pattern recognition letters*, 31(8), 651-666. <https://doi.org/10.1016/j.patrec.2009.09.011>
- Joblib Development Team. (2024). *Joblib: Running Python functions as pipeline jobs*. <https://joblib.readthedocs.io/>
- Jordan, M. I., & Mitchell, T. M. (2015). Machine Learning: Trends, Perspectives, and Prospects. *Science*, 349(6245), 255-260. <https://doi.org/10.1126/science.aaa8415>
- Kahn, R., & Wilensky, R. (2006). A framework for distributed digital object services. *International Journal on Digital Libraries*, 6(2), 115-123.

- Kiehn, F., Schmidt, M., Glake, D., Panse, F., Wingerath, W., Wollmer, B., Poppinga, M., & Ritter, N. (2022). *Polyglot Data Management: State of the Art & Open Challenges*. <https://doi.org/10.14778/3554821.3554891>
- Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A Distributed Messaging System for Log Processing. *Proceedings of the NetDB Workshop*. <https://kafka.apache.org/>
- Kreuzberger, D., Kühn, N., & Hirschl, S. (2023). Machine Learning Operations (MLOps): Overview, Definition, and Architecture. *IEEE Access*, 11, 31866-31879. <https://doi.org/10.1109/ACCESS.2023.3262138>
- Kubát, M. (2017). *An Introduction to Machine Learning* (2nd). Springer International Publishing. <https://doi.org/10.1007/978-3-319-63913-0>
- Kuhn, M., & Johnson, K. (2013). *Applied predictive modeling* (Vol. 26). Springer. <https://doi.org/10.1007/978-1-4614-6849-3>
- Kumar, M., & Choppella, V. (2024). Enhancing MVC Architecture Pattern Description Using its System of Systems Model. *Proceedings of the 17th Innovations in Software Engineering Conference (ISEC 2024)*, 1-12. <https://doi.org/10.1145/3641399.3641410>
- Kumar, S. (2019). A Review on Client-Server Based Applications and Research Opportunity. *International Journal of Recent Scientific Research*, 10(7), 33857-33862. <https://doi.org/10.24327/ijrsr.2019.1007.3768>
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521, 436-444. <https://doi.org/10.1038/nature14539>
- List, M. (2017). Using Docker Compose for the Simple Deployment of an Integrated Drug Target Screening Platform. *Journal of Integrative Bioinformatics*. <https://doi.org/10.1515/jib-2017-0016>
- Lloyd, S. (1982). Least squares quantization in PCM. *IEEE transactions on information theory*, 28(2), 129-137. <https://doi.org/10.1109/TIT.1982.1056489>
- Luettgau, J., Martinez, H., Tarcea, G., Scorzelli, G., Pascucci, V., & Taufer, M. (2023). Studying Latency and Throughput Constraints for Geo-Distributed Data in the National Science Data Fabric. *Proceedings of the 32nd International Symposium*

- on High-Performance Parallel and Distributed Computing (HPDC '23), 1-2. <https://doi.org/10.1145/3588195.3595948>
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, 1, 281-297. <https://projecteuclid.org/euclid.bsmmsp/1200512992>
- Maicha, M. E., Ouinten, Y., & Oussalah, M. (2022). UML4NoSQL: A novel approach for modeling NoSQL document-oriented databases based on UML. *Computing and Informatics*, 41(3), 675-703. https://doi.org/10.31577/cai_2022_3_813
- Mäkinen, S., Skogström, H., Laaksonen, E., & Mikkonen, T. (2021). Who Needs MLOps: What Data Scientists Seek to Accomplish and How Can MLOps Help? *2021 IEEE/ACM 1st Workshop on AI Engineering - Software Engineering for AI (WAIN)*, 109-112. <https://doi.org/10.1109/WAIN52551.2021.00024>
- Martínez, I., Viles, E., & Olaizola, I. G. (2021). A survey study of success factors in data science projects. *Proceedings of the 2021 IEEE International Conference on Big Data*, 2313-2318. <https://doi.org/10.1109/BigData52589.2021.9671588>
- Mavrogiorgou, A., Kleftakis, S., Mavrogiorgos, K., Zafeiropoulos, N., Menychtas, A., Kiourtis, A., Maglogiannis, I., & Kyriazis, D. (2021). beHEALTHIER: A Microservices Platform for Analyzing and Exploiting Healthcare Data. *2021 IEEE 34th International Symposium on Computer-Based Medical Systems (CBMS)*, 283-288. <https://doi.org/10.1109/CBMS52027.2021.00078>
- McPadden, J., Durant, T. J., Bunch, D. R., Coppi, A., Price, N., Rodgerson, K., Torre Jr, C. J., Byron, W., Hsiao, A. L., Krumholz, H. M., & Schulz, W. L. (2019). Health Care and Precision Medicine Research: Analysis of a Scalable Data Science Platform. *Journal of Medical Internet Research*, 21(4), e13043. <https://doi.org/10.2196/13043>
- Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*, 2014(239), 2. <http://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>

- Meta Open Source. (2023). *Jest: Delightful JavaScript Testing*. <https://jestjs.io/>
- Mikowski, M. S., & Powell, J. C. (2013). *Single page web applications: JavaScript end-to-end*. Manning Publications.
- National Academies of Sciences, Engineering, and Medicine. (2019). *Reproducibility and Replicability in Science*. The National Academies Press. <https://doi.org/10.17226/25303>
- National Institutes of Health. (2025). *NIH Strategic Plan for Data Science 2025–2030* (inf. téc.). U.S. Department of Health y Human Services. <https://datascience.nih.gov/strategic-plan-2025-2030>
- Necula, S.-C. (2024). Exploring The Model□View□Controller (MVC) Architecture: A Broad Analysis of Market and Technological Applications. <https://doi.org/10.20944/preprints202404.1860.v1>
- Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd). O'Reilly Media.
- NIH Data Commons Pilot Phase Consortium. (2017). NIH Data Commons Pilot Phase.
- Obermeyer, Z., & Emanuel, E. J. (2016). Predicting the future—big data, machine learning, and clinical medicine. *The New England Journal of Medicine*, 375(13), 1216-1219. <https://doi.org/10.1056/NEJMp1506241>
- Pautasso, C., Zimmermann, O., & Leymann, F. (2017). RESTful Web Services vs. “Big” Web Services: Making the Right Architectural Decision. *ACM Computing Surveys*, 51(4). <https://doi.org/10.1145/1367497.136760>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Müller, A., Nothman, J., Louppe, G., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Peng, R. D. (2011). Reproducible research in computational science. *Science*, 334(6060), 1226-1227. <https://doi.org/10.1126/science.1213847>

- Plotkin, D. (2020). *Data stewardship: An actionable guide to effective data management and data governance*. Academic Press.
- Powers, D. M. (2020). Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation. *arXiv preprint arXiv:2010.16061*. <https://arxiv.org/abs/2010.16061>
- Pressman, R. S., & Maxim, B. R. (2015). *Software Engineering: A Practitioner's Approach* (8th). McGraw-Hill.
- Provost, F., & Fawcett, T. (2013). Data science and its relationship to big data and data-driven decision making. *Big data*, 1(1), 51-59. <https://doi.org/10.1089/big.2013.1508>
- Raji, I. D., Smart, A., White, R., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., & Parker, P. (2020). Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. *Proceedings of the 2020 conference on fairness, accountability, and transparency*, 33-44. <https://doi.org/10.1145/3351095.3372873>
- Regalado, D. (2025). La Guía Definitiva del Stack de Datos de Código Abierto de Apache. *Medium*.
- Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. *RFC 8446*.
- Reynolds, D. A. (2009). Gaussian mixture models. En *Encyclopedia of biometrics* (pp. 741-745). Springer. https://doi.org/10.1007/978-0-387-73003-5_196
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135-1144. <https://doi.org/10.1145/2939672.2939778>
- Richards, M. (2015). *Software Architecture Patterns*. O'Reilly Media, Inc. <https://www.oareilly.com/library/view/software-architecture-patterns/9781491971437/>
- Richardson, C. (2018). *Microservices Patterns: With examples in Java*. Manning Publications.

- Rousseeuw, P. J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics*, 20, 53-65. [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7)
- Sadalage, P. J., & Fowler, M. (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional.
- Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide — The Definitive Guide to Scrum: The Rules of the Game*.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28. <https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf>
- Shabani, I., Mëziu, E., Berisha, B., & Biba, T. (2021). Design of Modern Distributed Systems Based on Microservices Architecture. *International Journal of Advanced Computer Science and Applications*, 12(2), 161-168. <https://doi.org/10.14569/IJACSA.2021.0120220>
- Shilo, S., Rossman, H., & Segal, E. (2020). Axes of a Revolution: Challenges and Promises of Big Data in Healthcare. *Nature Medicine*, 26(1), 29-38. <https://doi.org/10.1038/s41591-019-0727-5>
- Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84-90.
- Sommerville, I. (2011). *Software Engineering* (9th). Addison-Wesley.
- Song, M., & Brooks, P. (2025). The Role of MLOps in Scalable Healthcare AI Systems.
- Sutton, R. S., & Barto, A. G. (1998). Reinforcement Learning: An Introduction. *IEEE Transactions on Neural Networks*. <https://doi.org/10.1109/TNN.1998.712192>
- Taye, M. M. (2023). Understanding of Machine Learning with Deep Learning: Architectures, Workflow, Applications and Future Directions. *Computers*, 12(5), 91. <https://doi.org/10.3390/computers12050091>

- The Cancer Genome Atlas Research Network. (2013). The Cancer Genome Atlas Pan-Cancer analysis project. *Nature Genetics*, 45(10), 1113-1120. <https://doi.org/10.1038/ng.2764>
- Tiangolo. (2024). *FastAPI Documentation: Alternatives, Web Servers and ASGI*. <https://fastapi.tiangolo.com/deployment/server-workers/>
- Vartak, M., Subramanyam, H., Lee, W.-E., Viswanathan, S., Husnoo, S., Madden, S., & Zaharia, M. (2016). ModelDB: a system for machine learning model management. *Proceedings of the Workshop on Human-In-the-Loop Data Analytics*, 1-3.
- Venkata, N. S. V. S. (2025). AI Model Deployment in Healthcare: MLOps Innovations and Challenges. *International Journal of Computer Science and Information Technology Research*, 6(1), 1-13. https://ijcsitr.com/index.php/home/article/view/IJCSITR_2025_06_01_001
- Voron, F. (2021). *Building Data Science Applications with FastAPI: Develop, manage, and deploy efficient machine learning applications with Python*. Packt Publishing.
- Waehner, K. (2020). *Use Cases and Architectures for HTTP and REST APIs with Apache Kafka*. <https://www.confluent.io/blog/http-and-rest-api-use-cases-and-architecture-with-apache-kafka/>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Bonino da Silva Santos, L., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., ... Mons, B. (2016). The FAIR Guiding Principles for Scientific Data Management and Stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- Willmott, C. J., & Matsuura, K. (2005). Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Climate research*, 30(1), 79-82. <https://doi.org/10.3354/cr030079>
- World Wide Web Consortium. (2015). *Server-Sent Events*. W3C Recommendation. <https://www.w3.org/TR/eventsource/>

Anexo A. Fundamentos Matemáticos de Algoritmos de Aprendizaje Automático

En este anexo se detallan las expresiones matemáticas y funciones de optimización que rigen el comportamiento de los algoritmos de ML.

1. Regresión Lineal (*Linear Regression*)

El modelo busca minimizar la suma de los cuadrados de los residuos (OLS) usando la Ecuación A.1, donde la función de costo a minimizar es la Ecuación A.2.

$$y = \beta_0 + \sum_{j=1}^p \beta_j X_j + \epsilon \quad (\text{A.1})$$

$$J(\beta) = \sum_{i=1}^n (y_i - (\beta_0 + \sum_{j=1}^p \beta_j x_{ij}))^2 \quad (\text{A.2})$$

2. Regresión Ridge

Añade una penalización L_2 a la función de costo de la regresión lineal para controlar la magnitud de los coeficientes, como se define en la Ecuación A.3.

$$J(\beta) = \sum_{i=1}^n (y_i - X_i \beta)^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad (\text{A.3})$$

Donde $\lambda \geq 0$ es el hiperparámetro de regularización.

3. Regresión Logística

Utiliza la función sigmoide para mapear valores reales al rango $(0, 1)$, representando una probabilidad mediante la Ecuación A.4.

$$P(y = 1|X) = \sigma(\beta^T X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots)}} \quad (\text{A.4})$$

4. Máquinas de Vectores de Soporte (SVC)

El objetivo es maximizar el margen entre clases resolviendo el problema de optimización cuadrática presentado en la Ecuación A.5.

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{sujeto a: } y_i(w^T x_i + b) \geq 1 \quad (\text{A.5})$$

5. Random Forest (Impureza de Gini)

Cada nodo en los árboles de decisión se divide minimizando la impureza de Gini (G), calculada según la Ecuación A.6.

$$G = 1 - \sum_{i=1}^C p_i^2 \quad (\text{A.6})$$

Donde p_i es la probabilidad de que un elemento pertenezca a la clase i .

6. K-Nearest Neighbors (KNN)

La métrica de distancia más común para identificar los vecinos cercanos es la distancia euclidiana, definida en la Ecuación A.7.

$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2} \quad (\text{A.7})$$

7. K-Means

El algoritmo busca minimizar la inercia o suma de cuadrados intra-clúster mostrada en la Ecuación A.8.

$$J = \sum_{j=1}^k \sum_{i \in C_j} \|x_i - \mu_j\|^2 \quad (\text{A.8})$$

Donde μ_j es el centroide del clúster C_j .

8. DBSCAN (Densidad)

Se basa en el concepto de ϵ -vecindad de un punto p , establecido en la Ecuación A.9.

$$N_\epsilon(p) = \{q \in D \mid \text{dist}(p, q) \leq \epsilon\} \quad (\text{A.9})$$

Un punto se considera núcleo si $|N_\epsilon(p)| \geq \text{MinPts}$.

9. Modelos de Mezcla Gaussiana (GMM)

Representa la distribución de los datos como una suma ponderada de K distribuciones normales, formulada en la Ecuación A.10.

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k) \quad (\text{A.10})$$

Donde π_k es la probabilidad de mezcla, μ_k la media y Σ_k la matriz de covarianza.

Anexo B. Especificación Casos de Uso

A continuación se presentan las fichas descriptivas detalladas de los casos de uso que componen el sistema. Estas especificaciones desglosan las precondiciones, postcondiciones y el flujo secuencial de eventos entre los actores y la plataforma.

Tabla 15. Especificación: CU-01 Gestión de Usuarios.

Caso de uso	CU-01 Gestión de Usuarios y Roles
Descripción	El administrador gestiona las cuentas de acceso a la plataforma y roles (RBAC).
Actores	Administrador
Precondiciones	Autenticación mediante JWT con privilegios de administrador.
Postcondiciones	Usuario registrado o modificado en PostgreSQL.
Descripción Flujo normal	
Actor	Sistema
1. Ingresar datos y rol.	2. Valida unicidad e inserta en PostgreSQL con clave encriptada.

Tabla 16. Especificación: CU-02 Ingesta de Datos.

Caso de uso	CU-02 Ingesta de Datos y Persistencia Híbrida
Descripción	Carga de datos crudos. El sistema atomiza los registros para almacenamiento masivo y registra su metadata.
Actores	Usuario, Investigador
Precondiciones	Usuario autenticado.
Postcondiciones	Metadata en PostgreSQL y documentos BSON en MongoDB (<i>raw_data</i>).
Descripción Flujo normal	
Actor	Sistema
1. Sube archivo tabular.	2. Node.js recibe y envía al microservicio de ingesta en Python.
3. Confirma carga.	4. Python registra en MongoDB y retorna el identificador de la colección.
5. Visualiza dataset.	6. Node.js vincula el <i>owner_id</i> con la referencia de MongoDB en PostgreSQL.

Tabla 17. Especificación: CU-03 Preprocesamiento de Datos.

Caso de uso	CU-03 Preprocesamiento y Vectorización
Descripción	Limpieza de nulos, selección de <i>target</i> y generación de diccionarios de traducción para variables categóricas.
Actores	Investigador
Precondiciones	Dataset original cargado (CU-02).
Postcondiciones	Dataset numérico y <i>mappings</i> de codificación guardados en MongoDB (<i>processed_data</i>).
Descripción Flujo normal	
Actor	Sistema
1. Selecciona columnas y método de codificación.	2. Node.js orquesta la petición al microservicio analítico correspondiente.
3. Valida resultados.	4. Python aplica <i>encoding</i> , genera diccionarios de mapeo y persiste la estructura final en MongoDB.

Tabla 18. Especificación: CU-04 Experimentación Visual.

Caso de uso	CU-04 Experimentación y Streaming Visual
Descripción	Evaluación interactiva de algoritmos en memoria mediante streaming de eventos.
Actores	Investigador
Postcondiciones	Visualización interactiva sin persistencia inmediata.
Descripción Flujo normal	
Actor	Sistema
1. Inicia prueba con hiperparámetros.	2. Node.js establece conexión SSE (Server-Sent Events) hacia Python.
3. Observa progreso.	4. Python emite progreso, métricas e inercia en tiempo real al frontend.

Tabla 19. Especificación: CU-05 MLOps y Repertorio de Modelos.

Caso de uso	CU-05 MLOps y Repertorio de Modelos
Descripción	Serialización y registro de modelos óptimos para su uso productivo.
Actores	Investigador
Postcondiciones	Artefacto <i>.joblib</i> en disco y registro en PostgreSQL.
Descripción Flujo normal	
Actor	Sistema
1. Etiqueta clusters y guarda.	2. Python serializa el modelo y guarda metadata en MongoDB.
3. Verifica catálogo.	4. Node.js registra la versión y la URI física en PostgreSQL.

Tabla 20. Especificación: CU-06 Ejecución de Inferencia.

Caso de uso	CU-06 Ejecución de Inferencia
Descripción	El usuario aplica un modelo entrenado sobre nuevos datos. El sistema procesa la información y muestra una vista previa inmediata.
Actores	Usuario, Investigador
Postcondiciones	Resultados visualizados en pantalla y persistencia automática en MongoDB para auditoría futura.
Descripción Flujo normal	
Actor	Sistema
1. Selecciona modelo y sube archivo.	2. Carga el artefacto <i>joblib</i> y realiza el cálculo matricial.
	3. Despliega tabla de resultados y habilita el punto de extensión para exportación.

Tabla 21. Especificación: CU-07 Exportación de Reportes.

Caso de uso	CU-07 Exportación de Reportes de Inferencia
Descripción	Generación de archivos descargables (Excel/CSV) con el detalle de las predicciones. Este caso extiende a CU-06.
Actores	Usuario, Investigador
Precondiciones	Inferencia ejecutada exitosamente (CU-06).
Postcondiciones	Archivo binario csv/pdf en el equipo local del usuario.
Descripción Flujo normal	
Actor	Sistema
1. Solicita descarga en formato específico.	2. Formatea la inferencia procesada según el estándar (XLSX/CSV/PDF) y gatilla la descarga.

Tabla 22. Especificación: CU-08 Historial de Inferencias.

Caso de uso	CU-08 Gestión de Historial y Auditoría de Uso
Descripción	Módulo de consulta. Permite recuperar resultados pasados y auditar el uso de la plataforma.
Actores	Usuario, Investigador, Administrador
Postcondiciones	Visualización de historial de inferencias y recuperación de activos desde MongoDB.
Descripción Flujo normal	
Actor	Sistema
1. Consulta historial.	2. Filtra registros por <i>usuario_id</i> (si es usuario) o muestra lista global (si es administrador).

Tabla 23. Especificación: CU-09 Historial de Entrenamientos.

Caso de uso	CU-09 Auditoría de Experimentos
Descripción	Revisión de métricas técnicas (R^2 , Loss) e hiperparámetros de todos los entrenamientos realizados.
Actores	Investigador, Administrador
Postcondiciones	Metadata técnica recuperada desde MongoDB (<i>model_metadata</i>).
Descripción Flujo normal	
Actor	Sistema
1. Ingresa a historial de modelos.	2. Recupera logs de ejecución y configuraciones JSON de los algoritmos.

Anexo C. Infraestructura y Orquestación de Contenedores

La arquitectura de la plataforma se fundamenta en la contenerización de procesos para garantizar la portabilidad y escalabilidad de los servicios. En este anexo se detallan las configuraciones de *Docker* y los mecanismos de orquestación empleados tanto en el entorno de desarrollo como en el de producción.

C.1. Empaquetamiento de Imágenes (Dockerfiles)

A continuación, se presentan los archivos de configuración para la construcción (*build*) de imágenes individuales. Los microservicios analíticos se basan en *Python 3.9-slim*, mientras que el servidor central requiere herramientas adicionales para la gestión de contenedores.

C.1.1. Microservicios de Procesamiento Analítico

Cada microservicio basado en FastAPI se encapsula en una imagen ligera. El Código C.1 muestra la estructura estándar aplicada, ejemplificada con la configuración del servicio de Clasificación.

Código C.1. Dockerfile representativo de Microservicios Python

```
1 FROM python:3.9-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY . .
9
10 # El puerto varia segun el servicio (ej. 8008 para clasificacion)
```

```
11 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8008"]
```

C.1.2. Backend Gateway

La imagen del *Backend* (Código C.2) incorpora la *CLI de Docker* para permitir que la aplicación Node.js interactúe con el motor de contenedores del sistema host.

Código C.2. Dockerfile del Backend (API Gateway)

```
1 FROM node:20-alpine
2
3 # Instalacion de herramientas para interaccion con el motor de Docker
4 RUN apk add --no-cache docker-cli
5
6 WORKDIR /app
7 COPY package*.json ./
8 RUN npm ci --no-audit --no-fund
9 COPY . .
10 EXPOSE 3000
11 CMD ["npm", "start"]
```

C.1.3. Frontend y Estrategia de Construcción

Para el *Frontend*, en el Código C.3 se detalla la construcción en múltiples etapas (*Multi-stage Build*) para producción.

Código C.3. Dockerfile de producción para el Frontend (Multi-stage)

```
1 # Etapa 1: Construcción (build)
2 FROM node:20-alpine AS builder
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci --no-audit --no-fund
6 COPY . .
7 RUN npm run build
8
9 # Etapa 2: Servidor estatico con NGINX
10 FROM nginx:1.27-alpine
11 RUN rm /etc/nginx/conf.d/default.conf
12 COPY nginx.conf /etc/nginx/conf.d/default.conf
13 COPY --from=builder /app/build /usr/share/nginx/html
14 EXPOSE 80
15 CMD ["nginx", "-g", "daemon off;"]
```

C.2. Orquestación y Levantamiento con Docker Compose

Si bien en la sección anterior se detalló el empaquetamiento de las imágenes de forma aislada, en la práctica, la construcción (*build*) y el levantamiento de los servicios se ejecutan de manera conjunta utilizando *Docker Compose*.

Integrar ambas fases en una única herramienta presenta varias ventajas desde la perspectiva de desarrollo. Permite a los desarrolladores levantar todo el ecosistema con un solo comando sin necesidad de compilar manualmente cada imagen, asegura la consistencia entre el código fuente y las variables de red local, y facilita enormemente la automatización en pipelines de integración y despliegue continuo (CI/CD). Al delegar la responsabilidad en Compose, la infraestructura como código se vuelve más mantenible, cómoda y reproducible.

Para la gestión integral de los microservicios, se definieron dos configuraciones de orquestación. La primera, orientada al desarrollo, prioriza la actualización en tiempo real (*hot-reload*) y la exposición de herramientas de administración.

En C.4 se observa el código para la configuración de desarrollo.

Código C.4. Archivo de orquestación para el entorno de desarrollo
(`docker-compose.dev.yml`)

```

1  services:
2    back:
3      build:
4        context: ./back
5      container_name: dscience-back-dev
6      command: npm run dev
7      ports:
8        - "3000:3000"
9      volumes:
10       - ./back:/app
11       - /app/node_modules
12       - /var/run/docker.sock:/var/run/docker.sock
13       - models_storage:/app/models_storage
14      env_file:
15       - .env.dev
16      depends_on:
17       - postgres
18       - mongo
19       - ollama
20       - ingestion_service
21       - preprocessing_service
22       - prediction_service
23       - clustering_service
24       - clasification_service
25       - inference_service
26      networks:
27       - dscience-net
28
29
30  ollama:
31    image: ollama/ollama
32    container_name: dscience-ollama
33    restart: unless-stopped

```

```

34  ports:
35    - "11434:11434"
36  dns:
37    - 1.1.1.1
38    - 8.8.8.8
39  volumes:
40    - ollama_data:/root/.ollama
41  deploy:
42    resources:
43      reservations:
44        devices:
45          - driver: nvidia
46            count: all
47            capabilities: [gpu]
48  networks:
49    - dscience-net
50
51  ingestion_service:
52    build:
53      context: ./microservices/ingestion
54    container_name: dscience-ingestion
55    ports:
56      - "8020:8000"
57    environment:
58      - MONGO_URI=mongodb://mongo:27017
59    networks:
60      - dscience-net
61
62  preprocessing_service:
63    build:
64      context: ./microservices/preprocessing
65    container_name: dscience-preprocessing
66    ports:
67      - "8005:8005"
68    environment:
69      - MONGO_URI=mongodb://mongo:27017
70    networks:
71      - dscience-net
72
73  prediction_service:
74    build:
75      context: ./microservices/prediction
76    container_name: dscience-prediction
77    ports:
78      - "8006:8006"
79    volumes:
80      # Volumen PERSISTENTE para modelos guardados
81      - models_storage:/app/models_storage
82    environment:
83      - MONGO_URI=mongodb://mongo:27017
84    networks:
85      - dscience-net
86
87  clustering_service:
88    build:
89      context: ./microservices/clustering
90    container_name: dscience-clustering
91    ports:
92      - "8007:8007"
93    volumes:
94      # Volumen PERSISTENTE para modelos guardados
95      - models_storage:/app/models_storage
96    environment:
97      - MONGO_URI=mongodb://mongo:27017
98    networks:
99      - dscience-net
100
101  clasification_service:
102    build:
103      context: ./microservices/clasificacion
104    container_name: dscience-clasification
105    ports:
106      - "8008:8008"
107    volumes:
108      # Volumen PERSISTENTE para modelos guardados
109      - models_storage:/app/models_storage
110    environment:

```

```

111     - MONGO_URI=mongodb://mongo:27017
112     networks:
113     - dscience-net
114
115 inference_service:
116     build:
117     context: ./microservices/inferencia
118     container_name: dscience-inferencia
119     ports:
120     - "8010:8010"
121     volumes:
122     - models_storage:/app/models_storage
123     environment:
124     - MONGO_URI=mongodb://mongo:27017
125     depends_on:
126     - mongo
127     - postgres
128     networks:
129     - dscience-net
130
131 postgres:
132     image: postgres:16
133     container_name: dscience-postgres-dev
134     restart: always
135     env_file:
136     - .env.dev
137     ports:
138     - "5432:5432"
139     volumes:
140     - postgres_data:/var/lib/postgresql/data
141     - ./back/src/db/init.sql:/docker-entrypoint-initdb.d/init.sql
142     networks:
143     - dscience-net
144
145 mongo:
146     image: mongo:7
147     container_name: dscience-mongo-dev
148     restart: always
149     ports:
150     - "27017:27017"
151     volumes:
152     - mongo_data:/data/db
153     networks:
154     - dscience-net
155
156 pgadmin:
157     image: dpage/pgadmin4
158     container_name: dscience-pgadmin
159     restart: always
160     env_file:
161     - .env.dev
162     ports:
163     - "5050:80"
164     depends_on:
165     - postgres
166     networks:
167     - dscience-net
168
169 mongo-express:
170     image: mongo-express
171     container_name: dscience-mongo-express
172     restart: always
173     env_file:
174     - .env.dev
175     ports:
176     - "8082:8081"
177     depends_on:
178     - mongo
179     networks:
180     - dscience-net
181
182 front:
183     build:
184     context: ./front
185     dockerfile: Dockerfile.dev
186     container_name: dscience-front-dev
187     command: npm start

```

```

188     depends_on:
189       - back
190     ports:
191       - "8081:3000"
192     volumes:
193       - ./front:/app
194       - /app/node_modules
195     env_file:
196       - .env.dev
197     networks:
198       - dscience-net
199
200 networks:
201   dscience-net:
202     driver: bridge
203
204 volumes:
205   postgres_data:
206   mongo_data:
207   models_storage:
208   ollama_data:

```

Para el despliegue en producción, se utiliza una configuración que incorpora un servidor *Nginx* como proxy inverso y elimina la exposición pública de los puertos internos de las bases de datos. El Código C.5 presenta el archivo correspondiente.

Código C.5. Archivo de orquestación para el entorno de producción (docker-compose.yml)

```

1  services:
2    front:
3      build:
4        context: ./front
5        dockerfile: Dockerfile
6      container_name: dscience-front-prod
7      restart: unless-stopped
8      depends_on:
9        - back
10     env_file:
11       - .env.prod
12     ports:
13       - "80:80"
14     networks:
15       - dscience-net
16
17   back:
18     build:
19       context: ./back
20     container_name: dscience-back
21     restart: unless-stopped
22     env_file:
23       - .env.prod
24     ports:
25       - "3000:3000"
26     volumes:
27       - models_storage:/app/models_storage
28       - /var/run/docker.sock:/var/run/docker.sock
29     depends_on:
30       postgres:
31         condition: service_healthy
32       mongo:
33         condition: service_started
34       ollama:
35         condition: service_started
36     networks:
37       - dscience-net
38
39   ollama:
40     image: ollama/ollama

```

```

41  container_name: dscience-ollama
42  restart: unless-stopped
43  expose:
44    - "11434"
45  dns:
46    - 1.1.1.1
47    - 8.8.8.8
48  volumes:
49    - ollama_data:/root/.ollama
50  gpus: all
51  networks:
52    - dscience-net
53
54
55  postgres:
56    image: postgres:16
57    container_name: dscience-postgres
58    restart: unless-stopped
59    env_file:
60      - .env.prod
61    volumes:
62      - postgres_data:/var/lib/postgresql/data
63      - ./back/src/db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
64    expose:
65      - "5432"
66    networks:
67      - dscience-net
68    healthcheck:
69      test: ["CMD-SHELL", "pg_isready -U $$POSTGRES_USER -d $$POSTGRES_DB"]
70      interval: 10s
71      timeout: 5s
72      retries: 10
73
74  mongo:
75    image: mongo:7
76    container_name: dscience-mongo
77    restart: unless-stopped
78    volumes:
79      - mongo_data:/data/db
80    expose:
81      - "27017"
82    networks:
83      - dscience-net
84
85  ingestion_service:
86    build:
87      context: ./microservices/ingestion
88    container_name: dscience-ingestion
89    restart: unless-stopped
90    environment:
91      - MONGO_URI=mongodb://mongo:27017
92    expose:
93      - "8000"
94    networks:
95      - dscience-net
96
97  preprocessing_service:
98    build:
99      context: ./microservices/preprocessing
100    container_name: dscience-preprocessing
101    restart: unless-stopped
102    environment:
103      - MONGO_URI=mongodb://mongo:27017
104    expose:
105      - "8005"
106    networks:
107      - dscience-net
108
109  prediction_service:
110    build:
111      context: ./microservices/prediction
112    container_name: dscience-prediction
113    restart: unless-stopped
114    environment:
115      - MONGO_URI=mongodb://mongo:27017
116      - MODELS_DIR=/app/models_storage
117    expose:

```



```

118     - "8006"
119     volumes:
120     - models_storage:/app/models_storage
121     networks:
122     - dscience-net
123
124 clustering_service:
125     build:
126     context: ./microservices/clustering
127     container_name: dscience-clustering
128     restart: unless-stopped
129     environment:
130     - MONGO_URI=mongodb://mongo:27017
131     - MODELS_DIR=/app/models_storage
132     expose:
133     - "8007"
134     volumes:
135     - models_storage:/app/models_storage
136     networks:
137     - dscience-net
138
139 clasificacion_service:
140     build:
141     context: ./microservices/clasificacion
142     container_name: dscience-clasificacion
143     restart: unless-stopped
144     environment:
145     - MONGO_URI=mongodb://mongo:27017
146     - MODELS_DIR=/app/models_storage
147     expose:
148     - "8008"
149     volumes:
150     - models_storage:/app/models_storage
151     networks:
152     - dscience-net
153
154 inference_service:
155     build:
156     context: ./microservices/inferencia
157     container_name: dscience-inferencia
158     restart: unless-stopped
159     environment:
160     - MONGO_URI=mongodb://mongo:27017
161     - MODELS_DIR=/app/models_storage
162     expose:
163     - "8010"
164     volumes:
165     - models_storage:/app/models_storage
166     depends_on:
167     - mongo
168     networks:
169     - dscience-net
170
171 networks:
172     dscience-net:
173     driver: bridge
174
175 volumes:
176     postgres_data:
177     mongo_data:
178     models_storage:
179     ollama_data:

```

C.3. Guía de Despliegue de la Plataforma

Esta sección describe los pasos necesarios para el despliegue de la plataforma en la infraestructura de microservicios. Se asume un entorno basado en Linux/Unix con

privilegios de administrador para la ejecución de contenedores.

C.3.1. Requisitos Previos

Antes de iniciar el despliegue, el sistema anfitrión debe contar con las siguientes herramientas instaladas:

- **Git:** Para la obtención del código fuente.
- **Docker Engine:** Motor de ejecución de contenedores.
- **Docker Compose:** Orquestador de servicios.

C.3.2. Paso 1: Obtención del Código Fuente

El proyecto se encuentra centralizado en el repositorio institucional de CEDETEC. Para replicar la plataforma, se debe clonar el monorepo mediante el siguiente comando:

Código C.6. Clonación del repositorio oficial

```
1 git clone https://github.com/cedetec/dscience-platform.git
2 cd dscience-platform
```

C.3.3. Paso 2: Configuración del Entorno (.env)

La plataforma requiere variables de entorno para gestionar secretos y rutas de red. El contenido detallado y la definición de cada variable se encuentran especificados en el Anexo D.

1. **Variables de Infraestructura:** Crear el archivo `.env.dev` siguiendo el esquema de claves de seguridad de PostgreSQL y MongoDB detallado al final de este anexo.
2. **Variables de Cliente:** En la carpeta `/front`, crear un archivo `.env` que defina la variable `REACT_APP_API_URL` para que el navegador sepa dónde encontrar el Gateway.

C.3.4. Paso 3: Ejecución de la Plataforma

Dependiendo del objetivo de la sesión, se disponen de dos métodos de arranque:

A. Modo Desarrollo (con Hot-Reload y herramientas de administración): Este modo levanta las interfaces de *PgAdmin* y *Mongo Express*.

Código C.7. Levantamiento en Modo Desarrollo

```
1 docker-compose -f docker-compose.dev.yml up --build
```

B. Modo Producción (Optimizado con Nginx): Este modo utiliza la configuración del Código C.5, ideal para evaluaciones de rendimiento y uso final.

Código C.8. Levantamiento en Modo Producción

```
1 docker-compose -f docker-compose.yml up -d --build
```

C.3.5. Paso 4: Verificación del Despliegue

Una vez levantados los servicios, la plataforma es accesible a través de las siguientes rutas locales por defecto:

- **Interfaz Web:** `http://localhost:8081` (Desarrollo) o `http://localhost` (Producción).
- **Métricas de Microservicios:** Consultar el endpoint `/health` de cada puerto asignado (8000-8010).

Anexo D. Configuración y Seguridad del Sistema

Este anexo describe los parámetros de configuración necesarios para el despliegue de la plataforma. El sistema utiliza variables de entorno para gestionar credenciales y rutas de conexión de forma segura.

D.1. Parámetros del Entorno

La Tabla 24 resume las principales variables de entorno requeridas por la aplicación, clasificadas por su área de influencia técnica.

Tabla 24. Resumen de variables de entorno configuradas.

Categoría	Variable	Propósito
Base de Datos	POSTGRES_DB	Nombre de la instancia relacional.
Base de Datos	MONGO_URI	Cadena de conexión para la base documental.
Seguridad	JWT_ACCESS_SECRET	Clave privada para la firma de tokens.
Frontend	REACT_APP_API_URL	Endpoint de la API para el cliente.
Entorno	NODE_ENV	Define el modo de ejecución (development/production).

Finalmente, en el Código D.1, se presenta el archivo `.env.dev` completo utilizado para centralizar estas configuraciones.

Código D.1. Configuración del archivo de entorno (`.env.dev`)

1

=====

2

PostgreSQL

3

=====

4

POSTGRES_USER=postgres

```
5 POSTGRES_PASSWORD=1234
6 POSTGRES_DB=dscience
7 POSTGRES_HOST=postgres
8 # =====
9 # MongoDB
10 # =====
11 MONGO_URI=mongodb://mongo:27017/dscience
12 # =====
13 # Herramientas Administrativas
14 # =====
15 PGADMIN_DEFAULT_EMAIL=admin@admin.com
16 PGADMIN_DEFAULT_PASSWORD=admin
17 ME_CONFIG_MONGODB_SERVER=mongo
18 ME_CONFIG_MONGODB_PORT=27017
19 ME_CONFIG_BASICAUTH_USERNAME=admin
20 ME_CONFIG_BASICAUTH_PASSWORD=admin
21 # =====
22 # Configuracion de Aplicacion
23 # =====
24 NODE_ENV=development
25 CHOKIDAR_USEPOLLING=true
26 REACT_APP_API_URL=http://localhost:3000/api
27 # Seguridad JWT
28 JWT_ACCESS_SECRET=secretclave
29 JWT_REFRESH_SECRET=clavesecreta
30 JWT_ACCESS_EXPIRES=15m
31 JWT_REFRESH_EXPIRES=7d
```

Anexo E. Especificaciones Técnicas de los Microservicios Analíticos

Este anexo detalla los aspectos técnicos de implementación de los microservicios de la plataforma, fundamentados en el uso de *FastAPI* y *Scikit-Learn*. Se describen los protocolos de comunicación asíncrona, la estructura de persistencia políglota y la composición de los artefactos de modelos serializados.

E.1. Protocolo de Comunicación en Tiempo Real (Server-Sent Events)

Para reportar el progreso de los algoritmos de entrenamiento sin bloquear el hilo principal del *API Gateway*, se implementó el protocolo *SSE (Server-Sent Events)*. La implementación utiliza el generador asíncrono de *Python* y la clase `StreamingResponse` para emitir paquetes de datos bajo el tipo de contenido `text/event-stream`.

E.1.1. Estructura de Payloads (Mensajes JSON)

Los microservicios analíticos (*Classification*, *Clustering*, *Prediction*) emiten mensajes estructurados en formato JSON que permiten al frontend actualizar la interfaz de forma reactiva:

1. Estado de Progreso (`status: 'training'`):

- `epoch / iter`: Representa el paso actual del ciclo de entrenamiento.
- `accuracy / inertia`: Métrica de rendimiento parcial calculada en la iteración.

Tabla 25. Métricas por tipo de tarea analítica e implementación.

Tarea	Métricas Implementadas	Función / Método de Scikit-Learn
Clasificación	Accuracy, F1-Macro, Precision-Macro, Recall-Macro	accuracy_score, precision_recall_fscore_support
Predicción	R^2 Score, MSE, MAE	r2_score, mean_squared_error, mean_absolute_error
Clustering	Coeficiente de Silueta, Davies-Bouldin, Inercia	silhouette_score, davies_bouldin_score, model.inertia_

- `message`: Cadena de texto informativa (ej: "Iniciando entrenamiento...", "Agrupando iteración X/Y").

2. Estado Completado (`status: 'completed'`):

- `metrics`: Objeto con los resultados finales de validación técnica.
- `results / scatter_data`: Estructuras vectoriales optimizadas para visualización gráfica.

3. Estado de Error (`status: 'error'`): Reporta excepciones capturadas mediante `traceback`, enviando un detalle textual del fallo para su depuración en el cliente.

E.2. Especificación de Métricas Analíticas

El sistema automatiza el cálculo de métricas de rendimiento utilizando la suite de `sklearn.metrics`. Se priorizan las métricas ponderadas (*weighted*) y macro para mitigar el efecto de clases desbalanceadas.

E.3. Persistencia de Resultados de Inferencia (Auditoría)

El microservicio de **Inferencia** implementa un flujo de persistencia automática en la colección `inference_results` de *MongoDB*. Este mecanismo asegura que cada predicción entregada a un usuario sea auditable técnica y científicamente.

E.3.1. Estructura del Documento de Inferencia

Cada registro en la base de datos documental sigue el siguiente esquema:

- `file_name`: Identificador del archivo fuente subido por el usuario.
- `algorithm`: Identificador de la tarea (*classification*, *clustering*, *prediction*).
- `details`: Arreglo de objetos JSON que contiene los datos originales concatenados con la columna Predicción (o Valor_Predicho) y, de ser posible, la métrica de Confianza.
- `total_rows`: Cantidad de registros procesados en la inferencia.
- `created_at`: Marca de tiempo en formato UTC (*datetime.utcnow()*).

E.4. Estructura del Artefacto de Persistencia (.joblib)

Para garantizar la reproducibilidad y la consistencia dimensional, cada modelo se exporta como un "Artefacto de Inferencia" serializado mediante la librería `joblib`. Este diseño permite que el microservicio de inferencia sea totalmente agnóstico al entrenamiento original.

E.4.1. Componentes del Artefacto

El artefacto es un diccionario de *Python* que encapsula:

- `model`: Estimador de *Scikit-Learn* totalmente ajustado.
- `scaler`: Instancia de `StandardScaler` con los parámetros de media y desviación estándar del entrenamiento original.
- `label_encoder`: Instancia para la decodificación de etiquetas categóricas a su valor textual original.
- `mappings`: Diccionario de preprocesamiento para variables independientes categóricas.

- `feature_names_in`: Lista ordenada de columnas para asegurar la alineación de datos en la inferencia (CU-06).
- `algorithm`: Identificador técnico del algoritmo utilizado (ej: `rf`, `svc`, `kmeans`, `dbscan`).

E.5. Organización del Almacenamiento y Volúmenes

Los archivos binarios se almacenan en un volumen persistente de *Docker* (`models_storage`) mapeado a la ruta física `MODELS_DIR`.

E.5.1. Convención de Nomenclatura y Trazabilidad

Los archivos se nombran utilizando el prefijo de la tarea seguido de un UUID v4 (`clf_[UUID].joblib`, `pred_[UUID].joblib`, `cluster_[UUID].joblib`). Simultáneamente, se registra un documento en la colección `model_metadata` de *MongoDB* que vincula la URI del archivo con los hiperparámetros técnicos, la duración del entrenamiento en segundos (`duration_sec`) y el resumen estadístico del dataset original (`dataset_summary`).

Anexo F. Contratos de la API REST (API Gateway)

Este anexo describe la interfaz de programación de aplicaciones (API) que actúa como punto de entrada único al sistema. La API orquesta la comunicación entre el cliente web y los microservicios, gestionando una arquitectura de persistencia polígota donde la metadata reside en PostgreSQL y los grandes volúmenes de datos resultantes de las inferencias o entrenamientos se almacenan en MongoDB.

F.1. Convenciones Generales

Para garantizar consistencia en el consumo de los servicios, la API sigue las siguientes convenciones:

- **Base URL:** `http://<host>:3000/api`
- **Autenticación:** Uso de *JSON Web Tokens (JWT)* mediante el encabezado `Authorization: Bearer <token>`.
- **Headers de Identificación:** Para fines de trazabilidad, se emplean los encabezados opcionales `x-usuario-id` y `x-rol-id`.
- **Manejo de errores:** La API retorna códigos HTTP estándar (4xx/5xx) junto con un objeto JSON descriptivo tal como se muestra en el Código F.1:

Código F.1. Formato general de respuesta de error.

```
1 { "ok": false,  
2   "msg": "Descripcion detallada del error" }
```

Tabla 26. Endpoints de autenticación y administración de usuarios.

Método	Ruta	Descripción
POST	/auth/login	Inicio de sesión y generación de token JWT.
GET	/usuarios	Obtiene el listado de usuarios y sus roles asociados.
POST	/usuarios	Crea una nueva cuenta de usuario con contraseña encriptada.
PUT	/usuarios/:id	Actualiza datos de perfil, rol o contraseña de un usuario.
DELETE	/usuarios/:id	Elimina permanentemente un usuario del sistema.
GET	/usuarios/roles	Lista los roles disponibles (Admin, Investigador, Usuario).

F.2. Autenticación y Gestión de Usuarios

Este módulo gestiona la seguridad y el control de acceso basado en roles (RBAC). Las contraseñas son procesadas mediante funciones de *hashing* (*bcrypt*) antes de su persistencia en PostgreSQL. Los endpoints correspondientes se presentan en la Tabla 26.

F.3. Ingesta y Gestión de Datos

Este módulo coordina la persistencia inicial en MongoDB y el registro de la metadata transaccional en PostgreSQL, incluyendo la limpieza sistemática de activos como se aprecia en la Tabla 27.

F.4. Preprocesamiento de Datos

Coordina la ejecución de lógica de limpieza matricial y generación de *mappings* categóricos en el microservicio especializado como se aprecia en la Tabla 28.

Tabla 27. Endpoints del módulo de ingesta de datos.

Método	Ruta	Descripción
POST	/ingesta/upload	Carga de datasets originales y atomización masiva en MongoDB.
GET	/ingesta/documentos	Lista metadatos de documentos originales (Raw).
GET	/ingesta/documentosprocesados	Lista datasets vectorizados listos para entrenamiento.
DELETE	/ingesta/documentos/:id	Borra el registro SQL y las filas asociadas en MongoDB (Raw).
DELETE	/ingesta/documentosprocesados/:id	Elimina la matriz procesada de SQL y MongoDB.
GET	/ingesta/documentos/:id/preview	Recupera una vista previa (50 filas) desde MongoDB.
GET	/ingesta/documentos/:id/columnas	Analiza y retorna la estructura de columnas detectada.
GET	/ingesta/documentos/:id/download	Reconstruye y descarga el dataset en formato CSV.
GET	/ingesta/stats	Retorna contadores globales de documentos y estados.

Tabla 28. Endpoints del módulo de preprocesamiento.

Método	Ruta	Descripción
POST	/preprocessing/procesar	Orquesta el encoding y manejo de nulos, registrando el resultado en el esquema políglota.

F.5. Entrenamiento e Inferencia de Modelos

Este módulo implementa el flujo de persistencia políglota coordinada, capturando la identidad del creador y calculando la duración precisa de la computación como se aprecia en la Tabla 29.

Tabla 29. Endpoints asociados al ciclo de vida de modelos.

Método	Ruta	Descripción
POST	/modelos/prediction	Proxy SSE para entrenamiento de regresión en tiempo real.
POST	/modelos/classification	Proxy SSE para entrenamiento de clasificación en tiempo real.
POST	/modelos/clustering	Proxy SSE para ejecución de algoritmos no supervisados.
POST	/modelos/guardar	Serializa el artefacto (.joblib), asocia el created_by y guarda metadatos técnicos.
POST	/modelos/inferir	Ejecuta predicción y persiste automáticamente el resultado para auditoría.
GET	/modelos/disponibles	Lista modelos publicados aptos para inferencia.

Tabla 30. Endpoints de monitorización de infraestructura.

Método	Ruta	Descripción
GET	/infra/docker-stats	Recupera telemetría en tiempo real (CPU, Memoria, Uptime) de los microservicios.

F.6. Monitorización de Infraestructura

Módulo encargado de la observabilidad del sistema, permitiendo auditar el estado de salud de los contenedores Docker que componen la arquitectura como se aprecia en la Tabla 30.

F.7. Historial y Trazabilidad de Experimentos

Sección dedicada a la recuperación de auditoría científica y resultados históricos almacenados bajo el esquema de persistencia políglota como se aprecia en la Tabla 31.

Tabla 31. Endpoints de trazabilidad y recuperación de resultados.

Método	Ruta	Descripción
GET	/modelos/training-runs	Consulta la bitácora de experimentos, incluyendo métricas y autoría.
GET	/modelos/inferencias	Lista el historial de uso, filtrando por privacidad según el rol.
GET	/modelos/inferencias/:id/detalle	Recupera el dataset volumétrico de inferencia desde MongoDB.

F.8. Ejemplos de Respuestas JSON (Contratos de Salida)

A continuación se detallan los esquemas de respuesta de los *endpoints* críticos, los cuales sirven como base para la integración entre el *API Gateway* y el cliente web.

F.8.1. Autenticación y Sesión

Endpoint: POST /api/auth/login

Respuesta tras un inicio de sesión exitoso. Incluye el *token* para la cabecera *Authorization* y el perfil del usuario para la gestión del *localStorage* y rutas protegidas v.

Código F.2. Respuesta de Login exitoso.

```

1 {
2   "ok": true,
3   "msg": "Login exitoso",
4   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
5   "user": {
6     "id": 1,
7     "nombre": "Vicente Investigador",
8     "email": "vicente.santos@usach.cl",
9     "rol_id": 2
10  }
11 }
```

F.8.2. Gestión de Ingesta y Estadísticas

Endpoint: GET /api/ingesta/stats

Respuesta del motor de búsqueda de metadatos. Demuestra el cruce de información entre la base de datos relacional y documental para el conteo global de activos como se aprecia en el Código F.3.

Código F.3. Respuesta de estadísticas globales de activos.

```
1 {  
2   "ok": true,  
3   "stats": {  
4     "total": 15,  
5     "procesados": 8,  
6     "pendientes": 7  
7   }  
8 }
```

F.8.3. Monitorización de Microservicios

Endpoint: GET /api/infra/docker-stats

Datos obtenidos directamente del *daemon* de Docker a través del *socket* mapeado. Permite la observabilidad del clúster en tiempo real como se aprecia en el Código F.4.

Código F.4. Respuesta de telemetría de contenedores (Docker Stats).

```
1 {  
2   "ok": true,  
3   "data": [  
4     {  
5       "id": "9d730fa92484",  
6       "name": "dscience-front-prod",  
7     }  
8   ]  
9 }
```

```

7     "status": "running",
8     "cpu": "0.05%",
9     "mem": "12.4MB",
10    "uptime": "Up 3 hours"
11  },
12  {
13    "id": "3281c7176f36",
14    "name": "dscience-back",
15    "status": "running",
16    "cpu": "0.12%",
17    "mem": "45.8MB",
18    "uptime": "Up 3 hours"
19  }
20 ]
21 }

```

F.8.4. Historial de Entrenamientos y Trazabilidad

Endpoint: GET /api/modelos/training-runs

Respuesta del historial técnico. Se observa la inclusión de la identidad del creador y las métricas detalladas, evidenciando el cumplimiento de la trazabilidad científica como se aprecia en el Código F.5.

Código F.5. Respuesta de listado de Training Runs.

```

1 [
2   {
3     "run_id": 6,
4     "nombre_modelo": "Prediccion cargo seguros",
5     "algoritmo": "prediction",
6     "status": "SUCCESS",

```



```

7   "duration_sec": 0.45,
8   "creador_nombre": "Vicente Investigador",
9   "creador_email": "vicente.santos@usach.cl",
10  "metrics": {
11    "r2_score": 0.892,
12    "mae": 12.45,
13    "mse": 154.2
14  },
15  "started_at": "2026-03-10T19:36:36.000Z"
16 }
17 ]

```

F.8.5. Canal de Streaming (Server-Sent Events)

Endpoint: POST /api/modelos/prediction (o similares analíticos)

Ejemplo de un *chunk* de datos enviado durante el proceso de entrenamiento asíncrono para la actualización reactiva de la interfaz como se aprecia en el Código F.6.

Código F.6. Mensaje de progreso en tiempo real (SSE).

```

1 {
2   "status": "training",
3   "epoch": 3,
4   "accuracy": 0.75,
5   "message": "Optimizando pesos: Epoch 3/5"
6 }

```

Anexo G. Esquema Relacional y Estructura de Persistencia

Este anexo detalla la implementación física de la base de datos relacional (PostgreSQL) utilizada por el API Gateway para la gestión de usuarios, gobernanza de documentos y trazabilidad de experimentos. Se incluye el script DDL completo para garantizar la reproducibilidad del entorno.

G.1. Script de Creación de Objetos (DDL)

El siguiente script define las extensiones, tablas, restricciones de integridad referencial e inserciones iniciales de configuración como se aprecia en el Código G.1.

Código G.1. Definición de tablas y relaciones en PostgreSQL

```
1
2  -- Extensiones para identificadores únicos
3  CREATE EXTENSION IF NOT EXISTS "uuid-oss";
4
5  -- Gestión de Identidades y Roles (RBAC)
6  CREATE TABLE rol (
7      id SERIAL PRIMARY KEY,
8      nombre VARCHAR(50) NOT NULL
9  );
10
11  CREATE TABLE usuario (
```

```

12     id SERIAL PRIMARY KEY,
13     email VARCHAR(150) UNIQUE NOT NULL,
14     nombre VARCHAR(150) NOT NULL,
15     password VARCHAR(100) NOT NULL,
16     rol_id INTEGER REFERENCES rol(id) ON DELETE SET NULL,
17     created_at TIMESTAMP DEFAULT NOW()
18 );
19
20 -- Gobernanza y Linaje de Datos
21 CREATE TABLE documentos (
22     id SERIAL PRIMARY KEY,
23     titulo VARCHAR(255),
24     owner_id INTEGER REFERENCES usuario(id) ON DELETE CASCADE,
25     mongo_ref VARCHAR(255),
26     size BIGINT,
27     creado_en TIMESTAMP DEFAULT NOW()
28 );
29
30 CREATE TABLE datasets_procesados (
31     id SERIAL PRIMARY KEY,
32     documento_original_id INTEGER REFERENCES documentos(id)
33         ON DELETE CASCADE,
34     nombre_configuracion VARCHAR(150),
35     target_column VARCHAR(100),
36     mongo_ref VARCHAR(255),
37     task_type VARCHAR(255),
38     size BIGINT,
39     created_at TIMESTAMP DEFAULT NOW()
40 );
41

```

```
42 -- Ciclo de Vida de Modelos (MLOps)
43 CREATE TABLE modelos (
44     id SERIAL PRIMARY KEY,
45     nombre VARCHAR(255),
46     descripcion TEXT,
47     tipo_problema VARCHAR(50),
48     owner_id INTEGER REFERENCES usuario(id) ON DELETE SET NULL,
49     created_at TIMESTAMP DEFAULT NOW()
50 );
51
52 CREATE TABLE versiones_modelos (
53     id SERIAL PRIMARY KEY,
54     modelo_id INTEGER REFERENCES modelos(id) ON DELETE CASCADE,
55     version VARCHAR(50),
56     modelo_uri TEXT,
57     mongo_ref VARCHAR(255),
58     created_at TIMESTAMP DEFAULT NOW()
59 );
60
61 -- Trazabilidad de Experimentos y Ejecución
62 CREATE TABLE entrenamientos (
63     id SERIAL PRIMARY KEY,
64     modelo_id INTEGER REFERENCES modelos(id) ON DELETE CASCADE,
65     version VARCHAR(50),
66     documento_id INTEGER REFERENCES documentos(id) ON DELETE CASCADE,
67     estado VARCHAR(50),
68     mongo_id VARCHAR(255),
69     creado_en TIMESTAMP DEFAULT NOW()
70 );
71
```

```

72 CREATE TABLE training_runs (
73     run_id          BIGSERIAL PRIMARY KEY,
74     version_id      BIGINT NULL,
75     algoritmo        TEXT NOT NULL,
76     status           TEXT NOT NULL,
77     created_at       TIMESTAMPTZ DEFAULT NOW(),
78     started_at       TIMESTAMPTZ,
79     ended_at         TIMESTAMPTZ,
80     duration_sec     NUMERIC,
81     dataset_name     TEXT,
82     n_rows           INTEGER,
83     hyperparams      JSONB,
84     metrics          JSONB,
85     error_message    TEXT
86 );
87
88 CREATE TABLE inferencias (
89     id SERIAL PRIMARY KEY,
90     version_modelo_id INTEGER REFERENCES versiones_modelos(id)
91         ON DELETE CASCADE,
92     usuario_id INTEGER REFERENCES usuario(id) ON DELETE CASCADE,
93     mongo_detail_ref VARCHAR(255),
94     created_at TIMESTAMP DEFAULT NOW()
95 );
96
97 -- Datos Maestros Iniciales
98 INSERT INTO rol (nombre) VALUES ('Admin'), ('Investigador'),
99     ('Usuario');

```

G.2. Estructura del Volumen de Modelos (`models_storage`)

La persistencia física de los artefactos se organiza de forma plana dentro de un volumen persistente de Docker para optimizar la velocidad de lectura durante la fase de inferencia.

- `/app/models_storage/`
 - `clf_[uuid].joblib` — Artefactos de clasificación serializados.
 - `pred_[uuid].joblib` — Artefactos de predicción numérica.
 - `cluster_[uuid].joblib` — Artefactos de agrupamiento.

Anexo H. Manual de Usuario

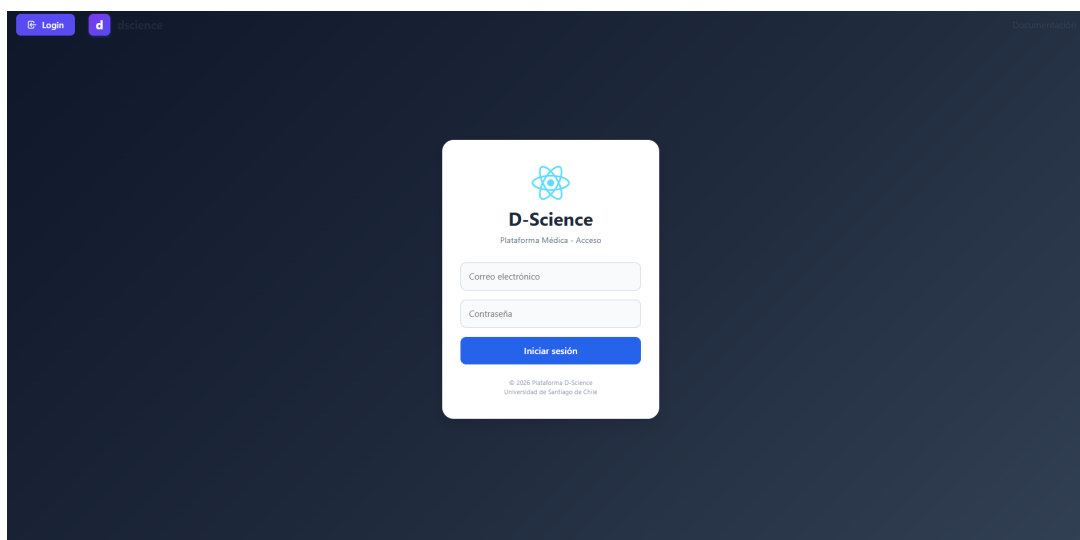


Figura 34. Pantalla de inicio de sesión de D-Science.

H.1. Introducción

El presente manual tiene como propósito proporcionar una guía detallada para el uso de **D-Science**, una plataforma de ciencia de datos orientada a la investigación colaborativa. El sistema permite ejecutar flujos completos de Machine Learning: desde la ingesta de datos crudos hasta la obtención de predicciones mediante modelos entrenados.

H.2. Acceso al sistema

1. Ingrese a la plataforma mediante su navegador.
2. En la pantalla de inicio (Figura 34), introduzca sus credenciales institucionales.
3. Tras autenticarse, accederá al **Laboratorio Virtual**, donde podrá monitorear el

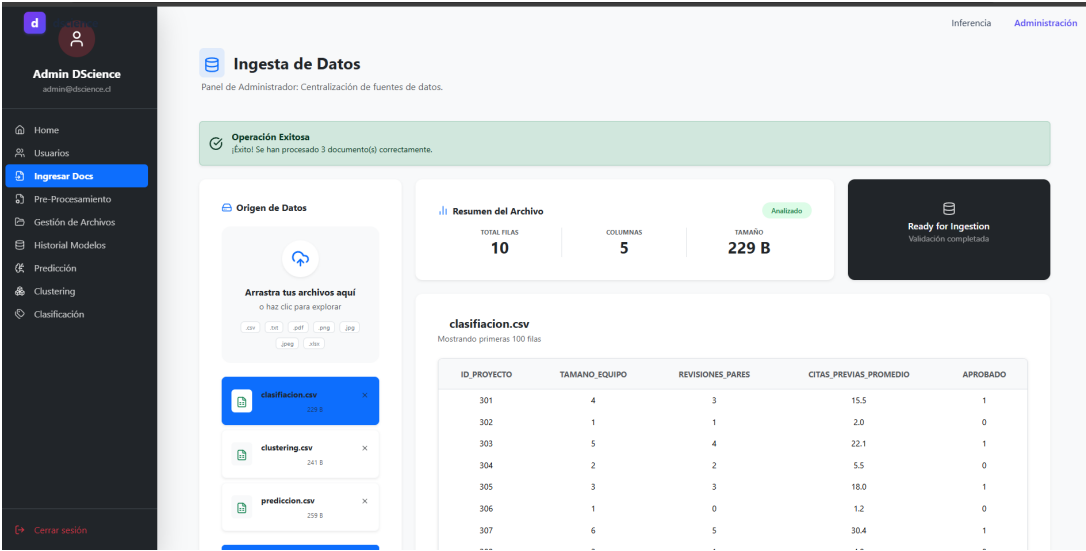


Figura 35. Confirmación de carga exitosa y resumen del dataset.

estado de los servicios y recursos del sistema.

H.3. Flujo de Operación

El sistema ha sido diseñado para guiar al usuario a través de las etapas fundamentales de un proyecto de ciencia de datos.

H.3.1. Ingesta de Datos

El proceso comienza en el módulo **Ingresar Docs**. Aquí, el usuario puede cargar archivos CSV o Excel. El sistema realiza una validación automática, mostrando el número de filas, columnas y una vista previa de la tabla (Figura 35).

H.3.2. Preprocesamiento y Selección de Variables

Una vez cargados los datos, se debe definir la estructura del experimento en el módulo de **Pre-Procesamiento**:

- 1. **Configuración:** Seleccione el documento origen y el tipo de modelo a entrenar.
- 2. **Selección de Variables:** El sistema desplegará las columnas detectadas. El usuario debe marcar cuáles desea usar como *Features (X)* y cuál será el *Target*

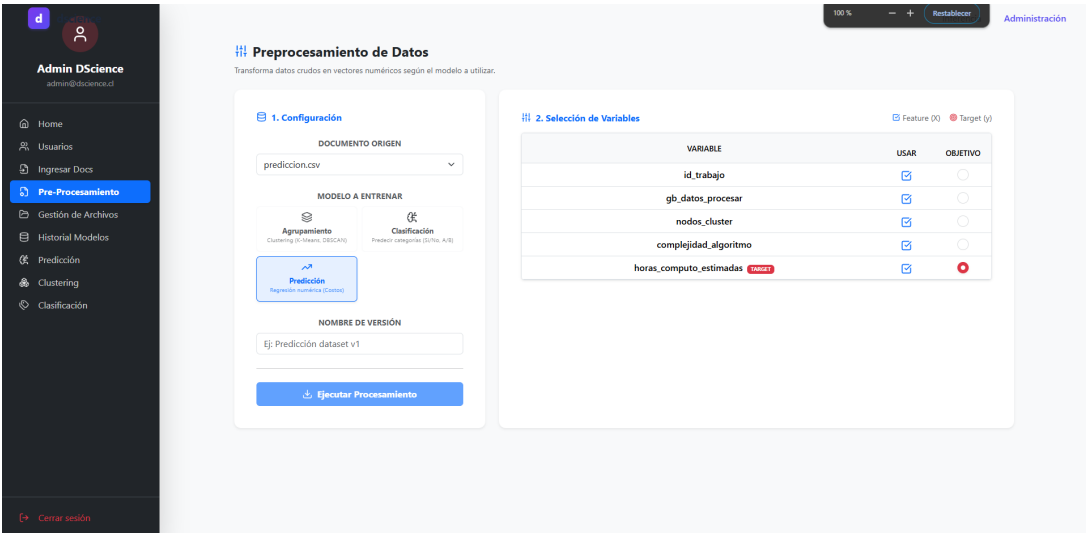


Figura 36. Selección de variables y objetivo (Target).

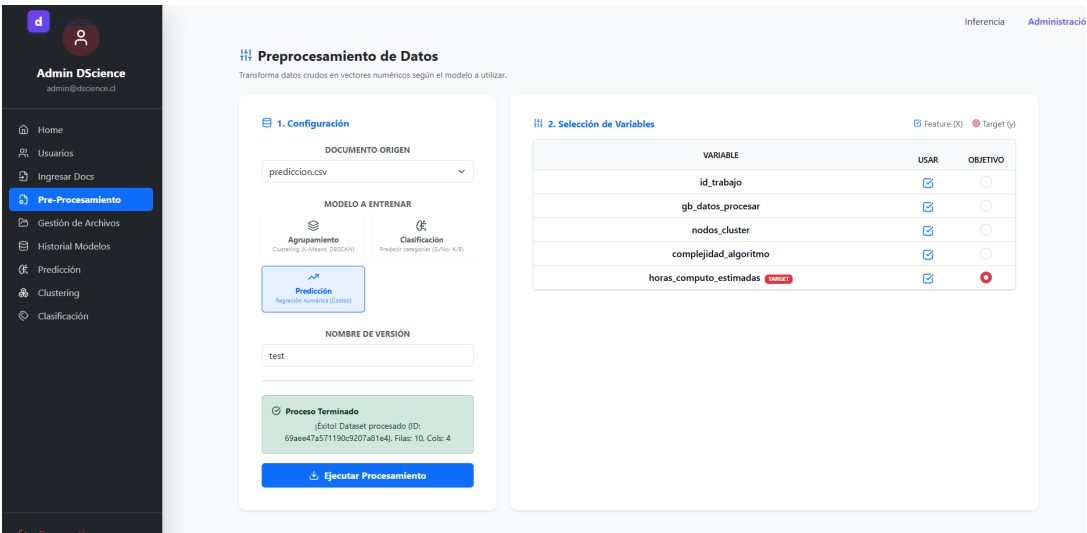


Figura 37. Preprocesamiento exitoso.

(y) (Figura 36).

3. **Ejecución:** Al finalizar, se generará un dataset procesado listo para el entrenamiento (Figura 37).

H.3.3. Entrenamiento de Modelos

En el módulo de **Entrenamiento**, el usuario configura los hiperparámetros del algoritmo. Tras terminar, el sistema muestra las métricas finales (Figura 38).

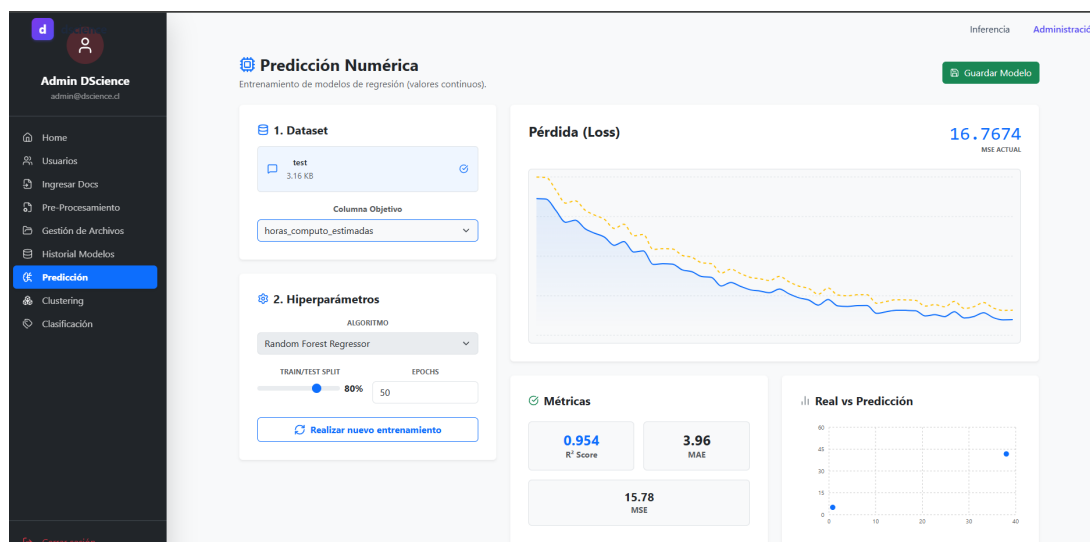


Figura 38. Métricas de desempeño post-entrenamiento.

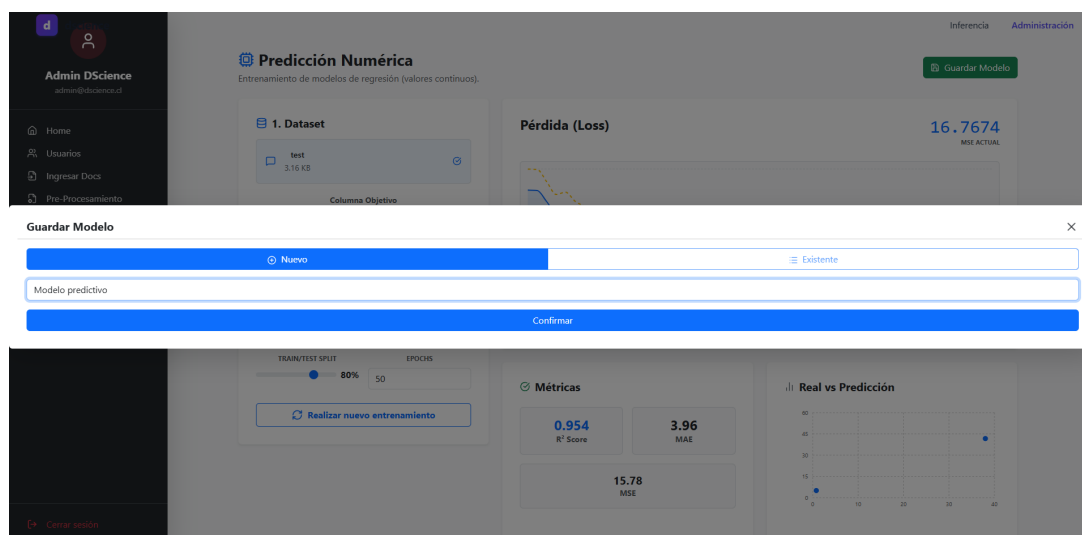


Figura 39. Modal de registro de modelo analítico.

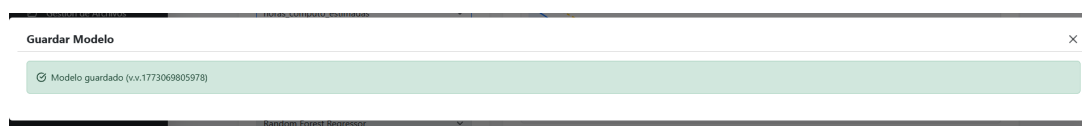


Figura 40. Confirmación de versionado de modelo.

H.3.4. Registro y Serialización del Modelo

Tras evaluar los resultados, el investigador debe decidir si el modelo es apto para su uso. Si las métricas son satisfactorias, se procede a la serialización; de lo contrario, se debe reintentar con nuevos parámetros.

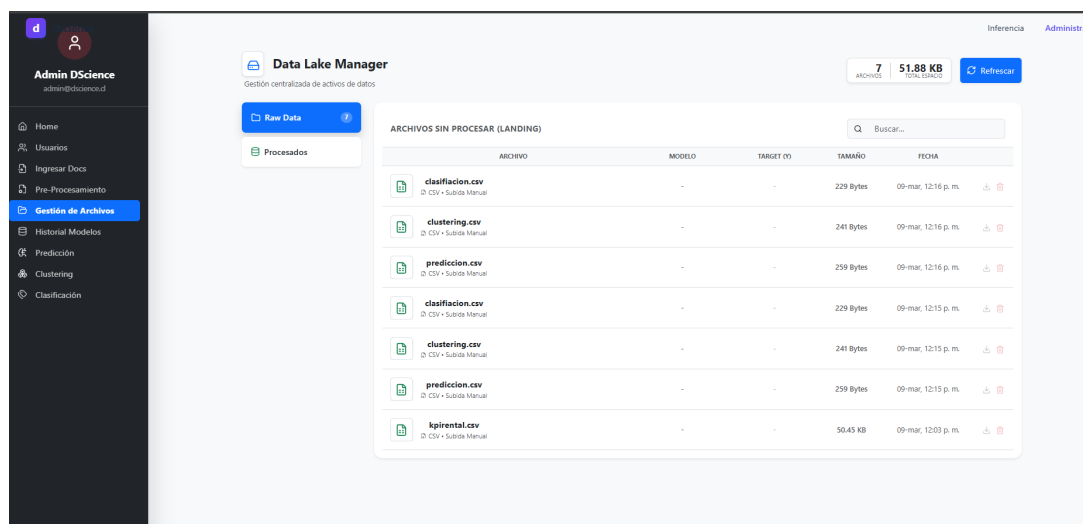


Figura 41. Data Lake Manager para la gestión centralizada de archivos.

1. Haga clic en el botón **Guardar Modelo**.
2. En la ventana emergente, asigne un nombre descriptivo al activo analítico (Figura 39).
3. Una vez confirmado, el sistema emitirá una alerta indicando que el modelo ha sido versionado y guardado correctamente (Figura 40).

H.4. Gestión de Activos y Auditoría

La plataforma garantiza la trazabilidad mediante módulos de gestión centralizada.

H.4.1. Data Lake Manager (Gestión de Archivos)

Desde el módulo **Gestión de Archivos**, el usuario puede administrar el repositorio de activos de datos (Figura 41). Esta vista permite visualizar el inventario de archivos en bruto (*Raw Data*) y procesados, detallando su tamaño y fecha de carga.

H.4.2. Historial de Entrenamientos

Para mantener un control riguroso del ciclo de vida de los modelos, el **Historial de Modelos** registra cada experimento realizado (Figura 42). Aquí se pueden auditar los hiperparámetros utilizados y las métricas obtenidas en cada iteración del investigador.

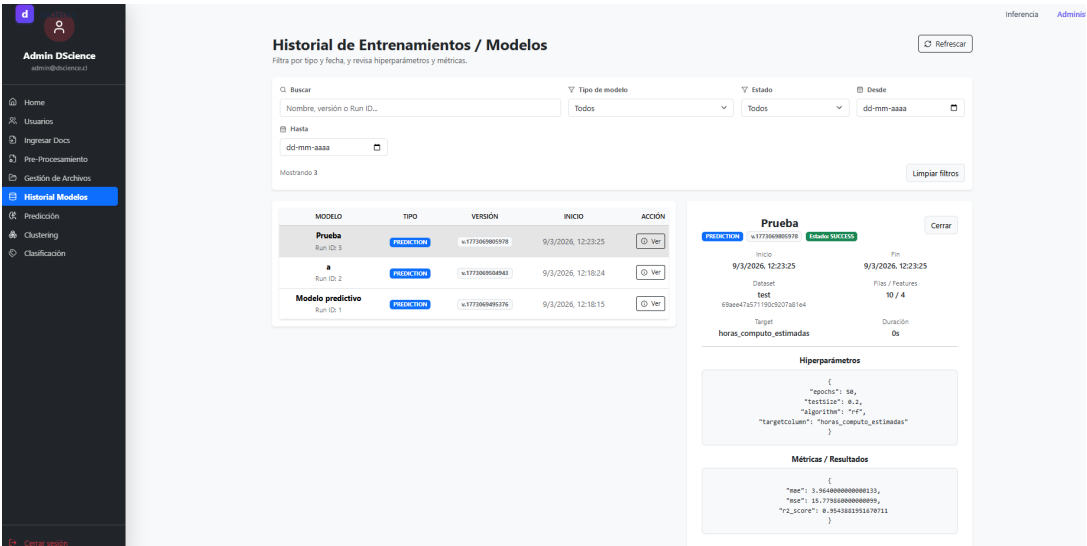


Figura 42. Historial detallado de modelos entrenados y auditoría de parámetros.

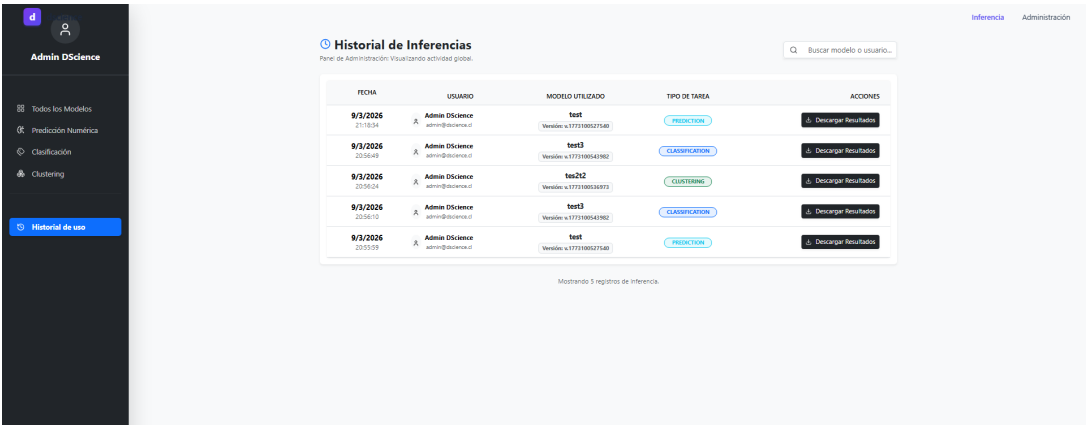


Figura 43. Historial y descarga de inferencias.

H.4.3. Historial de Inferencias

Este módulo permite la recuperación diferida de los resultados analíticos y la auditoría del uso de los modelos (Figura 43).

- 1. **Búsqueda Dinámica:** Utilice la barra de búsqueda para localizar inferencias por nombre de modelo o por el usuario responsable (disponible para Administradores).
- 2. **Recuperación de Activos:** El botón **Descargar Resultados** permite obtener el dataset completo de la predicción en formato Excel o CSV.

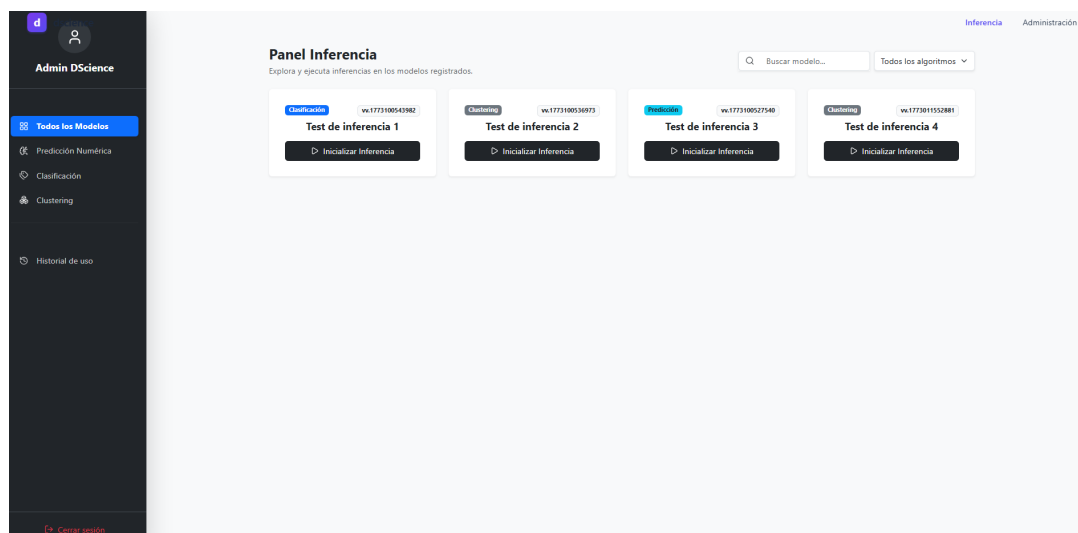


Figura 44. Panel principal de inferencia.

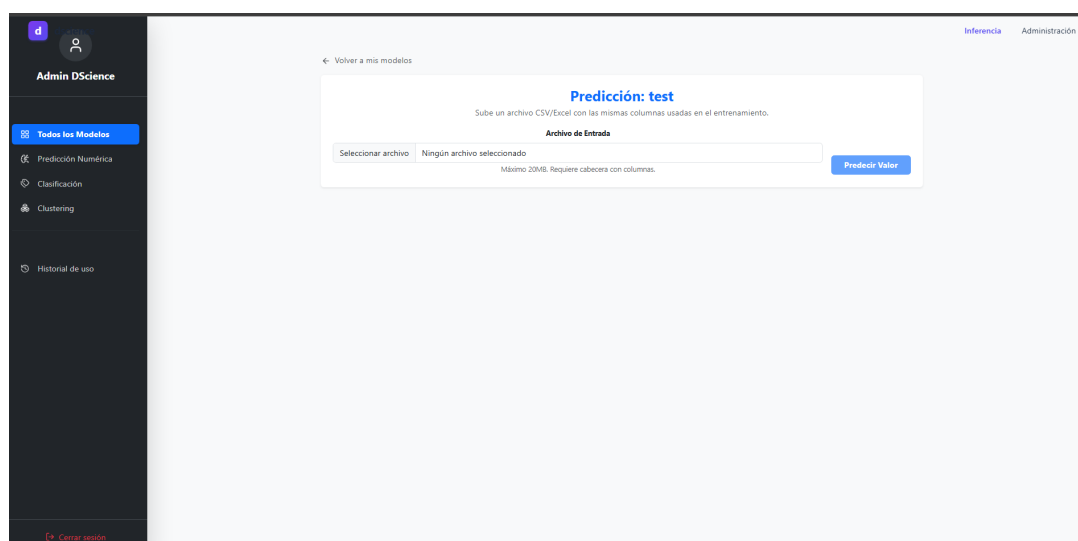


Figura 45. Vista de predicción inferencia.

H.5. Ejecución de Inferencia

Para aplicar un modelo a datos nuevos, se accede al módulo de **Inferencia**.

1. **Selección de Modelo:** En el panel principal, elija el modelo deseado y haga clic en "Iniciar Inferencia"(Figura 44).
2. **Carga de Datos:** Suba el archivo con los nuevos datos a evaluar (Figura 45).
3. **Resultados:** El sistema entregará una tabla con los valores predichos y permitirá exportar los resultados a Excel (Figura 46).

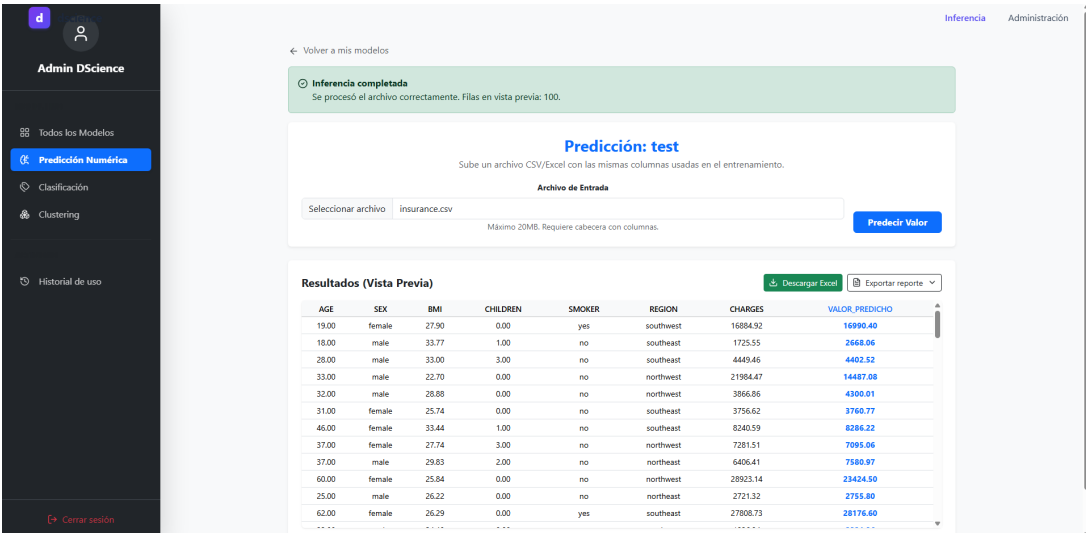


Figura 46. Resultados finales de la inferencia con opción de exportación.

Anexo I. Despliegue en Entorno de Producción (VPS)

Este anexo detalla los procedimientos técnicos, configuraciones de red y comandos operativos utilizados para el aprovisionamiento de la plataforma en un Servidor Privado Virtual (VPS) expuesto a Internet con dominio. Se documentan las soluciones aplicadas para el enrutamiento de tráfico, la monitorización de infraestructura y el mantenimiento continuo.

I.1. Configuración del Proxy Inverso (Nginx)

Para exponer la plataforma de manera segura, el contenedor del frontend integra un servidor Nginx configurado como proxy inverso. Esta capa intercepta el tráfico entrante en el puerto 80 y enruta dinámicamente las peticiones.

Durante el despliegue, se debe asegurar que las directivas de enrutamiento hacia el *API Gateway* no alteren la estructura de la URL original (evitando errores HTTP 404). El Código I.1 detalla la configuración utilizada, destacando el uso de la directiva `proxy_pass` hacia el contenedor `dscience-back`.

Código I.1. Configuración de enrutamiento en `nginx.conf`

```
1 server {
2     listen 80;
3     server_name https://dscience.cedetec.cl;
4
5     # Servir archivos estaticos de React (SPA)
6     location / {
```

```

7      root    /usr/share/nginx/html;
8      index  index.html index.htm;
9      try_files $uri $uri/ /index.html;
10  }
11
12  # Enrutamiento de peticiones al API Gateway (Node.js)
13  location /api/ {
14      proxy_pass http://dscience-back:3000;
15      proxy_http_version 1.1;
16      proxy_set_header Upgrade $http_upgrade;
17      proxy_set_header Connection 'upgrade';
18      proxy_set_header Host $host;
19      proxy_cache_bypass $http_upgrade;
20  }
21 }
```

I.2. Monitorización de Infraestructura (Docker Socket)

Uno de los requerimientos de administración es la capacidad de visualizar el estado de los contenedores y el consumo de recursos (CPU/RAM) directamente desde el *Dashboard* web. Para que el *API Gateway* (Node.js) adquiriera permisos de lectura sobre el motor Docker del servidor anfitrión (VPS), se implementó un mapeo del *socket* de Unix.

Como se muestra en el extracto del Código I.2, el volumen `/var/run/docker.sock` se monta en modo de solo lectura, permitiendo al contenedor ejecutar comandos `docker stats` y `docker ps` de forma nativa sin comprometer la seguridad del host.

Código I.2. Definición del contenedor dscience-back en producción

```

1  back:
2      build:
```



```

3     context: ./back
4     container_name: dscience-back
5     restart: unless-stopped
6     env_file:
7         - .env.prod
8     ports:
9         - "3000:3000"
10    volumes:
11        - models_storage:/app/models_storage
12        - /var/run/docker.sock:/var/run/docker.sock
13    depends_on:
14        postgres:
15            condition: service_healthy
16        mongo:
17            condition: service_started
18        ollama:
19            condition: service_started
20    networks:
21        - dscience-net
22networks:
23    dscience-net:
24        driver: bridge
25volumes:
26    models_storage:

```

I.3. Comandos de Operación y Mantenimiento

El uso del entorno de producción requiere un manejo específico del ciclo de vida de los contenedores, especialmente al momento de aplicar parches o actualizaciones de código fuente. A continuación, se documentan los comandos operativos estándar utilizados en el VPS:

I.3.1. Construcción y Despliegue Inicial

Para levantar la infraestructura completa por primera vez, forzando la compilación de las imágenes a partir del código fuente y ejecutando el proceso en segundo plano (*detached mode*):

```
1 docker compose -f docker-compose.yml up -d --build
```

I.3.2. Actualización en Caliente de Microservicios

En escenarios de mantenimiento donde se actualiza la lógica de un servicio específico (por ejemplo, el *Frontend* o el *Backend*) sin interrumpir la persistencia de las bases de datos, se utiliza la reconstrucción selectiva. El siguiente comando ilustra la actualización del cliente web tras modificar los componentes de React:

```
1 docker compose -f docker-compose.yml up -d --build front
```

I.3.3. Reinicio Forzado y Depuración

Si un contenedor presenta problemas de asignación de volúmenes persistentes (como conflictos con el socket de Docker) o se requiere una limpieza de su estado efímero, se procede a su destrucción y posterior reconstrucción:

```
1 # Eliminar el contenedor específico forzosamente
2 docker compose -f docker-compose.yml rm -sf back
3 # Reconstruir y levantar el contenedor actualizado
4 docker compose -f docker-compose.yml up -d --build back
5 # Monitorear los registros de salida en tiempo real
6 docker logs --tail 50 -f dscience-back
```